

FRI

UNIVERZA V LJUBLJANI | Fakulteta za računalništvo in informatiko

Osnove programiranja

Zbirka rešenih kolokvijev in izpitov

za študijska leta od 2023/2024 dalje

Ana Halužan Vasle

Študijsko leto 2026/2027

Kazalo

O zbirki **5**

Strategija reševanja **6**

1. kolokvij 2025/2026 **7**

1. kolokvij 2024/2025* **14**

1. kolokvij 2023/2024* **33**

2. kolokvij 2025/2026 **53**

2. kolokvij 2024/2025* **62**

2. kolokvij 2023/2024* **78**

1. izpit 2025/2026 **91**

2. izpit 2025/2026 **97**

3. izpit 2025/2026 **105**

1. izpit 2024/2025 **114**

2. izpit 2024/2025 **124**

3. izpit 2024/2025 **133**

1. izpit 2023/2024 **142**

2. izpit 2023/2024 **152**

3. izpit 2023/2024 **161**

O zbirki

Stari kolokviji in izpiti so urejeni po obsegu snovi - v preteklih letih so se pri predmetu pisali kolokviji, pri čemer je 1. kolokvij vseboval snov do vključno funkcij, 2. kolokvij pa celotno snov (podobno kot izpiti).

Rešeni kolokviji/izpiti, ki imajo ob naslovu zvezdico (∗), vsebujejo rešitve z obsežnejšo razlago.

Strategija reševanja

Vsak od nas ima svojo strategijo reševanja nalog. Katero boste ubrali, je odvisno od vas in vašega načina razmišljanja. Za navdih spodaj opisujem pristop, ki se v praksi običajno najbolje obnese.

Najprej vse naloge dobro preberite. Pri nekaterih boste takoj vedeli, kako sprogramirati rešitev; če pa vam pri prvem branju ni povsem jasno, kako se je lotiti, to nalogo preskočite in se vrnite k njej kasneje.

V besedilu naloge si označite ali izpišite pomembne dele. Naredite hitro skico v obliki psevdokode, kamor si zapišete potek kode, dodate drobce, metode ipd. Nato se lotite programiranja. Če se vam zatakne, poskusite rešiti vsaj del naloge - pogosto se rešitev pokaže sproti.

Če niste prepričani, kaj neka funkcija počne, jo ločeno preizkusite v lupini (*IDLE Shell*).

Ko napišete kodo, jo poženite in preverite delovanje na primerih. Če deluje pravilno, poženite še priložene teste. Previdno preglejte, kaj vam testi sporočijo – če test ne gre skozi, se bo na koncu izpisa o neujemajočih izhodih izpisalo tudi kratko sporočilo, kaj je narobe.

Testne primere, uporabljene v avtomatskih testih, si lahko ogledate tako, da odprete mapo `public`, izberete podmapo `input` in kliknete na datoteko s številko testa, ki vas zanima. Datoteke s končnico `.in` lahko odprete z Beležnico ali TextEditom.

1. kolokvij 2025/2026

Nalogo 1 shranite v datoteko z imenom `naloga1.py`, naloge 2-4 pa v datoteko `naloge.py`.

1. naloga

Napišite program, ki organizatorjem fakultetnega polmaratona pomaga pri vnašanju prijav in razporejanju tekačev v ustrezne kategorije (A = 10 km, B = 21 km) glede na tedensko razdaljo (`float`), ki jo v povprečju pretečejo.

Program naj najprej vpraša po imenu tekmovalca, nato pa po pretečeni tedenski razdalji. Če tedenska razdalja:

- znaša manj kot 15 kilometrov, je prijava zavrnjena;
- znaša med 15 in 30 kilometrov, je tekač avtomatsko uvrščen v kategorijo A;
- dosega ali presega 30 kilometrov, se tekač sam odloči, v kateri kategoriji bo tekel.

Program naj vsakič sporoči, v katero kategorijo je bil uvrščen tekač. Če tekač javi kategorijo, ki ne obstaja, naj program pozove po ponovnem vnosu.

Za prijave naj sprašuje tako dolgo, dokler prijavna služba na vprašanje o imenu ne odgovori z "0". Nato program vpraša, ali naj izpiše število tekmovalcev po kategorijah (`int`) ali poimenski seznam tekmovalcev po kategorijah.

Primera delovanja programa:

```
>>> Vnesi ime: Janez
>>> Vnesi tedensko razdaljo v km: 16
Uvrstitev v kategorijo A.
Naslednji tekmovalec.
>>> Vnesi ime: Franc
>>> Vnesi tedensko razdaljo v km: 30
>>> Izberi kategorijo (A, B): B
Uvrstitev v kategorijo B.
Naslednji tekmovalec.
>>> Vnesi ime: 0
Prijave zaključene.
>>> Želite izpis števila tekmovalcev ali seznama tekmovalcev? (s/l): s
Kategorija A: 1; kategorija B: 1
```

```
>>> Vnesi ime: Janez
>>> Vnesi tedensko razdaljo v km: 10
Zavrnjena prijava.
Naslednji tekmovalec.
>>> Vnesi ime: Franc
>>> Vnesi tedensko razdaljo v km: 35
>>> Izberi kategorijo (A, B): C
Neveljavna kategorija! Poskusi znova.
>>> Vnesi ime: Franc
>>> Vnesi tedensko razdaljo v km: 35
>>> Izberi kategorijo (A, B): B
Uvrstitev v kategorijo B.
Naslednji tekmovalec.
>>> Vnesi ime: 0
Prijave zaključene.
>>> Želite izpis števila tekmovalcev ali seznama tekmovalcev? (s/l): l
Kategorija A: []; kategorija B: ['Franc']
```

Rešitev

```python
tedenska_razdalja = 1

a = []
b = []

while True:
    ime = input("Vnesi ime: ")

    if ime == "0":
        print("Prijave zaključene.")
        izpis = input("Želite izpis števila tekmovalcev ali seznama
        ↪  tekmovalcev? (s/i): ").lower()
        break

    tedenska_razdalja = float(input("Vnesi tedensko razdaljo v km: "))

    if 15 <= tedenska_razdalja < 30:
        print("Uvrstitev v kategorijo A.")
        a.append(ime)
    elif tedenska_razdalja < 15:
        print("Zavrnjena prijava.")
    elif tedenska_razdalja >= 30:
        zelena_kat = input("Izberi kategorijo (A, B): ").upper()
        if zelena_kat not in ["A", "B"]:
            print("Neveljavna kategorija! Poskusi znova.")
            continue
        if zelena_kat == "A":
            print("Uvrstitev v kategorijo A.")
            a.append(ime)
        elif zelena_kat == "B":
            print("Uvrstitev v kategorijo B.")
            b.append(ime)

    print("Naslednji tekmovalec.")

if izpis == "s":
    print(f"Kategorija A: {len(a)}; kategorija B: {len(b)}")
else:
    print(f"Kategorija A: {a}; kategorija B: {b}")
```

2. naloga

Napišite funkcijo `povprecni_tempo`, ki kot argumenta sprejme dva seznama. Prvi seznam vsebuje število kilometrov za posamezni tek, drugi seznam pa čas, ki ga je tekač potreboval, da je pretekel to razdaljo. Funkcija naj vrača seznam treh elementov (vsi `float`): najpočasnejši tempo (najvišja številka), povprečni tempo in najhitrejši tempo (najnižja številka). Vsi trije elementi naj bodo zaokroženi na 2 decimalni mesti natančno.

Če se podana seznama ne ujemata v dolžini ali pa je dolžina kateregakoli od njiju manjša od 3, naj funkcija vrne prazen seznam.

Primera delovanja:

```
>>> kms = [5, 10, 21.1, 42.2]
>>> mins = [25, 50, 110, 240]
>>> povprecni_tempo(kms, mins)
[5.69, 5.23, 5.0]
```

```
>>> kms = [5, 10, 21.1]
>>> mins = [25, 50, 110, 240]
>>> povprecni_tempo(kms, mins)
[]
```

Rešitev

```
1 def povprecni_tempo(km, minute):
2     tempi = []
3     if len(km) != len(minute) or len(km)<3 or len(minute)<3:
4         return []
5 
6     for i in range(len(km)):
7         tempo = round(minute[i] / km[i],2)
8         tempi.append(tempo)
9     return [max(tempi), round(sum(tempi) / len(tempi),2), min(tempi)]
```

3. naloga

Napišite funkcijo `napoved`, ki kot argumenta sprejme tempo (`float`), podan v min/km, in kategorijo (`str`). Funkcija naj vrača niz oblike `"hh:mm:ss"` (`str`), pri čemer `hh` označuje ure, `mm` minute, `ss` pa sekunde, ki ponazarjajo čas, v katerem z danim tempom tekač preteče razdaljo izbrane kategorije.

Kategorije in pripadajoče razdalje so sledeče:

- `A` označuje 10 km,
- `B` 21.0975 km in
- `C` 42.195 km.

Če je podana katerakoli druga kategorija, naj funkcija vrne `"Neveljavna kategorija!"`.

Skupni čas izračunate po formuli: $tempo{*}razdalja$ in je podan v minutah, nato pa ga z deljenjem pretvorite v ure, minute in sekunde po naslednjih formulah:

- $ure = int(skupni_cas//60)$
- $minute = int(skupni_cas\%60)$
- $sekunde = int(((skupni_cas - int(skupni_cas)) * 60)$

Primeri delovanja:

```
>>> napoved(6.62, "A")
'01:06:12'
>>> napoved(6.62, "B")
'02:19:39'
>>> napoved(6.62, "D")
'Neveljavna kategorija!"
```

Rešitev

```python
def napoved(tempo, kategorija):
    if kategorija == "A":
        razdalja = 10
    elif kategorija == "B":
        razdalja = 21.0975
    elif kategorija == "C":
        razdalja = 42.195
    else:
        return "Neveljavna kategorija!"

    skupni_cas_min = tempo * razdalja
    ure = int(skupni_cas_min // 60)
    minute = int(skupni_cas_min % 60)
    sekunde = int((skupni_cas_min - int(skupni_cas_min)) * 60)

    return f"{ure:02}:{minute:02}:{sekunde:02}"
```

4. naloga

Napišite funkcijo `priprave`, ki sprejme dva argumenta v obliki `float`. Prvi argument je tekačev cilj, drugi pa začetna razdalja. Funkcija naj pripravi izračun števila treningov, ki jih tekač potrebuje za podaljšanje teka iz začetne razdalje na ciljno razdaljo, če na vsakem treningu razdaljo poveča za 10 % - torej: na vsakem treningu je razdalja 10 % daljša od prejšnjega. Funkcija naj vrne število treningov, ki jih tekač potrebuje za dosego cilja. V primeru, da je začetna razdalja večja od cilja, naj funkcija javi `"Cilj ste že dosegli."`.

Namig: $razdalja* = 1.1$

Primera delovanja:

```
>>> priprave(11,6)
7
>>> priprave(10,9)
2
```

Rešitev

```
1  def priprave(cilj,razdalja):
2      if razdalja > cilj:
3          return "Cilj ste že dosegli."
4
5      treningi = 0
6
7      while razdalja < cilj:
8          razdalja *= 1.1
9          treningi += 1
10
11     return treningi
```

1. kolokvij 2024/2025*

Nalogo 1 shranite v datoteko z imenom `naloga1.py`, naloge 2-4 pa v datoteko `naloge.py`.

1. naloga

Napišite program, ki sprašuje za geslo toliko časa, dokler ni to skladno z naslednjimi smernicami:

- geslo mora biti dolgo najmanj 12 znakov,
- vsebovati mora najmanj enega od znakov ,, ., !, ? ali _,
- vsebovati mora vsaj 1 številko,

Vsakič, ko vpisano geslo ne ustreza kateri od smernic, naj program to izpiše in uporabnika pozove k vnosu novega, s smernicami skladnega gesla. Za posamezen kriterij, ki ga geslo ne izpolnjuje, naj program izpiše sledeče:

- v primeru prekratkega gesla:
  `Geslo ni dovolj dolgo. Poskusite ponovno.`,
- če ne vsebuje omenjenih posebnih znakov:
  `Geslo ne vsebuje posebnega znaka. Poskusite ponovno.`,
- v primeru, da ne vsebuje številk:
  `Geslo ne vsebuje številk. Poskusite ponovno.`.

Ko je geslo skladno z vsemi smernicami, naj program izpiše `Geslo je ustrezno.`.

Primer delovanja:

```
Vnesi geslo: prviposkus
Geslo ni dovolj dolgo. Poskusite ponovno.
Geslo ne vsebuje posebnega znaka. Poskusite ponovno.
Geslo ne vsebuje številk. Poskusite ponovno.
Vnesi geslo: drugi_poskus
Geslo ne vsebuje številk. Poskusite ponovno.
Vnesi geslo: tr3tji_poskus
Geslo je ustrezno.
```

Rešitev

```
posebni_znaki = ['.',',','!','?','_']
stevilke = ['0','1','2','3','4','5','6','7','8','9']

while True:
    geslo = input("Vnesi geslo: ")
    ok = True

    if len(geslo) < 12:
        ok = False
        print("Geslo ni dovolj dolgo. Poskusite ponovno.")

    for znak in posebni_znaki:
        if znak in geslo:
            break
    else:
        ok = False
        print("Geslo ne vsebuje posebnega znaka. Poskusite ponovno.")

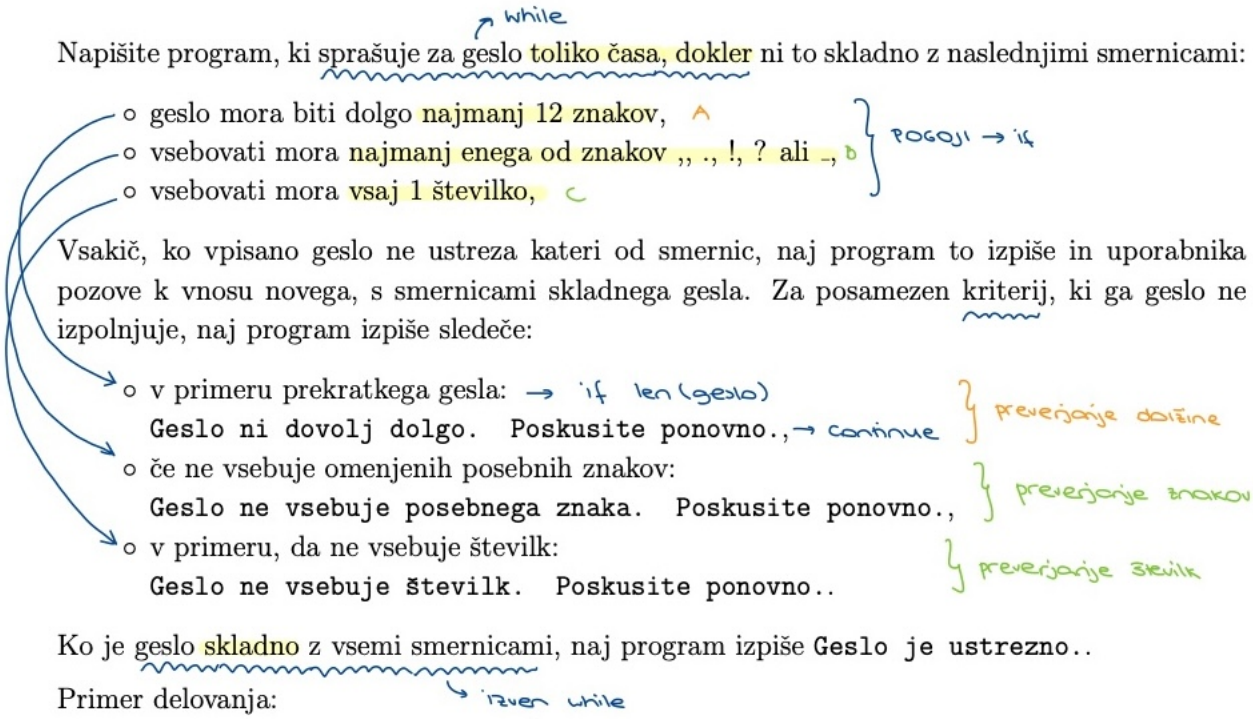
     for stevilo in stevilke:
        if stevilo in geslo:
            break
    else:
        ok = False
        print("Geslo ne vsebuje številk. Poskusite ponovno.")

    if ok:
        break

print("Geslo je ustrezno.")
```

Rešitev z razlago

Ob branju navodila si označimo pomembne dele:



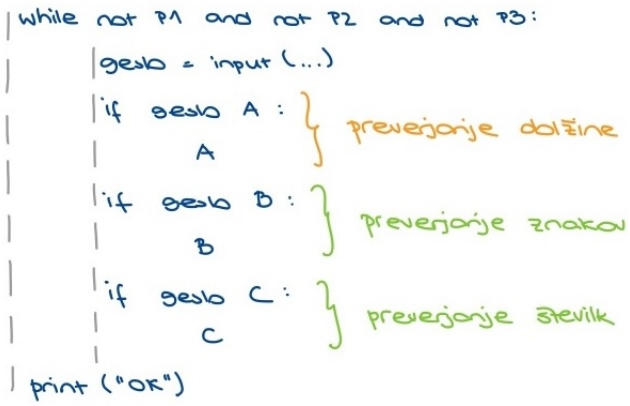

```
1 Vnesi geslo: prviposkus
2 Geslo ni dovolj dolgo. Poskusite ponovno.
3 Geslo ne vsebuje posebnega znaka. Poskusite ponovno.
4 Geslo ne vsebuje številk. Poskusite ponovno.
5 Vnesi geslo: drugi_poskus
6 Geslo ne vsebuje številk. Poskusite ponovno.
7 Vnesi geslo: tr3tji_poskus
8 Geslo je ustrezno.
```

3× if, NE if-elif

Naloga od nas zahteva uporabo zanke *while*, saj želimo uporabnika za vnos gesla spraševati vse dokler to ne bo skladno z našimi smernicami. Ker ne vemo, koliko poskusov bo uporabnik potreboval za vnos ustreznega gesla, naloge ne moremo rešiti z zanko *for*.

Navedeni so trije pogoji ustreznosti gesla: dolžina (A), vsebnost posebnih znakov (B) in vsebnost vsaj enega števila (C). Ker želimo, da se izpišejo vse pomanjkljivosti gesla, moramo uporabiti tri zaporedne stavke - *if A, if B, if C* in ne kombinacije *if A, elif B, elif C*, saj bi slednja povzročila, da se pri več pomanjkljivostih izpiše le prva, ki jo program najde, ne pa tudi ostali dve.

Pred programiranjem si lahko zapišemo še grobo psevdokodo, ki nas bo vodila skozi potek programa:



Najprej zapišimo ogrodje naše kode:

```
dolzina = False
znaki = False
st = False

while not dolzina or not znaki or not stevilke:
    geslo = input("Vnesi geslo: ")
    dolzina = False
    znaki = False
    st = False

    # preverjanje dolžine (A)

    # preverjanje znakov (B)

    # preverjanje števil (C)

    if not dolzina:
        print("Geslo ni dovolj dolgo. Poskusite ponovno.")
    if not znaki:
        print("Geslo ne vsebuje posebnega znaka. Poskusite ponovno.")
    if not st:
        print("Geslo ne vsebuje števil. Poskusite ponovno.")

print("Geslo je ustrezno.")
```

V tem primeru smo se odločili za uporabo treh pomožnih spremenljivk, v katere shranjujemo vrednosti `True` in `False` glede na to, ali so pogoji A, B in C izpolnjeni. Kadar se odločimo za tak način preverjanja, moramo poskrbeti, da se pri vsaki novi iteraciji zanke pogoji ponastavijo na izhodiščne vrednosti.

Preverjanje dolžine (A) preprosto izvedemo s sledečo kodo, ki jo vstavimo v 11. vrstico:

```
if len(geslo) >= 12:
    dolzina = True
else:
    dolzina = False
```

Preverjanje znakov (B) in preverjanje številk (C) bomo izvedli na podoben način - v obeh primerih namreč preverjamo prisotnost več znakov (spomnite se na nalogo Črka v nizu - 1. naloga z 2. vaj). To lahko storimo na več načinov:

1) uporaba kombinacije *if-elif*:
```
if "." in geslo:
    znaki = True
elif "," in geslo:
    znaki = True
# in tako naprej za vse znake
else:
    znaki = False
```

2) uporaba le enega stavka *if* z Boolovimi operatorji:
```
if "." in geslo or "," in geslo or "!" in geslo or "?" in geslo or "_" in
↪  geslo:
    znaki = True
else:
    znaki = False
```

3) uporaba kombinacija *for-else*:
```
for znak in posebni_znaki:
    if znak in geslo:
        znaki = True
        break
else:
    znaki = False
```
pri čemer je v spremenljivki `posebni_znaki` lahko bodisi seznam `['.',',','!','?','_']` ali niz `".,!?_"`.

Enako storimo za preverjanje številk, pri tem pa moramo paziti, da številke zapišemo kot niz (`str`) in ne kot števila (`int`), saj bodo v geslu del niza. Če se odločimo za uporabo zanke *for*, si pripravimo še seznam `stevilke = ['0','1','2','3','4','5','6','7','8','9']` ali niz `stevilke = '0123456789'` števil.

Rešitev z uporabo 2. načina preverjanja vsebnosti znakov je:

```
dolzina = False
znaki = False
st = False

while not dolzina or not znaki or not stevilke:
    geslo = input("Vnesi geslo: ")
    dolzina = False
    znaki = False
    st = False

    if len(geslo) >= 12:
        dolzina = True
    else:
        dolzina = False

    if "." in geslo or "," in geslo or "!" in geslo or "?" in geslo or "_" in
    ↪ geslo:
            znaki = True
    else:
        znaki = False

    if "1" in geslo or "2" in geslo or "3" in geslo or "4" in geslo or "5" in
    ↪ geslo or "6" in geslo or "7" in geslo or "8" in geslo or "9" in geslo:
        st = True
    else:
        st = False

    if not dolzina:
        print("Geslo ni dovolj dolgo. Poskusite ponovno.")
    if not znaki:
        print("Geslo ne vsebuje posebnega znaka. Poskusite ponovno.")
    if not st:
        print("Geslo ne vsebuje številk. Poskusite ponovno.")

print("Geslo je ustrezno.")
```

Učinkovitejša rešitev vsebuje preverjanje z zankama *for*:

```
posebni_znaki = ['.',',','!','?','_']
stevilke = ['0','1','2','3','4','5','6','7','8','9']

dolzina = False
znaki = False
st = False

while not dolzina or not znaki or not stevilke:
    geslo = input("Vnesi geslo: ")
    dolzina = False
    znaki = False
    st = False

    if len(geslo) >= 12:
        dolzina = True
    else:
        dolzina = False

    for znak in posebni_znaki:
        if znak in geslo:
            znaki = True
            break
    else:
        znaki = False

    for stevilka in stevilke:
        if stevilka in geslo:
            st = True
            break
    else:
        st = False

    if not dolzina:
        print("Geslo ni dovolj dolgo. Poskusite ponovno.")
    if not znaki:
        print("Geslo ne vsebuje posebnega znaka. Poskusite ponovno.")
    if not st:
        print("Geslo ne vsebuje številk. Poskusite ponovno.")

print("Geslo je ustrezno.")
```

Še krajša rešitev, kjer imamo le eno pomožno spremenljivko, je:

```
posebni_znaki = ['.',',','!','?','_']
stevilke = ['0','1','2','3','4','5','6','7','8','9']

while True:
    geslo = input("Vnesi geslo: ")
    ok = True

    if len(geslo) < 12:
        ok = False
        print("Geslo ni dovolj dolgo. Poskusite ponovno.")

    for znak in posebni_znaki:
        if znak in geslo:
            break
    else:
        ok = False
        print("Geslo ne vsebuje posebnega znaka. Poskusite ponovno.")

     for stevilo in stevilke:
        if stevilo in geslo:
            break
    else:
        ok = False
        print("Geslo ne vsebuje številk. Poskusite ponovno.")

    if ok:
        break

print("Geslo je ustrezno.")
```

2. naloga

Napišite funkcijo `vault`, ki vrača seznam (`list`) kombinacij uporabniških imen s terkami oblike `(uporabnisko_ime, geslo)`. Pri tem `uporabnisko_ime` predstavlja posamezno uporabniško ime, `geslo` pa geslo. Funkcija kot argumenta prejme seznam uporabniških imen zapisanih kot niz (`str`) in seznam gesel zapisanih kot niz (`str`). Kombinacije uporabniških imen in gesel naj funkcija generira z združevanjem istoležnih nizov podanih seznamov.

Funkcija naj vrne seznam terk oblike `(uporabnisko_ime, geslo)`, v katerem se posamezno uporabniško ime pojavi največ enkrat. V primeru, da se enako uporabniško ime pojavi večkrat, naj funkcija temu imenu priredi geslo, ki skupaj z imenom nastopa ob njegovi prvi pojavitvi v seznamu.

V primeru, da podana seznama nista enake dolžine, naj funkcija generira kombinacije, dokler ne pride do konca krajšega seznama.

Prvi primer delovanja:

```
>>> usernames1 = ["ShadowWalker", "CrimsonFang", "MysticWarden", "IronClad23",
↪  "FrostNova"]
>>> passwords1 = ["p@ssW0rd!", "Dr4g0n$cale", "Phant0m123", "C0ldBl@ze",
↪  "R0gueWolf"]
>>> vault(usernames1, passwords1)
[('ShadowWalker', 'p@ssW0rd!'), ('CrimsonFang', 'Dr4g0n$cale'),
↪  ('MysticWarden', 'Phant0m123'), ('IronClad23', 'C0ldBl@ze'), ('FrostNova',
↪  'R0gueWolf')]
```

Drugi primer delovanja:

```
>>> usernames2 = ["LunarEcho", "BlazeHawk", "VoidHunter", "CrystalShade",
↪  "NebulaFox"]
>>> passwords2 = ["St4rBurst!", "Gh0stTrail", "FireFl@re"]
>>> vault(usernames2, passwords2)
[('LunarEcho', 'St4rBurst!'), ('BlazeHawk', 'Gh0stTrail'), ('VoidHunter',
↪  'FireFl@re')]
```

Tretji primer delovanja:

```
>>> usernames3 = ["AstralBlaze", "DarkHorizon", "StormFang", "DarkHorizon",
↪  "SolarShade"]
>>> passwords3 = ["Tw1l1ght!", "R@zorEdge", "FrostByte", "N1ghtFall",
↪  "Fl@meWing"]
>>> vault(usernames3, passwords3)
[('AstralBlaze', 'Tw1l1ght!'), ('DarkHorizon', 'R@zorEdge'), ('StormFang',
↪  'FrostByte'), ('SolarShade', 'Fl@meWing')]
```

Rešitev

```python
def vault(imena, gesla):

    kombinacije = []
    imena2 = []

    for ime, geslo in zip(imena, gesla):
        if ime not in imena2:
            kombinacije.append((ime, geslo))
        imena2.append(ime)

    return kombinacije
```

Rešitev z razlago

Nalogo lahko rešimo z uporabo indeksiranja ali pa z uporabo funkcije `zip`. Najprej si poglejmo rešitev pri uporabi indeksiranja:

```
1  def vault(imena, gesla):
2
3      kombinacije = []
4      imena2 = []
5
6      if len(imena) < len(gesla):
7          dolzina = len(imena)
8      else:
9          dolzina = len(gesla)
10
11     for i in range(0, dolzina):
12         if imena[i] not in imena2:
13             kombinacije.append((imena[i], gesla[i]))
14         imena2.append(imena[i])
15
16     return kombinacije
```

V navodilih piše, da kombinacije uporabniških imen in gesel generiramo le do konca krajšega seznama, zato najprej določimo dolžino krajšega seznama. Pripravili smo si še pomožen seznam `imena2`, v katerega bomo shranili imena, na katera smo že naleteli - s tem se izognemo, da bi istemu uporabniškemu imenu priredili dve gesli. Če se torej uporabniško ime pojavi v tem seznamu, mu pripadajočega gesla ne pripišemo, saj smo to uporabniško ime z geslom dodali ob njegovi prvi pojavitvi. Ker uporabljamo zanko *for* v kombinaciji s pogojnim stavkom *if*, se bo preskočilo tudi geslo na tem indeksu, s čimer bomo zagotovili pravilno kombiniranje.

V seznam kombinacije shranjujemo terke, ki vsebujejo istoležne elemente s seznamov `imena` in `gesla`.

Če se odločimo za uporabo funkcije `zip`, nam ni treba preverjati, kateri seznam je krajši, saj bo funkcija sama združevala elemente do konca krajšega seznama. Pri vračanju moramo paziti, da pred `zip` zapišemo še `list`, s čimer tvorimo seznam terk.

```python
def vault(imena, gesla):

    kombinacije = []
    imena2 = []

    for ime, geslo in zip(imena,gesla):
        if ime not in imena2:
            kombinacije.append((ime, geslo))
        imena2.append(ime)

    return kombinacije
```

3. naloga

Napišite funkcijo `slaba_praksa`, ki sprejme seznam terk oblike `(uporabnisko_ime, geslo)`. Pri tem `uporabnisko_ime` predstavlja posamezno uporabniško ime, `geslo` pa geslo.

Funkcija naj vrne seznam (`list`) imen uporabnikov, katerih ime se pojavlja v njihovem geslu. Posamezno ime uporabnika naj se v tem seznamu pojavi največ enkrat.

Prvi primer delovanja:

```
>>> kombinacije1 = [("pixelPioneer", "orbit2024"),
 ("quantumQuokka", "waveRider!"),
 ("nebulaNinja", "matrix42"),
 ("codeCrafter", "stellarPass$codeCrafter"),
 ("syntaxSage", "quantumLeap!"),
 ("Stellar", "quantumStellarSphinxLeap!"),
 ("byteBandit", "cosmicEcho!")]
 >>> slaba_praksa(kombinacije1)
 ['codeCrafter', 'Stellar']
```

Drugi primer delovanja:

```
>>> kombinacije2 = [('ShadowWalker', 'p@ssW0rd!'),
('CrimsonFang', 'Dr4g0n$cale'),
('MysticWarden', 'Phant0m123'),
('IronClad23', 'C0ldBl@ze'),
('FrostNova', 'R0gueWolf')]
>>> slaba_praksa(kombinacije2)
[]
```

Rešitev

```
def slaba_praksa(seznam):
    seznam2 = []

    for ime, geslo in seznam:
        if ime in geslo and ime not in seznam2:
            seznam2.append(ime)

    return seznam2
```

Rešitev z razlago

Za vsako podano terko moramo preveriti, ali se uporabniško ime pojavi v geslu. Ponovno se spomnimo na nalogo Črka v nizu (1. naloga z 2. vaj) - v tem primeru ne iščemo ujemanja le ene črke, temveč celotnega niza:

```
if ime in geslo:
```

Kakor v prejšnji nalogi, moramo tudi tukaj poskrbeti, da se ime posameznega uporabnika v končnem seznamu pojavi le enkrat. Uberemo enako strategijo kot prej: pripravimo si pomožni seznam, v katerega bomo shranjevali že videna imena, ki so hkrati uporabljala ime v geslu:

```
if ime in geslo and ime not in seznam2:
```

Rešitev je:

```
1 def slaba_praksa(seznam):
2     seznam2 = []
3 
4     for ime, geslo in seznam:
5         if ime in geslo and ime not in seznam2:
6             seznam2.append(ime)
7 
8     return seznam2
```

Reševanje lahko poenostavimo z uporabo množic, ki pa jih obravnamo kasneje v semestru. Kljub temu prilagam rešitev:

```
1 def slaba_praksa(seznam):
2     seznam2 = set()
3 
4     for ime, geslo in seznam:
5         if ime in geslo:
6             seznam2.add(ime)
7 
8     return list(seznam2)
```

4. naloga

Napiši funkcijo `generator_gesel`, ki sprejme dva argumenta:

- niz (`str`) z znaki, ki naj se uporabijo pri generiranju gesla;
- seznam (`list`) števil.

Funkcija naj deluje na sledeči način:

- iz seznama števil naj jemlje števila od začetka proti koncu;
- vsako število v seznamu naj predstavlja indeks v podanem nizu z znaki;
- na podlagi podanih indeksov iz niza pridobi ustrezne znake in jih združi v končno geslo.

Funkcija naj vrne tako zgenerirano geslo zapisano kot niz (`str`). V primeru, da katerokoli število v seznamu števil presega obseg niza (število je večje ali enako dolžini niza) ali pa sta bodisi niz bodisi seznam prazna, naj funkcija vrne niz `"Neveljavni podatki!"`.

Prvi primer delovanja:

```
>>> niz1 = "nA7!x#G$Kd2L"
>>> sez1 = [0,7,4,10,11,2]
>>> generator_gesel(niz1,sez1)
n$x2L7
```

Drugi primer delovanja:

```
>>> niz2 = "9Z@c5&lQw*Rp8!Vm"
>>> sez2 = [5, 12, 3, 16, 8, 0, 17, 11, 4, 6]
>>> generator_gesel(niz2, sez2)
Neveljavni podatki!
```

Rešitev

```python
def generator_gesel(kljuc, seznam):
    geslo = ""

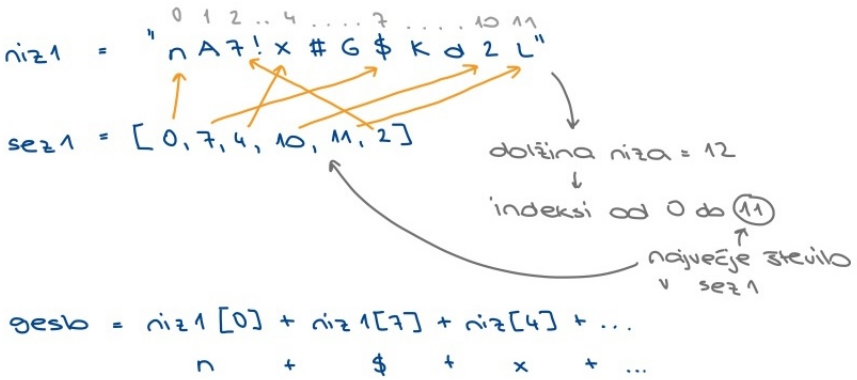
    if len(kljuc) == 0 or len(seznam) == 0:
        return "Neveljavni podatki!"

    for element in seznam:
        if element >= len(kljuc):
            return "Neveljavni podatki!"
        geslo += kljuc[element]

    return geslo
```

Rešitev z razlago

Na podanem primeru si najprej skicirajmo, kaj mora funkcija početi:



V seznamu imamo zbrano zaporedje indeksov, ki določajo črke iz niza za tvorbo gesla. Pri pisanju kode moramo najprej preveriti, ali so indeksi veljavni - največji indeks v seznamu ne sme preseči dolžine niza. Hkrati niz in seznam ne smeta biti prazna, kar moramo preveriti pred preverjanjem veljavnosti indeksa, sicer bo klic funkcije `max` nad praznim seznamom vrnil napako.

```
if seznam == [] or niz == "":
    return "Neveljavni podatki!"
if max(seznam) >= len(niz):
    return "Neveljavni podatki!"
```

Sledi del kode, v katerem bomo generirali geslo - z zanko *for* se pomikamo po seznamu in v geslo dodajamo element, ki se nahaja na tistem indeksu:

```
for indeks in seznam:
    geslo += niz[indeks]
```

Zgornja izseka povežemo v funkcijo:

```
1  def generator_gesel(niz, seznam):
2      geslo = ""
3
4      if seznam == [] or niz == "":
5          return "Neveljavni podatki!"
6      if max(seznam) >= len(niz):
7          return "Neveljavni podatki!"
8
9      for indeks in seznam:
10         geslo += niz[indeks]
11
12     return geslo
```

Preverjanje veljavnosti indeksa lahko izvedemo tudi znotraj zanke *for*, pri čemer se koda glasi:

```
def generator_gesel(niz, seznam):
    geslo = ""

    if len(niz) == 0 or len(seznam) == 0:
        return "Neveljavni podatki!"

    for indeks in seznam:
        if element >= len(niz):
            return "Neveljavni podatki!"
        geslo += niz[indeks]

    return geslo
```

1. kolokvij 2023/2024*

Nalogo 1 shranite v datoteko z imenom `naloga1.py`, naloge 2-4 pa v datoteko `naloge.py`.

1. naloga

Napiši program, ki mu uporabnik poda en znak naenkrat in na podlagi vnešenih znakov sestavlja zaporedje nukleotidov. Pri tem naj program upošteva sledeče omejitve:

- Uporabnik lahko vnese le en znak naenkrat, pri čemer mora biti vneseni znak enak bodisi `C`, `T`, `A` ali `G` (DNA) bodisi `C`, `U`, `A` ali `G` (RNA). Če uporabnik vnese znak, ki ne ustreza tem pogojem, naj program izpiše `Neveljaven nukleotid!`.

- Program naj uporabniku v istem zaporedju ne dovoli vnašati znakov `T` in `U` (glej sledeči alineji):
  - V primeru, da je uporabnik predhodno vnesel `T`, kasneje pa `U`, naj program izpiše `DNA zaporedje ne more vsebovati baze U!`.
  - V primeru, da je uporabnik predhodno vnesel `U`, kasneje pa `T`, naj program izpiše `RNA zaporedje ne more vsebovati baze T!`.

- Program naj prekine vnašanje, ko uporabnik vnese stop kodon (`TAG`, `TGA` ali `TAA` v primeru DNA zaporedja oziroma `UAG`, `UGA` ali `UAA` v primeru RNA zaporedja).

Program naj na koncu izpiše celotno zaporedje in dolžino tega zaporedja (glej primere spodaj). Izpiše naj tudi tip zaporedja glede na vsebovanost nukleotida `T` oziroma `U`.

Primeri delovanja programa:

Primer 1:

```
Vnesi nukleotid: A
Vnesi nukleotid: T
Vnesi nukleotid: G
Vnesi nukleotid: C
Vnesi nukleotid: a
Neveljaven nukleotid!
Vnesi nukleotid: AA
Neveljaven nukleotid!
Vnesi nukleotid: n
Neveljaven nukleotid!
Vnesi nukleotid: T
Vnesi nukleotid: A
Vnesi nukleotid: G
Vnesel si DNA zaporedje ATGCTAG dolžine 7
```

Primer 2:

```
Vnesi nukleotid: T
Vnesi nukleotid: T
Vnesi nukleotid: A
Vnesi nukleotid: U
DNA zaporedje ne more vsebovati baze U!
Vnesi nukleotid: G
Vnesel si DNA zaporedje TTAG dolžine 4
```

Primer 3:

```
Vnesi nukleotid: U
Vnesi nukleotid: U
Vnesi nukleotid: A
Vnesi nukleotid: T
RNA zaporedje ne more vsebovati baze T!
Vnesi nukleotid: G
Vnesel si RNA zaporedje UUAG dolžine 4
```

Rešitev

```
zaporedje = ""

while zaporedje[-3:] not in ["TAG", "TGA","TAA","UAG","UGA","UAA"]:
    nt = input("Vnesi nukleotid: ")
    if nt not in "ATGCU":
        print("Neveljaven nukleotid!")
    elif "U" in zaporedje and nt == "T":
        print("RNA zaporedje ne more vsebovati baze T!")
    elif "T" in zaporedje and nt == "U":
        print("DNA zaporedje ne more vsebovati baze U!")
    else:
        zaporedje += nt

if "T" in zaporedje:
    zap = "DNA"
elif "U" in zaporedje:
    zap = "RNA"

print(f"Vnesel si {zap} zaporedje {zaporedje} dolžine {len(zaporedje)}")
```

Rešitev z razlago

Najprej poglejmo, kaj navodilo zahteva od nas:

Napiši program, ki mu uporabnik poda en znak naenkrat in na podlagi vnešenih znakov sestavlja zaporedje nukleotidov. Pri tem naj program upošteva sledeče omejitve:

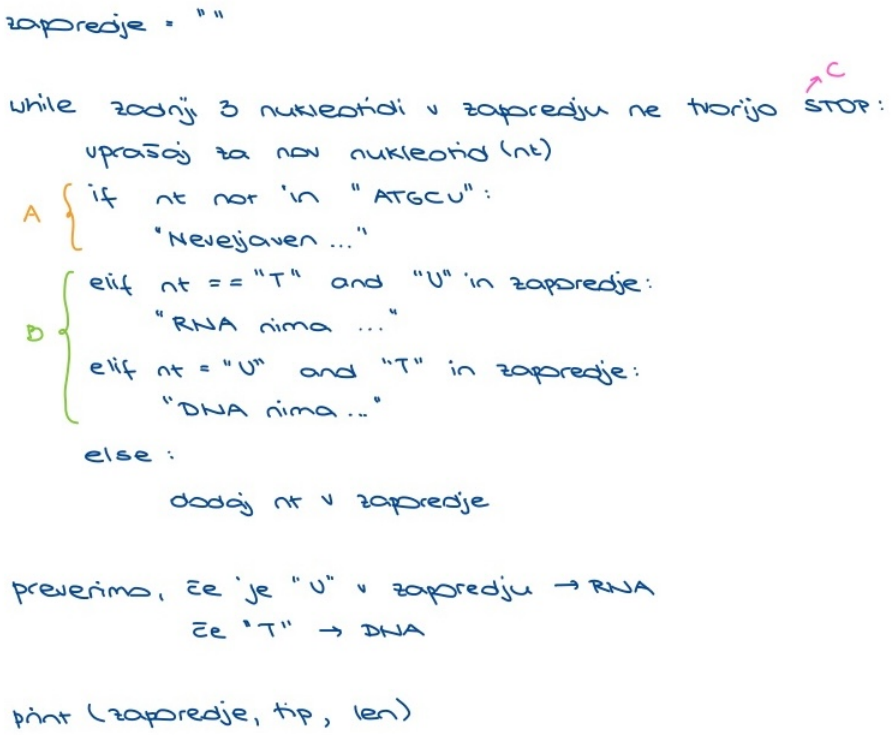
- Uporabnik lahko vnese le en znak naenkrat, pri čemer mora biti vneseni znak enak bodisi `C`, `T`, `A` ali `G` (DNA) bodisi `C`, `U`, `A` ali `G` (RNA). Če uporabnik vnese znak, ki ne ustreza tem pogojem, naj program izpiše `Neveljaven nukleotid!`.
- Program naj uporabniku v istem zaporedju ne dovoli vnašati znakov `T` in `U` (glej sledeči alineji):
  - V primeru, da je uporabnik predhodno vnesel `T`, kasneje pa `U`, naj program izpiše `DNA zaporedje ne more vsebovati baze U!`.
  - V primeru, da je uporabnik predhodno vnesel `U`, kasneje pa `T`, naj program izpiše `RNA zaporedje ne more vsebovati baze T!`.
- Program naj prekine vnašanje, ko uporabnik vnese stop kodon (`TAG`, `TGA` ali `TAA` v primeru DNA zaporedja oziroma `UAG`, `UGA` ali `UAA` v primeru RNA zaporedja).

Program naj na koncu izpiše celotno zaporedje in dolžino tega zaporedja (glej primere spodaj). Izpiše naj tudi tip zaporedja glede na vsebovanost nukleotida `T` oziroma `U`.

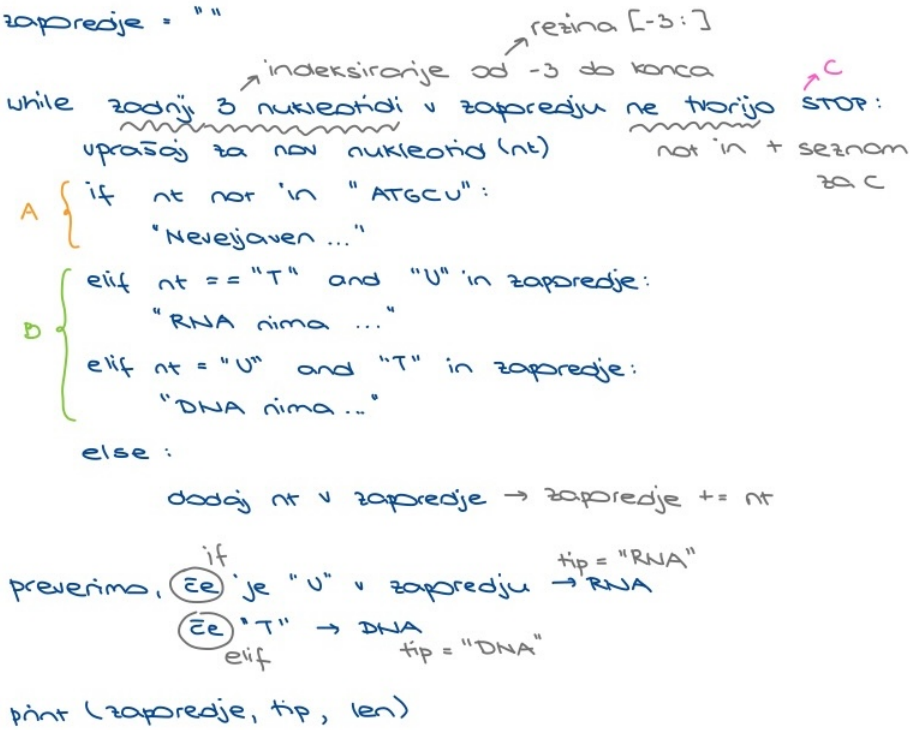
Primeri delovanja programa:

Ker želimo uporabniku vnašati poljubno dolgo zaporedje, bomo za izvedbo potrebovali zanko *while*. V zaporedje smemo dodajati le znake `A`, `T`, `C`, `G` in `U` (A). Če zaporedje že vsebuje `T`, uporabniku ne smemo dovoliti dodajanja `U` in obratno (B). Izvajanje vnašanja zaključimo, ko zadnji trije nukleotidi v zaporedju tvorijo enega izmed stop kodonov (C) - to bo naš pogoj za zaključek zanke.

Zapišemo si osnutek poteka kode:



Prazno zaporedje si pripravimo pred začetkom zanke *while*, sicer se bo dodani nukleotid vedno znova zbrisal. Zanka se mora zaključiti, ko zadnji trije nukleotidi v zaporedju tvorijo stop kodon, kar pomeni, da bo pogoj v *while* ravno obraten - dokler teh treh nukleotidov ni v zaporedju, se dodajajo novi nukleotidi.



Ogrodje naše kode je torej:

```
zaporedje = ""

while zaporedje[-3:] not in ["TAG", "TGA","TAA","UAG","UGA","UAA"]:
    nt = input("Vnesi nukleotid: ")

    # pogoj A

    # pogoj B

    # če je vse OK, dodamo nukleotid

if "U" in zaporedje:
    tip = "RNA"
elif "T" in zaporedje:
    tip = "DNA"

print(f"Vnesel si {tip} zaporedje {zaporedje} dolžine {len(zaporedje)}")
```

Sledi dodajanje pogojev - preverjali bomo, ali vnešeni nukleotid krši katero od pravil. Uporabimo kombinacijo *if, elif, elif, else*. Z *if* in dvema *elif* preprečimo dodajanje nukleotida, če ta ni v skladu z navodili. Pod *else* dodamo nukleotid v zaporedje, saj se noben od prejšnjih pogojev ni izvršil, kar pomeni, da nukleotid ustreza.

Rešitev je:

```
zaporedje = ""

while zaporedje[-3:] not in ["TAG", "TGA","TAA","UAG","UGA","UAA"]:
    nt = input("Vnesi nukleotid: ")
    if nt not in "ATGCU":
        print("Neveljaven nukleotid!")
    elif "U" in zaporedje and nt == "T":
        print("RNA zaporedje ne more vsebovati baze T!")
    elif "T" in zaporedje and nt == "U":
        print("DNA zaporedje ne more vsebovati baze U!")
    else:
        zaporedje += nt

if "T" in zaporedje:
    zap = "DNA"
elif "U" in zaporedje:
    zap = "RNA"

print(f"Vnesel si {zap} zaporedje {zaporedje} dolžine {len(zaporedje)}")
```

Če se vam zatakne pri določanju pogoja v zanki *while*, lahko kodo brez težav napišete z uporabo `while True`. Pri uporabi tega načina pazite le, da ne pozabite na `break`. V tem primeru je rešitev enaka kot zgoraj, spremeni se le pogoj pri *while* in koda pod *else*:

```
zaporedje = ""

while True:
    nt = input("Vnesi nukleotid: ")

    if ...

    elif ...

    elif ...

    else:
        zaporedje += nt
        if zaporedje[-3:] in ["TAG", "TGA","TAA","UAG","UGA","UAA"]:
            break
...
```

2. naloga

Osnova

Napiši funkcijo `komplement`, ki kot argument sprejme DNA zaporedje zapisano kot niz (`str`) in vrne komplementarno DNA zaporedje zapisano kot niz (`str`). Pri tem upoštevaj, da je nukelotid `T` komplementaren nukleotidu `A` (in obratno) ter da je nukleotid `G` komplementaren nukleotidu `C` (in obratno).

Funkcija naj preveri, če sekvenca res predstavlja DNA zaporedje (vsebuje lahko zgolj znake `C`, `T`, `A` ali `G`). V primeru, da sekvenca tega pogoja ne izpolnjuje, naj funkcija vrne prazen niz.

Prazen niz naj vrne tudi v primeru, ko funkcija kot vhodno sekvenco prejme prazen niz.

Primeri pravilne uporabe funkcije:

```
> komplement("TTAAGGCC")
'AATTCCGG'
> komplement("")
''
```

Dodatek

Funkcija naj sprejme izbiren (opcijski) argument `RNA`, ki naj bo privzeto postavljen na vrednost `False`. V tem primeru naj funkcija deluje enako kot v osnovni različici.

Če je argument `RNA` postavljen na vrednost `True`, naj funkcija vrne komplementarno RNA sekvenco, v kateri je `T` zamenjan za `U`.

Primeri pravilne uporabe funkcije:

```
> komplement("TTAAGG")
'AATTCC'
> komplement("TTAAGG", RNA=True)
'AAUUCC'
```

Rešitev

Osnova:

```
def komplement(zaporedje):
    komp_zap = ""

    for nt in zaporedje:
        if nt not in "ATGC":
            return ""
        if nt == "A":
            komp_zap += "T"
        elif nt == "T":
            komp_zap += "A"
        elif nt == "G":
            komp_zap += "C"
        elif nt == "C":
            komp_zap += "G"

    return komp_zap
```

Dodatek:

```
def komplement(zaporedje, RNA=False):
    komp_zap = ""

    for nt in zaporedje:
        if nt not in "ATGC":
            return ""
        if nt == "A":
            komp_zap += "T"
        elif nt == "T":
            komp_zap += "A"
        elif nt == "G":
            komp_zap += "C"
        elif nt == "C":
            komp_zap += "G"

    if RNA:
        return komp_zap.replace("T","U")

    return komp_zap
```

Rešitev z razlago

Osnova

Priprava komplementarnega zaporedja bomo izvedli z zanko *for*, s katero se pomikamo po črkah v nizu `zaporedje`. S kombinacijo *if-elif* preverimo, kateri nukleotid je v zaporedju in v spremenljivko `komp-zap` prištejemo njegov komplement.

```
def komplement(zaporedje):
    komp_zap = ""

    for nt in zaporedje:
        if nt not in "ATGC":
            return ""
        if nt == "A":
            komp_zap += "T"
        elif nt == "T":
            komp_zap += "A"
        elif nt == "G":
            komp_zap += "C"
        elif nt == "C":
            komp_zap += "G"

    return komp_zap
```

Če bi kodo zapisali z uporabo metode `.replace()` na sledeči način:

```
zaporedje.replace("T","A").replace("A","T").replace("G","C").replace("C","G")
```

bi najprej v zaporedju zamenjali vse `T` z `A`, nato pa vse `A`, tudi nove, s `T` in enako za `G` in `C`. S tem bi vračali enako zaporedje kot smo ga podali funkciji.

Če želimo uporabiti to metodo, moramo znake zamenjati z drugimi znaki in s tem preprečiti, da bi že menjane znake ponovno zamenjali:

```
komp_zap = zaporedje.replace("T","a").replace("A","t").replace("G","c").replac
↪  e("C","g")
komp_zap = komp_zap.upper()
```

Dodamo še metodo `.upper()`, s katero pretvorimo majhne črke v velike. Pri uporabi takšnega načina moramo paziti na ustrezno predhodno preverjanje veljavnosti zaporedja (vsebovati sme le črke `A`, `T`, `G` ali `C` - za to potrebujemo zanko *for* in na uporabo pravih spremenljivk - kje torej uporabimo `komp_zap` in kje `zaporedje`.

```
def komplement(zaporedje, RNA=False):

    for nt in zaporedje:
        if nt not in "ATGC":
            return ""

    komp_zap = zaporedje.replace("T","a").replace("A","t").replace("G","c").re
    ↪  place("C","g")
    komp_zap = komp_zap.upper()

    return komp_zap
```

Dodatek

Pri dodajanju opcijskega argumenta ohranimo jedro kode enako, dodamo le pogojni stavek *if*, s katerim preverimo, ali je argument `RNA` nastavljen na vrednost `True` - `if RNA:` namreč preverja `if RNA == True`.

Sledi priprava praznega niza `rna_komp`, v katerega bomo dodajali RNA zaporedje. Pomagamo si z že pripravljenim komplementom, ki se je tvoril v zanki *for* (4. vrstica). Z novo zanko *for* (18. vrstica) se pomikamo po komplementarnem zaporedju in v RNA komplement v primeru, da je nukleotid enak `T` dodamo `U`, sicer pa dodamo enak nukleotid kot je v komplementarnem zaporedju.

```
1  def komplement(zaporedje, RNA=False):
2      komp_zap = ""
3
4      for nt in zaporedje:
5          if nt not in "ATGC":
6              return ""
7          if nt == "A":
8              komp_zap += "T"
9          elif nt == "T":
10             komp_zap += "A"
11         elif nt == "G":
12             komp_zap += "C"
13         elif nt == "C":
14             komp_zap += "G"
15
16     if RNA:
17         rna_komp = ""
18         for nt in komp_zap:
19             if nt == "T":
20                 rna_komp += "U"
21             else:
22                 rna_komp += nt
23         return rna_komp
24
25     return komp_zap
```

V tem primeru je uporaba metode `.replace()` možna, saj želimo zamenjati le eno črko s črko, ki ni enaka nobeni od ostalih črk zaporedja:

```
def komplement(zaporedje, RNA=False):
    komp_zap = ""

    for nt in zaporedje:
        if nt not in "ATGC":
            return ""
        if nt == "A":
            komp_zap += "T"
        elif nt == "T":
            komp_zap += "A"
        elif nt == "G":
            komp_zap += "C"
        elif nt == "C":
            komp_zap += "G"

    if RNA:
        return komp_zap.replace("T","U")

    return komp_zap
```

Če to združimo z uporabo metode `.replace()` iz osnovnega dela kode, dobimo:

```
def komplement(zaporedje, RNA=False):

    for nt in zaporedje:
        if nt not in "ATGC":
            return ""

    komp_zap = zaporedje.replace("T","a").replace("A","t").replace("G","c").re
    ↪ place("C","g")
    komp_zap = komp_zap.upper()

    if RNA:
        return komp_zap.replace("T","U")

    return komp_zap
```

3. naloga

Napiši funkcijo `kvaliteta`, ki sprejme tri sezname:

- seznam nizov, ki predstavljajo DNA zaporedja zapisana kot `str`,
- seznam meritev količin DNA zapisanih kot `float`,
- seznam ocenjenega procenta nečistoč zapisanem kot `float`.

Predpostavljaj, da so vsi trije seznami poravnani - vrednosti na izbranem indeksu se v vseh treh seznamih nanašajo na isto DNA zaporedje. Prav tako lahko predpostavljaš, da so vsi trije seznami enako dolgi.

Funkcija naj vrne seznam terk oblike `(zaporedje, ocenjena_kolicina_DNA)`, pri čemer `zaporedje` predstavlja posamezno zaporedje zapisano kot `str`, `ocenjena_kolicina_DNA` pa oceno količine DNA brez nečistoč zapisano kot `float` zaokrožen na 2 decimalni mesti natančno.

Če so podani seznami prazni, naj funkcija vrne prazen seznam.

Primer pravilnega delovanja funkcije:

```
> zaporedja = ["AAA", "TTT", "C"]
> koncentracije = [10, 15, 5]
> procenti = [50.325235, 33.444, 15.4]
> kvaliteta(zaporedja, koncentracije, procenti)
[('AAA', 4.97), ('TTT', 9.98), ('C', 4.23)]
```

Rešitev

```
def kvaliteta(zaporedja, meritve, necistoce):

    if zaporedja == [] or meritve == [] or necistoce == []:
        return []

    kolicina_dna = []

    for i in range(0,len(meritve)):
        kolicina = round(meritve[i]*(1-necistoce[i]*0.01),2)
        kolicina_dna.append(kolicina)

    return list(zip(zaporedja, kolicina_dna))
```

Rešitev z razlago

Pri nalogah, kjer moramo preveriti, ali je podani argument prazen seznam/niz, moramo to preverjanje izvesti čim bolj v začetku funkcije. Če to preverjanje vključimo za kodo, ki že nekaj počne s seznamom, se lahko zgodi, da bomo pridelali napako, saj številne funkcije ne znajo operirati s praznimi podatkovnimi tipi (npr. funkcija `max` javi napako, če jo izvedemo nad praznim seznamom).

V besedilu je zapisano, da mora funkcija vračati prazen seznam, če je katerikoli od podanih seznamov prazen. To preverjanje izvedemo s stavkom *if*, med posamezne primerjave pa dodamo Boolov operator `or` - če je vsaj eden od seznamov prazen, bo funkcija vrnila prazen seznam.

Ker želimo, da funkcija vrača terke, ki vsebujejo ocenjeno količino DNA, moramo to naprej izračunati. V seznamih imamo podana zaporedja, njihove koncentracije in odstotek nečistoč. Da bi izračunali količino DNA, moramo odstotek najprej deliti s 100 (oz. množiti z 0.01) in ga odšteti od 1, da dobimo odstotek čiste DNA: `(1-procent/100)`, nato pa to pomnožiti še s koncentracijo DNA[1]. Obenem lahko že zaokrožimo rezultat na 2 decimalki:

```
kolicina = round(meritev*(1-procent*0.01), 2)
```

Ker imamo tri enako dolge sezname, lahko izračun količin izvedemo z uporabo indeksiranja - na istem indeksu bodo v različnih seznamih elementi, ki spadajo skupaj. Napišemo zanko *for*, kjer se pomikamo po indeksih in računamo koncentracije (8. vrstica).

```
1  def kvaliteta(zaporedja, meritve, necistoce):
2
3      if zaporedja == [] or meritve == [] or necistoce == []:
4          return []
5
6      kolicina_dna = []
7
8      for i in range(0,len(meritve)):
9          kolicina = round(meritve[i]*(1-necistoce[i]*0.01), 2)
10         kolicina_dna.append(kolicina)
11
12     return list(zip(zaporedja, kolicina_dna))
```

Pripravljeni seznam `kolicina_dna` bo vseboval toliko števil kot je zaporedij DNA v seznamu `zaporedja`, zato lahko pri vračanju tvorimo seznam terk z uporabo funkcije `zip`.

[1]Na vseh naslednjih kolokvijih in izpitih so bile vse formule, tudi trivialne, podane.

Funkcijo `zip` bi lahko uporabili že prej:

```
def kvaliteta(zaporedja, meritve, necistoce):

    if zaporedja == [] or meritve == [] or necistoce == []:
        return []

    seznam2 = []

    for z, m, n in zip(zaporedja, meritve, necistoce):
        kolicina = round(m*(1-n*0.01), 2)
        seznam2.append((z,kolicina))

    return seznam2
```

lahko pa bi se ji tudi povsem izognili:

```
def kvaliteta(zaporedja, meritve, necistoce):

    if zaporedja == [] or meritve == [] or necistoce == []:
        return []

    kolicina_dna = []

    for i in range(0,len(meritve)):
        kolicina = round(meritve[i]*(1-necistoce[i]*0.01), 2)
        kolicina_dna.append((zaporedja[i], kolicina))

    return kolicina_dna
```

4. naloga

Napiši funkcijo `naj_DNA`, ki sprejme seznam terk, kot ga vrača Naloga 3 (tj. seznam terk oblike `(zaporedje, ocenjena_kolicina_DNA)`, pri čemer `zaporedje` predstavlja posamezno zaporedje, `ocenjena_kolicina_DNA` pa oceno količine DNA brez nečistoč).

Funkcija naj vrne zaporedje DNA, ki ima največjo ocenjeno količino. Predpostavljaš lahko, da je tako največ eno zaporedje. V primeru, da funkciji podamo prazno zaporedje, naj vrne prazen niz.

Primeri pravilnega delovanja funkcije:

```
> naj_DNA([('AAA', 4.97), ('TTT', 9.98), ('C', 4.23)])
'TTT'
> naj_DNA([])
''
```

Rešitev

```
1  def naj_DNA(seznam):
2
3      if seznam == []:
4          return ""
5
6      maksimum = seznam[0][1]
7      naj_dna = seznam[0][0]
8
9      for zaporedje, kolicina in seznam:
10         if kolicina > maksimum:
11             maksimum = kolicina
12             naj_dna = zaporedje
13
14     return naj_dna
```

Rešitev z razlago

Da bi določili zaporedje z najvišjo koncentracijo DNA, moramo iterirati po seznamu terk. Ker ima vsaka terka dva elementa - zaporedje in koncentracijo - bomo uvedli pomožni spremenljivki `maksimum`, v katero bomo shranjevali največjo koncentracijo ter `naj_dna`, v katero bomo shranili zaporedje, ki spada k največji koncentraciji - podobno kot pri nalogi Največje in najmanjše število (2. naloga na 2. vajah).

Ker ne vemo, kakšne bodo vrednosti koncentracij, si pred začetkom iteracije z zanko *for* spremenljivko `maksimum` nastavimo na koncentracijo v prvi terki seznama, `naj_dna` pa na prvo zaporedje seznama:

```
maksimum = seznam[0][1]
naj_dna = seznam[0][0]
```

Z zanko *for* nato preverjamo, ali je količina DNA v trenutni iteraciji večja od dozdajšnjega maksimuma:

```
for zaporedje, kolicina in seznam:
    if kolicina > maksimum:
        maksimum = kolicina
        naj_dna = zaporedje
```

V nalogi je zapisano, da bo takšno zaporedje največ eno - če pa ne bi bilo, bi morali izvesti dve zanki *for*: v prvi zanki bi poiskali maksimum, v drugi pa zaporedja, ki ta maksimum dosegajo:

```
1 for zaporedje, kolicina in seznam:
2     if kolicina > maksimum:
3         maksimum = kolicina
4
5 naj_dna = []
6 for zaporedje, kolicina in seznam:
7     if kolicina == maksimum:
8         naj_dna.append(zaporedje)
```

2. kolokvij 2025/2026

Naloge shranite v datoteko `naloge.py`.

1. naloga

Podatki o zmagovalcih bostonskega maratona skozi leta so zapisani v tekstovni datoteki. Predpostavljajte, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljena vejica (`,`).

Primer vsebine datoteke `boston5.csv`:

```
leto,zmagovalec,drzava,cas,milje,kilometri
1946,Stylianos Kyriakides,Greece,02:29:27,26.2,42.2
1909,Henri Renaud,United States,02:53:36,24.5,39.4
1979,Joan Benoit,United States,2:35:15,26.2,42.2
1910,Fred S. Cameron,Canada,02:28:52,24.5,39.4
2001,Catherine Ndereba,Kenya,2:23:53,26.2,42.2
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(leto, zmagovalec, drzava, cas, kilometri)`. Pri tem naj:

- `leto` predstavlja leto maratona, zapisano kot `int`,
- `zmagovalec` predstavlja ime zmagovalca/zmagovalke v danem letu, zapisan kot `str`,
- `drzava` predstavlja državo, iz katere prihaja zmagovalec, zapisana kot `str`.
- `cas` predstavlja čas teka, pretvorjen v sekunde in zapisan kot `int` in
- `kilometri` predstavlja število pretečenih kilometrov, zapisanih kot `float`. Število naj bo zaokroženo na 1 decimalno mesto.

Predpostavljajte, da lahko katerakoli izmed spremenljivk v posamezni vrstici datoteke manjka. Vrstice z manjkajočimi podatki preskočite. Prav tako preskočite vrstico z imeni stolpcev.

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke('data/boston5.csv')
    [(1946, 'Stylianos Kyriakides', 'Greece', 8967, 42.2),
     (1909, 'Henri Renaud', 'United States', 10416, 39.4),
     (1979, 'Joan Benoit', 'United States', 9315, 42.2),
     (1910, 'Fred S. Cameron', 'Canada', 8932, 39.4),
     (2001, 'Catherine Ndereba', 'Kenya', 8633, 42.2)]
```

```
>>> preberi_podatke('data/boston_miss.csv')
    [(1946, 'Stylianos Kyriakides', 'Greece', 8967, 42.2),
     (1909, 'Henri Renaud', 'United States', 10416, 39.4),
     (1988, 'Rosa Mota', 'Portugal', 8670, 42.2),
     (1910, 'Fred S. Cameron', 'Canada', 8932, 39.4),
     (2001, 'Catherine Ndereba', 'Kenya', 8633, 42.2)]
```

Rešitev

```python
def preberi_podatke(datoteka):
    with open(datoteka, 'r', encoding='utf-8') as f:
        f.readline()

        podatki = []
        for vrstica in f:
            vrednosti = vrstica.strip().split(',')
            if "" not in vrednosti:
                leto, ime, drzava, cas, milje, kms = vrednosti
                cas = cas.split(":")
                cas = int(cas[0])*3600+int(cas[1])*60+int(cas[2])
                podatki.append((int(leto), ime, drzava, cas, float(kms)))

    return podatki
```

2. naloga

Osnova

Napišite funkcijo `v_slovar`, ki kot argument sprejme seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(leto, zmagovalec, drzava, cas, kilometri)`).

Funkcija naj vrača slovar, kjer so ključi države zmagovalcev, vrednosti pa seznami terk oblike `(leto, ime, cas)` s podatki zmagovalcev iz določene države.

Primer delovanja:

```
>>> podatki = [(1946, 'Stylianos Kyriakides', 'Greece', 8967, 42.2),
     (1909, 'Henri Renaud', 'United States', 10416, 39.4),
     (1979, 'Joan Benoit', 'United States', 9315, 42.2),
     (1910, 'Fred S. Cameron', 'Canada', 8932, 39.4),
     (2001, 'Catherine Ndereba', 'Kenya', 8633, 42.2)]
>>> v_slovar(podatki)
    {'Greece': [(1946, 'Stylianos Kyriakides', 8967)],
     'United States': [(1909, 'Henri Renaud', 10416), (1979, 'Joan Benoit',
     ↪  9315)],
    'Canada': [(1910, 'Fred S. Cameron', 8932)],
    'Kenya': [(2001, 'Catherine Ndereba', 8633)]}
```

Dodatek

V podatkih o zmagovalcih maratona so skozi leta spreminjali razdalje maratona, zato funkciji `v_slovar` dodajte opcijski argument `kilometri`.

Če je vrednost tega argumenta nastavljena na `0`, naj funkcija deluje kot je opisano prej, sicer pa naj v slovar dodaja le tiste terke oblike `(leto, ime, cas)`, ki dosegajo ali presegajo izbrano mejo.

Primer delovanja:

```
>>> podatki = [(1946, 'Stylianos Kyriakides', 'Greece', 8967, 42.2),
     (1909, 'Henri Renaud', 'United States', 10416, 39.4),
     (1979, 'Joan Benoit', 'United States', 9315, 42.2),
     (1910, 'Fred S. Cameron', 'Canada', 8932, 39.4),
     (2001, 'Catherine Ndereba', 'Kenya', 8633, 42.2)]
>>> v_slovar(podatki, kilometri = 42.2)
    {'Greece': [(1946, 'Stylianos Kyriakides', 8967)],
    'United States': [(1979, 'Joan Benoit', 9315)],
    'Kenya': [(2001, 'Catherine Ndereba', 8633)]}
```

Rešitev

```
def v_slovar(podatki, kilometri = 0):
    slovar = {}
    for leto, ime, drzava, cas, kms in podatki:
        if kilometri == 0:
            if drzava not in slovar:
                slovar[drzava] = []
            slovar[drzava].append((leto, ime, cas))
        else:
            if kms == kilometri:
                if drzava not in slovar:
                    slovar[drzava] = []
                slovar[drzava].append((leto, ime, cas))
    return slovar
```

3. naloga

Napišite funkcijo `veckrat`, ki kot argument sprejme seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(leto, zmagovalec, drzava, cas, kilometri)`), vrača pa množico imen večkratnih zmagovalcev - v množici naj bodo le zmagovalci, ki so zmagali več kot enkrat. Če je seznam terk prazen, naj funkcija vrne prazno množico.

Primer delovanja:

```
>>> podatki = [(1902, 'Sammy Mellor', 'United States', 9792, 39.4),
     (1903, 'John Lordan', 'United States', 9689, 39.4),
     (1904, 'Michael Spring', 'United States', 9484, 39.4),
     (1917, 'Bill Kennedy', 'United States', 8917, 39.4),
     (1919, 'Carl Linder', 'United States', 8953, 39.4),
     (1922, 'Clarence DeMar', 'United States', 8290, 39.4),
     (1923, 'Clarence DeMar', 'United States', 8627, 39.4),
     (1924, 'Clarence DeMar', 'United States', 8980, 42.0),
     (1925, 'Charles Mellor', 'United States', 9180, 42.0),
     (1926, 'Johnny Miles', 'Canada', 8740, 42.0),
     (1927, 'Clarence DeMar', 'United States', 9622, 42.2),
     (1928, 'Clarence DeMar', 'United States', 9427, 42.2),
     (1929, 'Johnny Miles', 'Canada', 9188, 42.2),
     (1930, 'Clarence DeMar', 'United States', 9288, 42.2),
     (1931, 'James Henigan', 'United States', 10005, 42.2),
     (1969, 'Sara Mae Berman', 'United States', 12166, 42.2),
     (1970, 'Sara Mae Berman', 'United States', 11107, 42.2),
     (1971, 'Sara Mae Berman', 'United States', 11310, 42.2),
     (1993, 'Olga Markova', 'Russia', 8727, 42.2),
     (1994, 'Uta Pippig', 'Germany', 8505, 42.2)]
>>> veckrat(podatki)
    {'Clarence DeMar', 'Johnny Miles', 'Sara Mae Berman'}
```

Rešitev

```
def veckrat(podatki):
    mn = set()
    imena = []
    if podatki == []:
        return mn
    for leto, ime, drzava, cas, kms in podatki:
        imena.append(ime)
    for ime in imena:
        if imena.count(ime) > 1:
            mn.add(ime)
    return mn
```

4. naloga

Napišite funkcijo `boljse`, ki kot argumenta sprejme:

- seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(leto, zmagovalec, drzava, cas, kilometri)`) in
- dolžino proge (število kilometrov), zapisano kot `float`.

Funkcija naj vrne množico imen tistih večkratnih zmagovalcev, ki so pri isti dolžini proge (istem številu kilometrov) vsaj enkrat izboljšali svoj čas glede na eno izmed svojih prejšnjih zmag. To pomeni, da ima zmagovalec v kasnejšem letu krajši čas teka kot v prejšnjem letu, pri čemer upoštevamo le zmage z izbranim številom kilometrov.

Funkcija naj vrne prazno množico v primerih, da je seznam terk prazen ali noben večkratni zmagovalec ni izboljšal svojega časa pri izbrani dolžini proge.

Predpostavljate lahko, da bodo podatki vedno urejeni po naraščajočem letu (starejša leta so v seznamu na nižjih indeksih kot novejša leta).

Namig: pri snovanju rešitve razmislite o uporabi slovarjev, pri tem pa pazite, da boste vedno primerjali čas iz starejšega leta s časom iz novejšega.

Primer delovanja:

```
>>> podatki = [(1922, 'Clarence DeMar', 'United States', 8290, 39.4),
     (1923, 'Clarence DeMar', 'United States', 8627, 39.4),
     (1924, 'Clarence DeMar', 'United States', 8980, 42.0),
     (1927, 'Clarence DeMar', 'United States', 9622, 42.2),
     (1928, 'Clarence DeMar', 'United States', 9427, 42.2),
     (1929, 'Johnny Miles', 'Canada', 9188, 42.2),
     (1930, 'Clarence DeMar', 'United States', 9288, 42.2),
     (1969, 'Sara Mae Berman', 'United States', 12166, 42.2),
     (1970, 'Sara Mae Berman', 'United States', 11107, 42.2),
     (1971, 'Sara Mae Berman', 'United States', 11310, 42.2)]
>>> boljse(podatki, 42.2)
{'Clarence DeMar', 'Sara Mae Berman'}
```

Rešitev

```
def boljse(podatki, kilometri):
    slovarcek = {}
    izboljsani = set()
    for _,ime,_,cas,kms in podatki:
        if ime in veckrat(podatki) and kms == kilometri:
            if ime not in slovarcek:
                slovarcek[ime] = []
            slovarcek[ime].append(cas)
    for ime, value in slovarcek.items():
        for i in range(len(value)-1):
            if value[i+1] < value[i]:
                izboljsani.add(ime)
                break
    return izboljsani
```

2. kolokvij 2024/2025*

Naloge shranite v datoteko z imenom `naloge.py`.

1. naloga

Podatki o dojemanju sreče po državah so zapisani v tekstovni datoteki. Predpostavljaj, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljeno podpičje (`;`).

Primer vsebine datoteke `sreca5.txt`:

```
drzava;regija;sreca;GDP;druzina;zivljenje;svoboda;korupcija
Norway;Western Europe;7.498;1.57744;1.1269;0.79579;0.59609;0.35776
Finland;Western Europe;7.413;1.40598;1.13464;0.81091;0.57104;0.41004
Slovenia;Central and Eastern
↪  Europe;5.768;1.29947;1.05613;0.79151;0.53164;0.03635
Peru;Latin America and Caribbean;5.743;0.99602;0.81255;0.62994;0.37502;0.05292
Turkmenistan;Central and Eastern
↪  Europe;5.658;1.08017;1.03817;0.44006;0.37408;0.28467
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(regija, drzava, sreca, GDP, druzina, zivljenje, svoboda, korupcija)`. Pri tem naj:

- `regija` predstavlja ime regije, v kateri se nahaja država, zapisano kot `str`,
- `drzava` predstavlja ime drzave zapisane kot `str`,
- `sreca` predstavlja t.i. indeks sreče, zapisan kot `float`,
- `GDP` predstavlja bruto domači proizvod, zapisan kot `float`,
- `druzina` predstavlja število, zapisano kot `float`,
- `zivljenje` predstavlja število, zapisano kot `float`,
- `svoboda` predstavlja faktor, zapisan kot `float`,
- `korupcija` predstavlja faktor, zapisan kot `float`.

Vsa števila, zapisana kot `float`, naj bodo zaokrožena na 3 decimalna mesta.

Predpostavljajte, da lahko katerakoli izmed številskih spremenljivk v posamezni vrstici datoteke manjka. V tem primeru naj terka na njenem mestu vsebuje vrednost False.

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke("sreca5.txt")
[('Western Europe', 'Norway', 7.498, 1.577, 1.127, 0.796, 0.596, 0.358),
 ('Western Europe', 'Finland', 7.413, 1.406, 1.135, 0.811, 0.571, 0.41),
 ('Central and Eastern Europe','Slovenia', 5.768, 1.299, 1.056, 0.792, 0.532,
 ↪  0.036),
```

```
 ('Latin America and Caribbean', 'Peru', 5.743, 0.996, 0.813, 0.63, 0.375,
 ↪  0.053),
 ('Central and Eastern Europe', 'Turkmenistan', 5.658, 1.08, 1.038, 0.44,
 ↪  0.374, 0.285)]
>>> preberi_podatke("sreca10.txt")
[('Central and Eastern Europe', 'Poland', False, 1.246, 1.047, 0.691, 0.452,
↪  0.055),
 ('Sub-Saharan Africa', 'Liberia', 3.622, 0.107, 0.504, 0.232, 0.257, 0.049),
 ('Sub-Saharan Africa', 'Mauritius', 5.648, 1.144, 0.757, 0.662, 0.461, 0.052),
 ('Eastern Asia', 'Mongolia', 4.907, 0.989, 1.09, 0.555, 0.36, 0.033),
 ('Western Europe', 'Switzerland', 7.509, 1.527, False, 0.863, 0.586, 0.412),
 ('Sub-Saharan Africa', 'Zimbabwe', 4.193, 0.35, 0.715, 0.16, 0.254, 0.086),
 ('Southern Asia', 'India', 4.404, 0.74, 0.292, 0.451, 0.403, 0.087),
 ('Central and Eastern Europe', 'Latvia', 5.56, 1.218, 0.95, 0.64, 0.28,
 ↪  0.089),
 ('Central and Eastern Europe', 'Slovakia', 6.078, 1.28, 1.083, False, 0.234,
 ↪  0.029),
 ('Latin America and Caribbean', 'Nicaragua', 5.992, 0.694, 0.895, 0.652,
 ↪  0.466, False)]
```

Rešitev

Daljša rešitev:

```
def preberi_podatke(datoteka):
    seznam = []
    with open(datoteka, "r", encoding="utf8") as f:
        f.readline()
        for vrstica in f:
            podatki = vrstica.strip().split(";")
            for i in range(len(podatki)):
                if podatki[i] == "":
                    podatki[i] = False
            drzava,regija,sreca,GDP,druzina,zivljenje,svoboda,korupcija =
            ↪ podatki
            sreca = round(float(sreca),3)
            GDP = round(float(GDP),3)
            druzina = round(float(druzina),3)
            zivljenje = round(float(zivljenje),3)
            svoboda = round(float(svoboda),3)
            korupcija = round(float(korupcija),3)
            seznam.append((regija, drzava, sreca, GDP, druzina, zivljenje,
            ↪ svoboda, korupcija))
    return seznam
```

Krajša rešitev:

```
def preberi_podatke(datoteka):
    seznam = []
    with open(datoteka, "r", encoding="utf8") as f:
        f.readline()
        for vrstica in f:
            podatki = vrstica.strip().split(";")
            for i in range(len(podatki)):
                if podatki[i] == "":
                    podatki[i] = False
                elif i in list(range(2,len(podatki))):
                    podatki[i] = round(float(podatki[i]),3)
            drzava,regija,sreca,GDP,druzina,zivljenje,svoboda,korupcija =
            ↪ podatki
            seznam.append((regija, drzava, sreca, GDP, druzina, zivljenje,
            ↪ svoboda, korupcija))
    return seznam
```

Rešitev z razlago

Ob branju navodil nalog, ki zahtevajo branje podatkov iz datoteke, si dobro poglejte primer datoteke - predvsem vrstni red podatkov v njej in uporabljeno ločilo -, ter vrstni red podatkov v terki, ki jo mora funkcija vrniti.

> Podatki o dojemanju sreče po državah so zapisani v tekstovni datoteki. Predpostavljaj, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljeno podpičje (;).
>
> Primer vsebine datoteke `sreca5.txt`:
>
> ```
> drzava;regija;sreca;GDP;druzina;zivljenje;svoboda;korupcija
> Norway;Western Europe;7.498;1.57744;1.1269;0.79579;0.59609;0.35776
> Finland;Western Europe;7.413;1.40598;1.13464;0.81091;0.57104;0.41004
> Slovenia;Central and Eastern
> ↪ Europe;5.768;1.29947;1.05613;0.79151;0.53164;0.03635
> Peru;Latin America and Caribbean;5.743;0.99602;0.81255;0.62994;0.37502;0.05292
> Turkmenistan;Central and Eastern
> ↪ Europe;5.658;1.08017;1.03817;0.44006;0.37408;0.28467
> ```
>
> Napišite funkcijo preberi_podatke, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike (regija, drzava, sreca, GDP, druzina, zivljenje, svoboda, korupcija). Pri

Kadar beremo podatke iz datoteke, moramo vedno najprej datoteko odpreti. To lahko storimo s kodo `with open(datoteka, "r", encoding="utf8") as f:` ali pa `f = open(datoteka, "r", encoding="utf8")`. Če se odločimo za uporabo drugega načina, moramo datoteko z uporabo `f.close()` pred zaključkom funkcije zapreti.

V prvi vrstici datoteke imamo imena stolpcev, ki jih želimo izpustiti, zato pred začetkom zanke `for` uporabimo `f.readline()`, s čimer preskočimo prvo vrstico.

Sledi zanka `for`, s katero se pomikamo po vrsticah. Vsaka vrstica je v obliki niza, ki mu s pomočjo metode `.strip()` odstranimo končne oznake za novo vrstico (`\n`). Tej metodi sledi metoda `.split(";")`, s katero razbijemo niz v seznam nizov. Niz razbijemo po podpičju.

Naloga od nas zahteva, da v primeru manjkajočih podatkov na njihovo mesto v terki shranimo vrednost `False`. Če podatek manjka, bosta v vrstici zapisani dve enaki ločili skupaj: `Slovenia;;5.768;1.29947;1.05613;0.79151;0.53164;0.03635`. Ko takšno vrstico razbijemo z metodo `.split(";")`, bo v seznamu na mestu, kjer sta dve ločili, prazen niz: `["Slovenia", "", "5.768", "1.29947", "1.05613", "0.79151", "0.53164", "0.03635"]`. Prazen niz se lahko pojavi kjerkoli v vrstici, zato se z zanko `for` in z indeksiranjem pomikamo po seznamu ter element na ustreznem indeksu zamenjamo (glejte 6. vaje v skripti Vaje pri predmetu Osnove programiranja):

```
1 for i in range(len(podatki)):
2     if podatki[i] == "":
3         podatki[i] = False
```

Ko prazne nize nadomestimo, lahko podatke v seznamu poimenujemo in pretvorimo v zahtevane podatkovne oblike ter jih hkrati še zaokrožimo:

```
drzava,regija,sreca,GDP,druzina,zivljenje,svoboda,korupcija = podatki
sreca = round(float(sreca),3)
...
```

Pretvorbo in zaokroževanje pa lahko izpeljemo kar v isti zanki `for`. Ob ogledu podatkov v datoteki opazimo, da je zadnjih 6 podatkov številskih. V naslednji vrstici tako preverimo, ali je trenutni indeks na intervalu `[2,8)`, in podatek pod tem indeksom pretvorimo v decimalno število ter ga zaokrožimo:

```
for i in range(len(podatki)):
    if podatki[i] == "":
        podatki[i] = False
    elif i in list(range(2,len(podatki))):
        podatki[i] = round(float(podatki[i]),3)
```

Preden dodamo novo terko v seznam, še enkrat preverimo vrstni red podatkov.

2. naloga

Osnova

Napišite funkcijo `v_slovar`, ki kot argument sprejme seznam terk, kot ga vrača funkcija `preberi_podatke` (1. naloga) - tj. seznam terk oblike `(regija, drzava, sreca, GDP, druzina, zivljenje, svoboda, korupcija)`.

Funkcija naj vrača slovar (`dict`), ki kot ključe vsebuje regije, kot vrednosti pa seznam terk oblike `(drzava, sreca)`.

Primer pravilne izvedbe funkcije:

```
>>> podatki1 = [('Western Europe', 'Norway', 7.498, 1.577, 1.127, 0.796, 0.596,
↪  0.358),
 ('Western Europe', 'Finland', 7.413, 1.406, 1.135, 0.811, 0.571, 0.41),
 ('Central and Eastern Europe', 'Slovenia', 5.768, 1.299, 1.056, 0.792, 0.532,
 ↪  0.036),
 ('Latin America and Caribbean', 'Peru', 5.743, 0.996, 0.813, 0.63, 0.375,
 ↪  0.053),
 ('Central and Eastern Europe', 'Turkmenistan', 5.658, 1.08, 1.038, 0.44,
 ↪  0.374, 0.285)]
>>> v_slovar(podatki1)
{'Western Europe': [('Norway', 7.498), ('Finland', 7.413)],
 'Central and Eastern Europe': [('Slovenia', 5.768), ('Turkmenistan', 5.658)],
 'Latin America and Caribbean': [('Peru', 5.743)]}
```

Dodatek

Funkcija naj v seznam terk shrani le podatke tistih držav, kjer je vrednost spremenljivke `sreca` različna od vrednosti `False`. Slovar naj kot ključe vsebuje le tiste regije, ki imajo v seznamu terk vsaj eno državo.

Primer pravilne izvedbe funkcije:

```
>>> podatki2 = [('Central and Eastern Europe', 'Poland', False, 1.246, 1.047,
↪  0.691, 0.452, 0.055),
 ('Western Europe', 'Switzerland', 7.509, 1.527, False, 0.863, 0.586, 0.412),
 ('Central and Eastern Europe', 'Latvia', False, 1.218, 0.95, 0.64, 0.28,
 ↪  0.089),
 ('Central and Eastern Europe', 'Slovakia', False, 1.28, 1.083, False, 0.234,
 ↪  0.029),
 ('Latin America and Caribbean',  'Nicaragua', 5.992, 0.694, 0.895, 0.652,
 ↪  0.466, False)]
>>> v_slovar(podatki2)
{'Western Europe': [('Switzerland', 7.509)],
 'Latin America and Caribbean': [('Nicaragua', 5.992)]}
```

Rešitev

Osnova:

```
def v_slovar(seznam):
    slovar = {}
    for terka in seznam:
        if terka[0] not in slovar:
            slovar[terka[0]] = []
        slovar[terka[0]].append((terka[1], terka[2]))
    return slovar
```

Dodatek:

```
def v_slovar(seznam):
    slovar = {}
    for terka in seznam:
        if terka[2] != False:
            if terka[0] not in slovar:
                slovar[terka[0]] = []
            slovar[terka[0]].append((terka[1], terka[2]))
    return slovar
```

Rešitev z razlago

Osnova

Najprej si pripravimo prazen slovar, v katerega bomo dodajali ključe in vrednosti. Z zanko `for` se začnemo pomikati po terkah v seznamu, znotraj zanke pa preverimo, ali ključ - regija -, v slovarju že obstaja. Če ne, kot pripadajočo vrednost ustvarimo prazen seznam, v katerega izven stavka `if` dodamo terko oblike `(drzava, sreca)`.

Dodatek

V slovar bomo shranjevali le podatke držav z vrednostjo podatka o sreči različno od `False`. Osnovno kodo dopolnimo le z enim stavkom `if`. Če ta podatek obstaja, potem ima regija vsaj eno državo, kar zadosti tudi drugemu pogoju naloge.

3. naloga

Napišite funkcijo `regijska_povprecja`, ki kot argument sprejme slovar kot ga vrača funkcija `v_slovar` (2. naloga).

Funkcija naj vrača slovar, v katerem kot ključ nastopa posamezna regija, kot vrednost pa regijsko povprečje spremenljivke `sreca`. To naj bo zaokroženo na dve decimalni mesti natančno.

Primera pravilne izvedbe funkcije:

```
>>> podatki = {'Western Europe': [('Norway', 7.498), ('Finland', 7.413)],
 'Central and Eastern Europe': [('Slovenia', 5.768), ('Turkmenistan', 5.658)],
 'Latin America and Caribbean': [('Peru', 5.743)]}
>>> regijska\_povprecja(podatki)
{'Western Europe': 7.46,
 'Central and Eastern Europe': 5.71,
 'Latin America and Caribbean': 5.74}
```

Rešitev

```
def regijska_povprecja(slovar):
    slovar2 = {}
    for kljuc, vrednost in slovar.items():
        vsota = 0
        for drzava, sreca in vrednost:
            vsota += sreca
        slovar2[kljuc] = round(vsota/len(vrednost),2)
    return slovar2
```

Rešitev z razlago

Ponovno si najprej pripravimo prazen slovar, v katerega bomo dodajali regijska povprečja. Sledi iteracija po slovarju, pripravljenem s funkcijo `v_slovar`: ker potrebujemo tako ključ kot tudi vrednosti, v zanki `for` uporabimo metodo `.items()`, ki nam ključe in vrednosti slovarja spremeni v seznam terk oblike `(kljuc, vrednost)`.

Vrednost pod določenim ključem je seznam terk: `[('Slovenia', 5.768), ('Turkmenistan', 5.658)]`, zato znotraj prve zanke `for` ugnezdimo še eno zanko, ki bo tekla po seznamu terk. Zunanja zanka `for` se pomika po regijah in seznamih, notranja pa po terkah v seznamu pod posamezno regijo.

V spremenljivko `vsota` prištevamo vrednosti podatka `sreca`. To spremenljivko z vsakim novim ključem nastavimo na 0, s čimer poskrbimo, da se seštevek prejšnje regije ne prenaša naprej.

```
1 for kljuc, vrednost in slovar.items():
2     vsota = 0
3     for drzava, sreca in vrednost:
4         vsota += sreca
5     slovar2[kljuc] = round(vsota/len(vrednost),2)
```

Ko z ugnezdeno zanko `for` zaključimo iteracijo po seznamu znotraj posamezne regije, v novem slovarju ustvarimo ključ - ime regije -, s pripadajočim povprečjem kot vrednostjo (vrstica 5).

Pri ugnezdenih zankah bodite še posebej pozorni na zamike.

4. naloga

Osnova

Napišite funkcijo `najvecje_bogastvo`, ki kot argument sprejme seznam terk, kot ga vrača funkcija `preberi_podatke` (1. naloga). Funkcija naj vrne ime države z najvišjo vrednostjo spremenljivke `GDP`. Pri tem lahko predpostavljaš, da je taka samo ena država.

Primer pravilnega delovanja funkcije:

```
>>> podatki1 = [('Central and Eastern Europe', 'Slovenia', 5.768, 1.299, 1.056,
↪  0.792, 0.532, 0.036)]
>>> najvecje_bogastvo(podatki1)
'Slovenia'
>>> podatki2 = [('Western Europe', 'Norway', 7.498, 1.577, 1.127, 0.796, 0.596,
↪  0.358),
 ('Western Europe', 'Finland', 7.413, 1.406, 1.135, 0.811, 0.571, 0.41),
 ('Central and Eastern Europe', 'Slovenia', 5.768, 1.299, 1.056, 0.792, 0.532,
 ↪  0.036),
 ('Latin America and Caribbean', 'Peru', 5.743, 0.996, 0.813, 0.63, 0.375,
 ↪  0.053),
 ('Central and Eastern Europe', 'Turkmenistan', 5.658, 1.08, 1.038, 0.44,
 ↪  0.374, 0.285)]
>>> najvecje_bogastvo(podatki2)
'Norway'
```

Dodatek

Funkcija naj sprejme še izbirni argument `izbrana_regija`. V primeru, da je ta argument pri klicu funkcije podan, naj funkcija išče le po izbrani regiji. V primeru, da ta argument ni podan, naj funkcija deluje kot opisano zgoraj.

Primer pravilnega delovanja funkcije:

```
>>> podatki = [('Western Europe', 'Norway', 7.498, 1.577, 1.127, 0.796, 0.596,
↪  0.358),
 ('Western Europe', 'Finland', 7.413, 1.406, 1.135, 0.811, 0.571, 0.41),
 ('Central and Eastern Europe', 'Slovenia', 5.768, 1.299, 1.056, 0.792, 0.532,
 ↪  0.036),
 ('Latin America and Caribbean', 'Peru', 5.743, 0.996, 0.813, 0.63, 0.375,
 ↪  0.053),
 ('Central and Eastern Europe', 'Turkmenistan', 5.658, 1.08, 1.038, 0.44,
 ↪  0.374, 0.285)]
>>> najvecje_bogastvo(podatki, 'Central and Eastern Europe')
'Slovenia'
>>> najvecje_bogastvo(podatki, 'Luna')
''
```

Rešitev

Osnova:

```
def najvecje_bogastvo(seznam):
    maksi = 0
    drzava = ""
    for terka in seznam:
        if terka[3] > maksi:
            maksi = terka[3]
            drzava = terka[1]
    return drzava
```

Dodatek:

```
def najvecje_bogastvo(seznam, izbrana_regija = False):
    maksi = 0
    drzava = ""
    for terka in seznam:
        if izbrana_regija != False:
            if izbrana_regija == terka[0] and terka[3] > maksi:
                maksi = terka[3]
                drzava = terka[1]
        else:
            if terka[3] > maksi:
                maksi = terka[3]
                drzava = terka[1]
    return drzava
```

Rešitev z razlago

Osnova

Pred zanko `for` nastavimo spremenljivko `maksi` na 0 (BDP ne more biti negativno število), spremenljivko `drzava`, kamor bomo shranjevali ime države z najvišjim BDP, pa nastavimo na prazen niz. Nato izvedemo iskanje maksimuma kot na 2. vajah, ime države, ki pripada maksimumu pa shranjujemo v spremenljivko `drzava`, ki jo po končani zanki `for` vrnemo kot rezultat funkcije.

Dodatek

Če želimo funkcijo prilagoditi za določeno regijo, uporabimo opcijski argument. Osnovno kodo razširimo z dodatnim stavkom `if`, s katerim preverimo, ali je opcijski argument različen od prednastavljene vrednosti `False`. Če je različen, iskanju maksimuma dodamo pogoj ujemanja regije z zapisom v opcijskem argumentu (vrstica 6). Če opcijskega argumenta ne nastavimo na drugačno vrednost, ostane iskanje maksimuma enako kot prej.

2. kolokvij 2023/2024*

Naloge shranite v datoteko z imenom `naloge.py`.

1. naloga

Podatke o Nobelovih nagrajencih imamo zapisane v tekstovnih datotekah. Predpostavljaj, da ima datoteka obliko, ki je skladna s spodnjim zgledom. V prvi vrstici so podana imena stolpcev, za ločilo pa je uporabljeno podpičje (`;`).

Primer datoteke `nobelove_S.csv`:

```
leto;ime;priimek;kategorija;spol;datum rojstva;drzava rojstva
1992;Derek;Walcott;literature;male;23-01-1930;LC
1967;George;Wald;medicine;male;18-11-1906;US
1945;Gabriela;Mistral;literature;female;1889-04-07;CL
2020;Paul;Milgrom;economics;male;20-04-1948;US
2016;David J.;Thouless;physics;male;21-09-1934;GB
```

Napiši funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke (`str`) in vrne vsebino datoteke kot seznam terk oblike:
`(leto, priimek, kategorija, spol, datum_rojstva, drzava_rojstva)`.

Pri tem naj

- `leto` predstavlja leto prejema nagrade zapisano kot `int`,
- `priimek` predstavlja priimek nagrajenca zapisan kot `str`,
- `kategorija` predstavlja kategorijo, v kateri je bila nagrada podeljena, zapisano kot `str`,
- `spol` predstavlja spol nagrajenca in lahko zavzame vrednost `'m'` (male - moški) ali `'f'` (female - ženska),
- `datum_rojstva` predstavlja datum rojstva zapisan kot `str`,
- `drzava_rojstva` predstavlja oznako države rojstva nagrajenca zapisano kot `str`.

Napotki in namigi:

- datoteka vsebuje stolpec, ki ga v seznam ne boste shranjevali (`ime`);
- pri shranjevanju v seznam izpustite vse vrstice, pri katerih podatek o priimku nagrajenca ali državi rojstva manjka;
- spol nagrajenca v datoteki zaseda vrednosti male in female; v seznamu terk moške predstavite z vrednostjo `m`, ženske pa z vrednostjo `f`;
- pri datumu rojstva ohranite obliko zapisa, kot je uporabljena v datoteki.

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke("nobelove_S.csv")
[(1992, 'Walcott', 'literature', 'm', '23-01-1930', 'LC'),
 (1967, 'Wald', 'medicine', 'm', '18-11-1906', 'US'),
 (1945, 'Mistral', 'literature', 'f', '1889-04-07', 'CL'),
 (2020, 'Milgrom', 'economics', 'm', '20-04-1948', 'US'),
 (2016, 'Thouless', 'physics', 'm', '21-09-1934', 'GB')]
>>> preberi_podatke("nobelove_S2.csv")
[(1998, 'Kohn', 'chemistry', 'm', '09-03-1923', 'AT'),
 (1975, 'Sakharov', 'peace', 'm', '21-05-1921', 'RU'),
 (2015, 'Lindahl', 'chemistry', 'm', '28-01-1938', 'SE'),
 (1937, 'Haworth', 'chemistry', 'm', '1883-03-19', 'GB')]
```

Rešitev

```
def preberi_podatke(datoteka):
    seznam = []
    with open(datoteka, "r", encoding = "utf8") as f:
        f.readline()
        for vrstica in f:
            podatki = vrstica.strip().split(";")
            leto, ime, priimek, kategorija, spol, datum_rojstva, drzava_rojstva
            ↪  = podatki

            if priimek == "" or drzava_rojstva == "":
                continue

            if spol == "female":
                spol = "f"
            else:
                spol = "m"

            terka = (int(leto), priimek, kategorija, spol, datum_rojstva,
            ↪  drzava_rojstva)
            seznam.append(terka)
        return seznam
```

Rešitev z razlago

Naloge se lotimo po že znanem receptu za takšen tip naloge (glejte 9. vaje v skripti Vaje pri predmetu Osnove programiranja, zgled 9.5.2): datoteko odpremo, zavržemo prvo vrstico z imeni stolpcev in se z zanko `for` začnemo pomikati po vrsticah. Posamezno vrstico razbijemo z metodo `.split(";")`, nato pa vsakemu podatku pripišemo ime spremenljivke, pod katero ga bomo kasneje klicali.

Naloga zahteva, da preskočimo tiste vrstice, kjer podatek o priimku ali državi rojstva manjka, zato pred nadaljnjim delom najprej izvedemo pogojni stavek `if`, pod katerim je `continue` (vrstici 9 in 10). Slednji omogoča, da preostanek kode v zanki `for` izpustimo in se pomaknemo na naslednjo vrstico.

Ta del lahko kodiramo tudi brez `continue` - s pogojnim stavkom preverimo, ali podatka o priimku in državi rojstva obstajata, sicer pa ne naredimo nič:

```
1 if priimek != "" and drzava_rojstva != "":
2
3     if spol == "female":
4         spol = "f"
5     else:
6         spol = "m"
7
8     terka = (int(leto), priimek, kategorija, spol, datum_rojstva,
      ↪  drzava_rojstva)
9     seznam.append(terka)
```

Če se odločimo za uporabo tega načina, moramo biti pozorni na logični operator (`and`), saj morata obstajati oba podatka, da lahko terko vključimo v seznam. Prav tako moramo paziti na zamike, ker želimo, da se terka v seznam doda le, kadar je pogoj o obstoju priimka in države izpolnjen, zato je dodajanje terke v seznam zamaknjeno pod ta pogoj.

2. naloga

Napiši funkcijo `slovar_drzav`, ki kot argument sprejme seznam terk, kot ga vrača Naloga 1, tj. seznam terk oblike `(leto, priimek, kategorija, spol, datum_rojstva, drzava_rojstva)`.

Funkcija naj vrne slovar (`dict`), ki kot ključe vsebuje oznake držav, ki se pojavijo v seznamu terk (`drzava_rojstva`). Na posamezno oznako države naj bo kot vrednost vezano število Nobelovih nagrajencev (`int`), ki so se rodili v državi.

Primer pravilne izvedbe funkcije:

```
>>> s = [(1901, 'Röntgen', 'physics', 'm', '1845-03-27', 'DE'),
 (1901, 'von Behring', 'medicine', 'm', '1854-03-15', 'PL'),
 (1901, 'Dunant', 'peace', 'm', '1828-05-08', 'CH'),
 (1901, 'Passy', 'peace', 'm', '1822-05-20', 'FR'),
 (1901, 'Prudhomme', 'literature', 'm', '1839-03-16', 'FR'),
 (1901, "van 't Hoff", 'chemistry', 'm', '1852-08-30', 'NL'),
 (1902, 'Fischer', 'chemistry', 'm', '1852-10-09', 'DE'),
 (1902, 'Mommsen', 'literature', 'm', '1817-11-30', 'DE'),
 (1902, 'Gobat', 'peace', 'm', '1843-05-21', 'CH'),
 (1902, 'Ducommun', 'peace', 'm', '1833-02-19', 'CH')]
>>> slovar_drzav(s)
{'DE': 3, 'PL': 1, 'CH': 3, 'FR': 2, 'NL': 1}
```

Rešitev

```
def slovar_drzav(seznam):
    slovar = {}
    for leto, priimek, kategorija, spol, datum_rojstva, drzava_rojstva in
    ↪  seznam:
        if drzava_rojstva not in slovar:
            slovar[drzava_rojstva] = 1
        else:
            slovar[drzava_rojstva] += 1
    return slovar
```

Rešitev z razlago

Najprej ustvarimo prazen slovar, nato pa se z zanko `for` pomikamo po posameznih terkah, ki jih razpakiramo že v sami glavi zanke. S pogojnim stavkom `if` preverimo, ali ključ - država rojstva -, že obstaja. Če ne obstaja, ustvarimo par ključ-vrednost, sicer pa samo prištejemo 1 k obstoječi vrednosti.

Ker operiramo le z državo rojstva, lahko zanko poenostavimo:

```
1 for _, _, _, _, _, drzava_rojstva in seznam:
2     if drzava_rojstva not in slovar:
3         slovar[drzava_rojstva] = 0
4     slovar[drzava_rojstva]+=1
```

V tem primeru pri preverjanju obstoja ključa vrednost nastavimo na 0, nato pa v vsakem primeru prištejemo 1 (vrstica 4). Ker sta zamika vrstic 2 in 4 enaka, se bo 4 izvajala vedno, ne glede na pravilnost pogojnega stavka pred njo.

3. naloga

Prvi del

Napiši funkcijo `pridobi_leto`, ki kot argument prejme datum zapisan kot `str`. Pri tem je datum lahko podan v enem izmed spodnjih zapisov:

- `DD-MM-YYYY`: primer takega zapisa je `'23-01-1930'`;
- `YYYY-MM-DD`: primer takega zapisa je `'1883-03-19'`.

Pri tem `DD` predstavlja dan, `MM` mesec, `YYYY` pa leto.

Funkcija `pridobi_leto` naj iz datuma izlušči leto in tega vrne zapisanega kot `int`. Funkcija naj deluje z obema zapisoma datumov.

Primera pravilne izvedbe funkcije:

```
>>> pridobi_leto('23-01-1930')
1930
>>> pridobi_leto('1883-03-19')
1883
```

Drugi del

Napiši funkcijo `slovar_starosti`, ki kot argument sprejme seznam terk, kot ga vrača Naloga 1, tj. seznam terk oblike:
`(leto, priimek, kategorija, spol, datum_rojstva, drzava_rojstva)`.

Funkcija naj vrne slovar (`dict`), ki kot ključe vsebuje terke (`tuple`) oblike `(leto, priimek)`, kjer `leto` predstavlja leto prejema nagrade, `priimek` pa priimek nagrajenca. Kot vrednosti naj slovar vsebuje starosti nagrajencev ob prejemu nagrade. Pri računanju starosti predpostavljaj, da je nagrajenec ob prejemu nagrade tisto leto že imel rojstni dan:

$starost = leto * prejemaleto * rojstva.$

Primer pravilne izvedbe funkcije:

```
>>> s = [(1992, 'Walcott', 'literature', 'm', '23-01-1930', 'LC'),
 (1967, 'Wald', 'medicine', 'm', '18-11-1906', 'US'),
 (1945, 'Mistral', 'literature', 'f', '1889-04-07', 'CL'),
 (2020, 'Milgrom', 'economics', 'm', '20-04-1948', 'US'),
 (2016, 'Thouless', 'physics', 'm', '21-09-1934', 'GB')]
>>> slovar_starosti(s)
{(1992, 'Walcott'): 62,
 (1967, 'Wald'): 61,
 (1945, 'Mistral'): 56,
 (2020, 'Milgrom'): 72,
 (2016, 'Thouless'): 82}
```

Rešitev

Prvi del:

```
def pridobi_leto(datum):
    datum = datum.split("-")
    if len(datum[0]) == 4:
        return int(datum[0])
    return int(datum[-1])
```

Drugi del:

```
def slovar_starosti(seznam):
    slovar = {}
    for leto, priimek, _, _, datum_rojstva, _ in seznam:
        slovar[(leto, priimek)] = leto - pridobi_leto(datum_rojstva)
    return slovar
```

Rešitev z razlago

Prvi del

Datum je v obliki niza (`str`), pri čemer so časovni elementi (dan, mesec in leto) ločeni s pomišljajem. Z metodo `.split("-")` niz razbijemo, nato pa preverimo, ali je dolžina prvega elementa novonastalega seznama enaka 4. Če je, smo uspešno našli letnico, v nasprotnem primeru pa je letnica zadnji element seznama, saj je datum lahko predstavljen le na ta dva načina.

Naloge se lahko lotimo tudi z zanko `for`, s katero iteriramo po začasno ustvarjenem seznamu in vrnemo element, katerega dolžina znaša 4 znake:

```
1 def pridobi_leto(datum):
2     for i in datum.split("-"):
3         if len(i) == 4:
4             return int(i)
```

Drugi del

Najprej ustvarimo prazen slovar, nato pa se z zanko `for` pomikamo po terkah v seznamu. Tokrat bo ključ v slovarju terka, pri vrednosti pa si pomagamo s funkcijo iz prvega dela.

4. naloga

Napiši funkcijo `priimki`, ki kot argumente sprejme:

- seznam terk, kot ga vrača Naloga 1, tj. seznam terk oblike `(leto, priimek, kategorija, spol, datum_rojstva, drzava_rojstva)`,
- naziv kategorije 1 (`str`),
- naziv kategorije 2 (`str`).

Funkcija naj vrne množico (`set`) priimkov, ki se v seznamu nagrajencev pojavijo v obeh kategorijah.

Opozorilo: v primeru, da je naziv kategorije 1 enak nazivu kategorije 2 (ista kategorija je podana dvakrat), naj funkcija vrne množico vseh priimkov iz podane kategorije.

Primeri pravilne izvedbe funkcije:

```
>>> s = [(1992, 'Walcott', 'literature', 'm', '23-01-1930', 'LC'),
(1952, 'Bloch', 'physics', 'm', '23-10-1905', 'CH'),
(2005, 'Hall', 'physics', 'm', '21-08-1934', 'US'),
(1964, 'Bloch', 'medicine', 'm', '21-01-1912', 'PL'),
(2017, 'Hall', 'medicine', 'm', '03-05-1945', 'US'),
(2016, 'Thouless', 'physics', 'm', '21-09-1934', 'GB')]
>>> priimki(s, 'physics', 'medicine')
{'Bloch', 'Hall'}
>>> priimki(s, 'physics', 'physics')
{'Bloch', 'Hall', 'Thouless'}
>>> priimki(s, 'physics', 'literature')
set()
```

Rešitev

```
def priimki(seznam, kat1, kat2):
    mn1 = set()
    mn2 = set()
    for leto, priimek, kategorija, spol, datum_rojstva, drzava_rojstva in
    ↪  seznam:
        if kat1 == kategorija:
            mn1.add(priimek)
        elif kat2 == kategorija:
            mn2.add(priimek)
    if kat1 == kat2:
        return mn1
    return mn1&mn2
```

Rešitev z razlago

Naloga od nas zahteva, da poiščemo presek dveh kategorij. Za pomikanje po terkah in iskanje priimkov nagrajencev, ki spadajo v določeno kategorijo, potrebujemo zanko `for`. Če se kategorija ujema s prvim argumentom, priimek dodamo v prvo množico, če pa se ujema z drugim argumentom, priimek dodamo v drugo množico.

Če sta kategoriji enaki, je dovolj, da vrnemo prvo množico, saj je bil pogoj v vrstici 5 vedno izpolnjen, zaradi česar se pogoj v vrstici 7 (kombinacija *if-elif*) ni preverjal. Če sta bili kategoriji različni, vrnemo presek obeh množic. `return` hkrati deluje kot `break`, zato `else` v vrstici 11 ni potreben - če je pogoj v vrstici 9 izpolnjen, se funkcija zaključi (preostanek kode se ne izvede več), sicer pa se izvede vrstica 11 in se funkcija zaključi na tem mestu.

1. izpit 2025/2026

Naloge shranite v datoteko `naloge.py`.

1. naloga

Podatki o učencih programiranja se nahajajo v datoteki `csv`. Predpostavljajte, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljena vejica (`,`).

Primer vsebine datoteke `python5.csv`:

```
id,starost,drzava,izkusnje,tedni,ure,naloge,videi,samoocena,tocke,opravljeno
600,43,Indonesia,Intermediate,15,8.2,62,30,10,32.5,0
1200,52,UK,Beginner,2,4.4,62,36,5,32.1,0
1800,16,UK,Beginner,7,10.1,54,40,7,41.6,0
2400,21,Indonesia,Beginner,15,11.2,53,29,3,34.9,0
3000,36,Pakistan,Advanced,12,3.6,57,48,7,59.8,0
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(starost, drzava, izkusnje, tedni, ure, naloge, videi, ocena)`. Pri tem naj:

- `starost` predstavlja starost učenca, zapisano kot `int`,
- `drzava` predstavlja državo, iz katere prihaja učenec, zapisana kot `str`,
- `izkusnje` predstavlja prehodne izkušnje učenca, zapisane kot `str`,
- `tedni` predstavlja število tednov tečaja, ki jih je učenec obiskoval, zapisanih kot `int`,
- `ure` predstavlja število ur, ki jih je učenec na teden porabil za tečaj, zapisane kot `float` in zaokrožene na 1 decimalno mesto,
- `naloge` predstavlja število rešenih nalog, zapisanih kot `int`,
- `videi` predstavlja število ogledanih videov s tečaja, zapisanih kot `int` in
- `ocena` predstavlja končno oceno izpita, zapisano kot `int`.

Vrednost spremenljivke `ocena` dobite tako, da spremenljivki `tocke` iz datoteke pripišete številsko oceno na intervalu 5-10 po sledečem kriteriju:

- manj kot 50 točk pomeni oceno 5,
- med vključno 50 do 60 točk je ocena 6,
- med vključno 60 do 70 točk je ocena 7,
- med vključno 70 do 80 točk je ocena 8,
- med vključno 80 do 90 točk je ocena 9 in
- od vključno 90 točk do vključno 100 je ocena 10.

Vrstico z imeni stolpcev pri tvorbi seznama preskočite.

Če je v spremenljivki `drzava` zapisana vrednost `'Other'`, podatkov iz te vrstice ne vključite v seznam terk.

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke('python5.csv')
    [(43, 'Indonesia', 'Intermediate', 15, 8.2, 62, 30, 5),
     (52, 'UK', 'Beginner', 2, 4.4, 62, 36, 5),
     (16, 'UK', 'Beginner', 7, 10.1, 54, 40, 5),
     (21, 'Indonesia', 'Beginner', 15, 11.2, 53, 29, 5),
     (36, 'Pakistan', 'Advanced', 12, 3.6, 57, 48, 6)]
```

Rešitev

```
def preberi_podatke(datoteka):
    seznam = []
    with open(datoteka, "r", encoding="utf8") as f:
        f.readline()
        for vrstica in f:
            podatki = vrstica.strip().split(",")
            id, starost, drzava, izkusnje, tedni, ure, naloge, videi,
            ↪  samoocena, tocke, opravljeno = podatki

            if drzava == "Other":
                continue

            starost = int(starost)
            tedni = int(tedni)
            ure=float(ure)
            naloge=int(naloge)
            videi=int(videi)

            tocke = float(tocke)
            if tocke < 50:
                ocena = 5
            elif 50 <= tocke < 60:
                ocena = 6
            elif 60 <= tocke < 70:
                ocena = 7
            elif 70 <= tocke < 80:
                ocena = 8
            elif 80 <= tocke < 90:
                ocena = 9
            else:
                ocena = 10

            seznam.append((starost, drzava, izkusnje, tedni, ure, naloge,
            ↪  videi, ocena))
    return seznam
```

2. naloga

Napišite funkcijo `v_slovar`, ki kot argument sprejme seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(starost, drzava, izkusnje, tedni, ure, naloge, videi, ocena)`).

Funkcija naj vrača slovar, kjer so ključi ocene, vrednosti pa seznami terk oblike `(starost, drzava, izkusnje, tedni, ure, naloge, videi)`.

Primer pravilne izvedbe funkcije:

```
>>> podatki = [(43, 'Indonesia', 'Intermediate', 15, 8.2, 62, 30, 5),
     (52, 'UK', 'Beginner', 2, 4.4, 62, 36, 5),
     (16, 'UK', 'Beginner', 7, 10.1, 54, 40, 5),
     (21, 'Indonesia', 'Beginner', 15, 11.2, 53, 29, 5),
     (36, 'Pakistan', 'Advanced', 12, 3.6, 57, 48, 6)]
>>> v_slovar(podatki)
    {5: [(43, 'Indonesia', 'Intermediate', 15, 8.2, 62, 30),
      (52, 'UK', 'Beginner', 2, 4.4, 62, 36),
      (16, 'UK', 'Beginner', 7, 10.1, 54, 40),
      (21, 'Indonesia', 'Beginner', 15, 11.2, 53, 29)],
     6: [(36, 'Pakistan', 'Advanced', 12, 3.6, 57, 48)]}
```

Rešitev

```
1 def v_slovar(podatki):
2     slovar = {}
3     for starost, drzava, izkusnje, tedni, ure, naloge, videi, ocena in podatki:
4         terka = (starost,drzava,izkusnje,tedni,ure,naloge,videi)
5         if ocena not in slovar:
6             slovar[ocena] = []
7         slovar[ocena].append(terka)
8     return slovar
```

3. naloga

Napiši funkcijo `statistika`, ki kot sprejme slovar, kot ga vrača 2. naloga (tj. slovar, kjer kot ključi nastopajo ocene, vrednosti pa so seznami terk oblike
`(starost, drzava, izkusnje, tedni, ure, naloge, videi)`. Funkcija naj vrača slovar, kjer kot ključi nastopajo ocene, kot vrednosti pa povprečno število vloženih ur. Povprečja naj bodo zaokrožena na dve decimalni mesti.

Primer pravilne izvedbe funkcije:

```
>>> slovar = {5: [(43, 'Indonesia', 'Intermediate', 15, 8.2, 62, 30),
      (52, 'UK', 'Beginner', 2, 4.4, 62, 36),
      (16, 'UK', 'Beginner', 7, 10.1, 54, 40),
      (21, 'Indonesia', 'Beginner', 15, 11.2, 53, 29)],
     6: [(36, 'Pakistan', 'Advanced', 12, 3.6, 57, 48)]}
>>> statistika(slovar)
    {5: 8.47, 6: 3.6}
```

Rešitev

```
1  def statistika(slovar):
2      slovar2={}
3
4      for ocena, seznam in slovar.items():
5          sez_ur = []
6
7          for x in seznam:
8              sez_ur.append(x[4])
9
10         slovar2[ocena]=round(sum(sez_ur)/len(sez_ur),2)
11
12     return slovar2
```

4. naloga

Napišite funkcijo `po_izkusnjah`, ki kot argument sprejme seznam terk oblike `(starost, drzava, izkusnje, tedni, ure, naloge, videi, ocena)` kot ga vrača 1. naloga in niz, ki predstavlja nivo izkušenj tekmovalcev.

Funkcija naj vrača ime države (`str`), iz katere prihaja največ tekmovalcev s podanim nivojem izkušenj. Predpostavljate lahko, da je v posameznem primeru vedno največ ena taka država.

V primeru, da funkciji podamo prazen seznam ali v seznamu ni nobenega tekmovalca s podanim nivojem izkušenj, naj funkcija vrača prazen niz.

Namig: v prvem koraku si pripravite slovar, v katerem so ključi države, vrednosti pa števila tekmovalcev s podanim nivojem izkušenj, ki prihajajo iz določene države.

Primer pravilne izvedbe funkcije:

```
>>> podatki = [(52, 'UK', 'Beginner', 2, 4.4, 62, 36, 5),
    (16, 'UK', 'Beginner', 7, 10.1, 54, 40, 5),
    (21, 'Indonesia', 'Beginner', 15, 11.2, 53, 29, 5),
    (36, 'Pakistan', 'Advanced', 12, 3.6, 57, 48, 6)]
>>> po_izkusnjah(podatki, 'Beginner')
    'UK'
>>> po_izkusnjah(podatki, 'Zacetnik')
    ''
```

Rešitev

```
1  def po_izkusnjah(podatki, izkusnje):
2      slovar = {}
3      for x in podatki:
4          drzava = x[1]
5          izkusnje2 = x[2]
6          if izkusnje2 == izkusnje:
7              if drzava in slovar:
8                  slovar[drzava] += 1
9              else:
10                 slovar[drzava] = 1
11
12     naj_drzava=""
13     naj_izkusnje=0
14     for drzava, izkusnje2 in slovar.items():
15         if izkusnje2 > naj_izkusnje:
16             naj_izkusnje = izkusnje2
17             naj_drzava = drzava
18     return naj_drzava
```

2. izpit 2025/2026

Naloge shranite v datoteko `naloge.py`.

1. naloga

Podatki o nakupih se nahajajo v datoteki `csv`. Predpostavljajte, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljena vejica (`,`).

```
starost,spol,izdelek,kategorija,cena,lokacija,velikost,barva,sezona,ocena,
↪  popust,placilo,pogostost
59,Female,Coat,Outerwear,75,Texas,M,Black,Summer,4.8,No,PayPal,Quarterly
60,Male,Shirt,Clothing,76,Rhode Island,XL,Teal,Spring,4.7,No,Cash,Annually
31,Female,Socks,Clothing,43,Utah,M,Silver,Fall,3.6,No,PayPal,Quarterly
40,Male,Hat,Accessories,45,Iowa,M,Silver,Fall,4.4,No,PayPal,Fortnightly
61,Female,T-shirt,Clothing,57,Vermont,M,Gray,Summer,2.8,No,Cash,Weekly
#63,Female,Skirt,Clothing,90,Oregon,L,Cyan,Spring,2.9,No,Venmo,Every 3 Months
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(spol, kategorija, izdelek, velikost, cena, ocena)`. Pri tem naj:

- `spol` predstavlja spol kupca, zapisan kot `'M'` ali `'F'`,
- `kategorija` predstavlja kategorijo izdelka, zapisana kot `str`,
- `izdelek` predstavlja ime izdelka, zapisano kot `str`,
- `velikost` predstavlja velikost izdelka, zapisano kot `str`,
- `cena` predstavlja ceno izdelka, zapisano kot `int` in
- `ocena` predstavlja oceno izdelka, zapisano kot `float`.

Vrednost spremenljivke `spol` določite tako, da spol spremenite v `'M'`, če je v datoteki zapisana vrednost `'Male'`, in v `'F'`, če je v datoteki zapisana vrednost `'Female'`.

V datoteki se lahko pojavijo tudi vrstice, ki se začnejo z znakom `#` - take vrstice pri obdelavi podatkov preskočite. Prav tako preskočite vrstico z imeni stolpcev.

Primer pravilne izvedbe funkcije:

```
>>> preberi_podatke('data/shop5.csv')
    [('F', 'Outerwear', 'Coat', 'M', 75, 4.8),
     ('M', 'Clothing', 'Shirt', 'XL', 76, 4.7),
     ('F', 'Clothing', 'Socks', 'M', 43, 3.6),
     ('M', 'Accessories', 'Hat', 'M', 45, 4.4),
     ('F', 'Clothing', 'T-shirt', 'M', 57, 2.8)]
```

Rešitev

```python
def preberi_podatke(datoteka):
    with open(datoteka, "r",encoding="utf8") as f:
        f.readline()
        seznam = []
        for vrstica in f:

            if vrstica.startswith("#"):
                continue

            starost,spol,izdelek,kategorija,cena,lokacija,velikost, barva,
            ↪  sezona, ocena, popust, placilo, pogostost =
            ↪  vrstica.strip().split(",")

            starost = int(starost)

            if spol == "Male":
                spol = "M"
            else:
                spol = "F"

            cena = int(cena)
            ocena = float(ocena)

            seznam.append((spol, kategorija, izdelek, velikost, cena, ocena))

        return seznam
```

2. naloga

Napišite funkcijo `pripravi_slovar`, ki kot argumente sprejme:

- seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(spol, kategorija, izdelek, velikost, cena, ocena)`),
- indeks elementa v terki, ki predstavlja ključ slovarja,
- indeks elementa v terki, ki predstavlja vrednost slovarja.

Funkcija naj vrača slovar, kjer so ključi vrednosti elementov na mestu z indeksom drugega argumenta, vrednosti pa seznami elementov na mestu z indeksom tretjega argumenta funkcije.

Če je podani seznam terk prazen, naj funkcija **vrne** niz `"Slovarja ni mogoce pripraviti."`

Če je katerikoli izmed podanih indeksov neveljaven, naj funkcija **vrne** niz `"Neveljaven indeks."`.

Primer pravilne izvedbe funkcije:

```
>>> podatki = [('F', 'Outerwear', 'Coat', 'M', 75, 4.8),
     ('M', 'Clothing', 'Shirt', 'XL', 76, 4.7),
     ('F', 'Clothing', 'Socks', 'M', 43, 3.6),
     ('M', 'Accessories', 'Hat', 'M', 45, 4.4),
     ('F', 'Clothing', 'T-shirt', 'M', 57, 2.8)]
>>> pripravi_slovar(podatki, 1, 2)
    {'Outerwear': ['Coat'],
    'Clothing': ['Shirt', 'Socks', 'T-shirt'],
    'Accessories': ['Hat']}
```

Rešitev

```python
def pripravi_slovar(podatki, idx_kljuc, idx_vrednost):
    slovar = {}

    if podatki == []:
        return "Slovarja ni mogoče pripraviti."

    for terka in podatki:
        if idx_kljuc >= len(terka) or idx_vrednost >= len(terka):
            return "Neveljaven indeks."

        kljuc= terka[idx_kljuc]
        vrednost=terka[idx_vrednost]

        if kljuc not in slovar:
            slovar[kljuc] = []
        slovar[kljuc].append(vrednost)

    return slovar
```

3. naloga

Napišite funkcijo `najvec_vrednosti`, ki kot argument sprejme slovar, kjer so ključi poljubne vrednosti, vrednosti pa seznami elementov.

Funkcija naj vrne terko oblike `(kljuc, vrednost)`, kjer `kljuc` predstavlja ključ z vrednostjo, ki ima največ elementov v slovarju, `vrednost` pa pripadajočo vrednost.

Če je slovar prazen, naj funkcija **vrne** `None`. Pazite, da ne vračate niza `'None'`.

Predpostavljate lahko, da največ vrednosti vsebuje le en ključ v celotnem slovarju.

Primer pravilne izvedbe funkcije:

```
>>> slovar = {'Outerwear': ['Coat'],
    'Clothing': ['Shirt', 'Socks', 'T-shirt'],
    'Accessories': ['Hat']}
>>> najvec_vrednosti(slovar)
    ('Clothing', ['Shirt', 'Socks', 'T-shirt'])
```

Rešitev

```python
def najvec_vrednosti(slovar):
    najvec = 0
    naj_kljuc = None

    for kljuc in slovar:
        if len(slovar[kljuc]) > najvec:
            najvec = len(slovar[kljuc])
            naj_kljuc = kljuc

    if naj_kljuc != None:
        return (naj_kljuc, slovar[naj_kljuc])
    else:
        return None
```

4. naloga

Napišite funkcijo `po_spolih`, ki kot argument sprejme seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(spol, kategorija, izdelek, velikost, cena, ocena)`) ter opcijski argument `nacin`, ki ima privzeto vrednost `"skupno"`.

Funkcija naj glede na vrednost argumenta `nacin` vrača:

- množico izdelkov, ki jih kupujejo izključno ženske (razlika množic) v načinu `nacin = 'f'`,
- množico izdelkov, ki jih kupujejo izključno moški (razlika množic) v načinu `nacin = 'm'`,
- sicer množico izdelkov, ki jih kupujejo tako ženske kot moški (presek, `nacin = 'skupno'`.

Primer pravilne izvedbe funkcije:

```
>>> podatki = [('M', 'Footwear', 'Sneakers', 'M', 20, 2.9),
     ('F', 'Clothing', 'Socks', 'L', 41, 4.3),
     ('M', 'Clothing', 'Sweater', 'L', 85, 3.7),
     ('M', 'Accessories', 'Gloves', 'M', 71, 3.2),
     ('M', 'Clothing', 'T-shirt', 'L', 31, 4.8),
     ('F', 'Clothing', 'Sweater', 'M', 26, 4.8)]
>>> po_spolih (podatki, nacin = 'skupno')
    {'Sweater'}
>>> po_spolih(podatki, nacin = 'f')
    {'Socks'}
>>> po_spolih(podatki, nacin = 'm')
    {'Gloves', 'Sneakers', 'T-shirt'}
```

Rešitev

```python
def po_spolih(podatki, nacin="skupno"):

    f = set()
    m = set()

    for spol, kategorija, izdelek, velikost, cena, ocena in podatki:
        if spol == "F":
            f.add(izdelek)
        if spol == "M":
            m.add(izdelek)

    if nacin == "f":
        return f-m
    elif nacin == "m":
        return m-f
    else:
        return f&m
```

3. izpit 2025/2026

Naloge shranite v datoteko `naloge.py`. Naloge niso razvrščene po težavnosti in so neodvisne druga od druge.

1. naloga

Podatki o naročilih v kavarni se nahajajo v datoteki `csv`. Predpostavljajte, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljena vejica (`,`), prva vrstica pa predstavlja glave stolpcev.

Primer vsebine datoteke `kava5.csv`:

```
id,starost,spol,izdelek,velikost,cena,ocena,placilo,lokacija,clan
501,25,Female,Cappuccino,M,3.5,4.7,Card,Ljubljana,Yes
502,34,Male,Latte,L,4.0,4.2,Cash,Maribor,No
503,19,Female,Espresso,S,2.5,4.9,Card,Ljubljana,Yes
504,45,Male,Cappuccino,L,3.8,3.9,App,Celje,No
505,29,Female,Cold Brew,M,3.9,4.5,Card,Unknown,Yes
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(starost, spol, izdelek, velikost, cena, ocena)`. Pri tem naj:

- `starost` predstavlja starost kupca, zapisano kot `int`,
- `spol` predstavlja spol kupca, zapisan kot `'M'` ali `'F'`,
- `izdelek` predstavlja ime naročene kave, zapisano kot `str`,
- `velikost` predstavlja izbrano velikost napitka, zapisano kot `str`,
- `cena` predstavlja ceno napitka, zapisano kot `float` in zaokroženo na 2 decimalni mesti ter
- `ocena` predstavlja oceno naročila, zapisano kot `float`.

Vrednost spremenljivke `spol` določite tako, da spol spremenite v `'M'`, če je v datoteki zapisana vrednost `'Male'`, in v `'F'`, če je v datoteki zapisana vrednost `'Female'`.

Če je v spremenljivki `lokacija` zapisana vrednost `'Unknown'`, podatkov iz te vrstice ne vključite v seznam.

Primer pravilne izvedbe funkcije:

```
>>> preberi_podatke('data/kava5.csv')
    [(25, 'F', 'Cappuccino', 'M', 3.5, 4.7),
     (34, 'M', 'Latte', 'L', 4.0, 4.2),
     (19, 'F', 'Espresso', 'S', 2.5, 4.9),
     (45, 'M', 'Cappuccino', 'L', 3.8, 3.9)]
```

```
    [(25, 'F', 'Cappuccino', 'M', 3.5, 4.7), (34, 'M', 'Latte', 'L', 4.0,
    ↪  4.2), (19, 'F', 'Espresso', 'S', 2.5, 4.9), (45, 'M', 'Cappuccino',
    ↪  'L', 3.8, 3.9)]
```

Rešitev

```python
def preberi_podatke(datoteka):
    seznam = []
    with open(datoteka, "r", encoding="utf8") as f:
        f.readline()
        for vrstica in f:
            podatki = vrstica.strip().split(",")
            zap_st, starost, spol, izdelek, velikost, cena, ocena, placilo,
            ↪  lokacija, clan = podatki

            if lokacija == "Unknown":
                continue

            starost = int(starost)

            if spol == "Female":
                spol = "F"
            else:
                spol = "M"

            cena = round(float(cena), 2)
            ocena = float(ocena)

            seznam.append((starost, spol, izdelek, velikost, cena, ocena))
    return seznam
```

2. naloga

Napišite funkcijo `v_slovar`, ki kot argument sprejme seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(starost, spol, izdelek, velikost, cena, ocena)`), ter opcijski argument `kljuc`, ki ima privzeto vrednost `False`.

Če je vrednost argumenta `kljuc` enaka `False`, naj funkcija vrača slovar, kjer so ključi imena izdelkov, vrednosti pa seznami terk oblike `(starost, spol, velikost, cena, ocena)`. Če pa je vrednost argumenta `kljuc` enaka `True`, pa naj bosta ključa slovarja oznaki za spola, vrednosti pa seznami terk oblike `(starost, izdelek, velikost, cena, ocena)`.

V primeru, da funkciji podamo prazen seznam terk, naj funkcija vrača prazen slovar.

Namig: najprej si pripravite prazen slovar.

Primer pravilne izvedbe funkcije:

```
>>> podatki = [(25, 'F', 'Cappuccino', 'M', 3.5, 4.7),
    (34, 'M', 'Latte', 'L', 4.0, 4.2),
    (19, 'F', 'Espresso', 'S', 2.5, 4.9),
    (45, 'M', 'Cappuccino', 'L', 3.8, 3.9)]
>>> v_slovar(podatki)
    {'Cappuccino': [(25, 'F', 'M', 3.5, 4.7), (45, 'M', 'L', 3.8, 3.9)],
     'Latte': [(34, 'M', 'L', 4.0, 4.2)],
     'Espresso': [(19, 'F', 'S', 2.5, 4.9)]}
>>> v_slovar(podatki, kljuc=True)
    {'F': [(25, 'Cappuccino', 'M', 3.5, 4.7), (19, 'Espresso', 'S', 2.5, 4.9)],
     'M': [(34, 'Latte', 'L', 4.0, 4.2), (45, 'Cappuccino', 'L', 3.8, 3.9)]}
```

Rešitev

```
def v_slovar(podatki, kljuc=False):
    slovar = {}
    for starost, spol, izdelek, velikost, cena, ocena in podatki:
        if kljuc == False:
            k = izdelek
            terka = (starost, spol, velikost, cena, ocena)
        else:
            k = spol
            terka = (starost, izdelek, velikost, cena, ocena)

        if k not in slovar:
            slovar[k] = []
        slovar[k].append(terka)
    return slovar
```

3. naloga

Napišite funkcijo `statistika`, ki kot argument sprejme slovar, kot ga vrača 2. naloga s `kljuc = False` (tj. slovar, kjer so ključi imena izdelkov, vrednosti pa seznami terk oblike `(starost, spol, velikost, cena, ocena)`).

Funkcija naj vrača slovar, kjer so ključi enaki kot v vhodnem slovarju, kot vrednosti pa nastopajo povprečne cene posameznega izdelka. Povprečja naj bodo zaokrožena na dve decimalni mesti.

Primera pravilne izvedbe funkcije:

```
>>> slovar = {
    'Cappuccino': [(25, 'F', 'M', 3.5, 4.7), (45, 'M', 'L', 3.8, 3.9)],
     'Latte': [(34, 'M', 'L', 4.0, 4.2)],
     'Espresso': [(19, 'F', 'S', 2.5, 4.9)]}
>>> statistika(slovar)
    {'Cappuccino': 3.65, 'Latte': 4.0, 'Espresso': 2.5}
>>> statistika({})
    {}
```

Rešitev

```python
def statistika(slovar):
    slovar2 = {}

    for izdelek, seznam in slovar.items():
        cene = []
        for _, _, _, cena, _ in seznam:
            cene.append(cena)
        slovar2[izdelek] = round(sum(cene) / len(cene), 2)

    return slovar2
```

4. naloga

Napišite funkcijo `po_spolu`, ki kot argument sprejme slovar, kot ga vrača 2. naloga s `kljuc=True` (tj. slovar, kjer sta ključa oznaki za spola, vrednosti pa seznami terk oblike `(starost, izdelek, velikost, cena, ocena)`).

Funkcija naj vrne slovar s tremi ključi:

- `'skupno'`: množica izdelkov, ki jih naročajo tako ženske kot moški (presek),
- `'samo_m'`: množica izdelkov, ki jih naročajo izključno moški (razlika množic) in
- `'samo_f'`: množica izdelkov, ki jih naročajo izključno ženske (razlika množic).

Predpostavite lahko, da manjkajoč ključ (`'M'` ali `'F'`) v vhodnem slovarju pomeni, da za ta spol ni bilo nobenih naročil.

Primera pravilne izvedbe funkcije:

```
>>> slovar =
    {'F': [(25, 'Cappuccino', 'M', 3.5, 4.7), (19, 'Espresso', 'S', 2.5, 4.9)],
     'M': [(34, 'Latte', 'L', 4.0, 4.2), (45, 'Cappuccino', 'L', 3.8, 3.9)]}
>>> po_spolu(slovar)
    {'skupno': {'Cappuccino'}, 'samo_m': {'Latte'}, 'samo_f': {'Espresso'}}
```

```
>>> slovar2 =
    {'M': [(30, 'Latte', 'L', 4.57, 4.0), (50, 'Mocha', 'S', 3.0, 3.5)]}
>>> po_spolu(slovar2)
    {'skupno': set(), 'samo_m': {'Latte', 'Mocha'}, 'samo_f': set()}
```

Rešitev

```python
def po_spolu(slovar):
    slovar2 = {}

    m_izdelki = set()
    z_izdelki = set()

    for key, terka in slovar.items():
        if key == "M":
            for terka in slovar["M"]:
                m_izdelki.add(terka[1])
        elif key == "F":
            for terka in slovar["F"]:
             z_izdelki.add(terka[1])

    slovar2["skupno"] = m_izdelki & z_izdelki
    slovar2["samo_m"] = m_izdelki - z_izdelki
    slovar2["samo_f"] = z_izdelki - m_izdelki

    return slovar2
```

1. izpit 2024/2025

Naloge shranite v datoteko z imenom `naloge.py`.

1. naloga

Podatki o lastnostih izbranih skladb, ki jih najdemo v aplikaciji Spotify, so zapisani v tekstovni datoteki. Predpostavljaj, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljeno podpičje (`;`), prva vrstica pa predstavlja glave stolpcev.

Primer vsebine datoteke `spotify5.txt`:

```
naslov;izvajalec;leto;priljubljenost;album;ples;zivost;energicnost;trajanje
Right As Rain;Adele;2008;54;19;0.842;0.0891;0.679;197347
One Way Street;Aerosmith;1973;41;Aerosmith;0.457;0.115;0.834;422607
Sweet Emotion;Aerosmith;1975;75;Toys In The Attic;0.379;0.104;0.76;274088
Party;Alan Silvestri;1991;1;Dutch (Original Motion Picture
↪  Score);0.469;0.121;0.383;337173
Canyons;Avicii;2013;35;True (Bonus Edition);0.663;0.613;0.682;449360
```

Napišite funkcijo preberi_podatke, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike:
`(naslov, izvajalec, leto, priljubljenost, ples, zivost, energicnost, sekunde)`.

Pri tem naj:

- `naslov` predstavlja naslov skladbe, zapisan kot `str`,
- `izvajalec` predstavlja ime izvajalca, zapisano kot `str`,
- `leto` predstavlja leto izida skladbe, zapisano kot `int`,
- `priljubljenost` predstavlja oceno priljubljenosti skladbe, zapisane kot `int`,
- `ples` predstavlja primernost skladbe za ples, zapisano kot `float`,
- `zivost` predstavlja število, zapisano kot `float`,
- `energicnost` predstavlja faktor, zapisan kot `float`,
- `sekunde` predstavlja dolžino skladbe v sekundah.

Vsa števila, zapisana kot `float`, naj bodo zaokrožena na 3 decimalna mesta.

V primeru, da je naslov daljši od 20 znakov, ga skrajšajte na 20 znakov in na konec dodajte tri pike (`...`).

Dolžino skladbe iz milisekund pretvorite v sekunde po formuli:

$sekunde = trajanje * 0.001$

Sekunde naj bodo v obliki celih števil (`int`) - za pretvorbo uporabite funkcijo `int`.

Predpostavljajte, da lahko katerakoli izmed številskih spremenljivk v posamezni vrstici datoteke manjka. V tem primeru takšne vrstice ne vključite v seznam terk.

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke("spotify5.txt")
    [('Right As Rain', 'Adele', 2008, 54, 0.842, 0.089, 0.679, 197),
    ('One Way Street', 'Aerosmith', 1973, 41, 0.457, 0.115, 0.834, 422),
    'Sweet Emotion', 'Aerosmith', 1975, 75, 0.379, 0.104, 0.76, 274),
    ('Party', 'Alan Silvestri', 1991, 1, 0.469, 0.121, 0.383, 337),
    ('Canyons', 'Avicii', 2013, 35, 0.663, 0.613, 0.682, 449)]
>>> preberi_podatke("spotify_missing.txt")
    [('The Scientist', 'Coldplay', 2024, 2, 0.557, 0.11, 0.442, 309),
    ('Ronin', 'Hans Zimmer', 2003, 37, 0.207, 0.082, 0.077, 113),
    ('Open Your Heart', 'Madonna', 1986, 51, 0.624, 0.195, 0.951, 253),
    ('Antologia', 'Shakira', 2002, 56, 0.844, 0.095, 0.465, 251),
    ('Bad Blood', 'Taylor Swift', 2024, 7, 0.656, 0.201, 0.792, 211)]
>>> preberi_podatke("spotify_long.txt")
    [('Hometown Glory', 'Adele', 2008, 61, 0.436, 0.102, 0.336, 271),
    ('Melt My Heart to Sto...', 'Adele', 2008, 53, 0.353, 0.215, 0.388, 203),
    ("You'll Be Queen One ...", 'Ramin Djawadi', 2011, 40, 0.279, 0.083, 0.006,
    ↪  95),
    ('Victory Does Not Mak...', 'Ramin Djawadi', 2011, 38, 0.125, 0.103, 0.021,
    ↪  96),
    ('Vigilante Shit', 'Taylor Swift', 2023, 66, 0.867, 0.118, 0.272, 164)]
```

Rešitev

Možnost 1:

```
def preberi_podatke(datoteka):
    seznam = []
    f = open(datoteka, "r", encoding="utf8")
    f.readline()
    for vrstica in f:
        podatki = vrstica.strip().split(";")

        if "" in podatki:
            continue

        naslov, izvajalec, leto, priljubljenost, album, ples, zivost,
        ↪  energicnost, trajanje = podatki

        ples = round(float(ples),3)
        zivost = round(float(zivost),3)
        energicnost = round(float(energicnost),3)

        if len(naslov) > 20:
            naslov = naslov[:20] + "..."

        trajanje = int(trajanje)
        sekunde = int(trajanje*0.001)

        seznam.append((naslov, izvajalec, int(leto), int(priljubljenost), ples,
        ↪  zivost, energicnost, sekunde))
    f.close()
    return seznam
```

Možnost 2:

```
def preberi_podatke(datoteka):
    seznam = []
    f = open(datoteka, "r", encoding="utf8")
    f.readline()
    for vrstica in f:
        podatki = vrstica.strip().split(";")

        if "" not in podatki:

            podatki[-1] = int(podatki[-1])*0.001

            for i in range(len(podatki)):
                if i in [2,3,8]:
                    podatki[i] = int(podatki[i])
                elif i in [5,6,7]:
                    podatki[i] = round(float(podatki[i]),3)

            naslov, izvajalec, leto, priljubljenost, album, ples, zivost,
            ↪ energicnost, trajanje = podatki

            if len(naslov) > 20:
                naslov = naslov[:20] + "..."

            seznam.append((naslov, izvajalec, leto, priljubljenost, ples,
            ↪ zivost, energicnost, sekunde))
    f.close()
    return seznam
```

2. naloga

Napišite funkcijo `dolzina_skladb`, ki kot argumenta sprejme seznam terk, kot ga vrača funckija `preberi_podatke` (1. naloga, tj. seznam terk oblike:
`(naslov, izvajalec, leto, priljubljenost, ples, zivost, energicnost, trajanje)`)
in ime izvajalca, zapisano kot niz (`str`).

Funkcija naj vrača skupno dolžino trajanja skladb izbranega izvajalca v obliki terke `(minute, sekunde)`, kjer `minute` predstavljajo skupno dolžino v minutah, `sekunde` pa številko, manjšo od 60.

Primera pravilno izvedene funkcije:

```
>>> podatki = [('One Way Street', 'Aerosmith', 1973, 41, 0.457, 0.115, 0.834,
↪  422),
    ('It Will Rain (From T...', 'Bruno Mars', 2024, 5, 0.572, 0.082, 0.835,
    ↪  257),
    ("You'll Be Queen One ...", 'Ramin Djawadi', 2011, 40, 0.279, 0.083,
    ↪  0.0065, 95),
    ('The Shadows Betray Y...', 'Hans Zimmer', 2012, 31, 0.534, 0.1, 0.351,
    ↪  320),
     ('Goodbye Brother', 'Ramin Djawadi', 2011, 44, 0.159, 0.118, 0.0272, 187),
    ('Bad Blood', 'Taylor Swift', 2024, 7, 0.656, 0.201, 0.792, 211)]
>>> dolzina_skladb(podatki, "Taylor Swift")
    (3, 31)
>>> dolzina_skladb(podatki, "Ramin Djawadi")
    (4, 42)
```

Rešitev

```python
def dolzina_skladb(seznam, pevec):
    vsota = 0
    for _, izvajalec, _, _, _, _, _, trajanje in seznam:
        if izvajalec == pevec:
            vsota += trajanje

    minute = vsota // 60
    sekunde = vsota % 60

    return (minute, sekunde)
```

3. naloga

Napišite funkcijo `tocke`, ki kot argumenta sprejme seznam terk, kot ga vrača funkcija `preberi_podatke` (1. naloga, tj. seznam terk oblike `(naslov, izvajalec, leto, priljubljenost, ples, zivost, energicnost, trajanje)`) in seznam imen izvajalcev, zapisanih kot niz (`str`).

Funkcija naj vrača slovar, kjer so ključi izvajalci iz seznama, vrednosti pa povprečja primernosti vseh skladb danega izvajalca za ples (povprečja spremenljivke `ples` posameznega izvajalca). Povprečja naj bodo zaokrožena na dve decimalni mesti natančno. Če v podatkih ni nobene skladbe nekega izvajalca, naj bo pripadajoča vrednost v slovarju 0.

Namig: najprej si pripravite slovar, v katerem kot ključi nastopajo izvajalci, kot vrednosti pa seznam plesnosti.

Primera pravilno izvedene funkcije:

```
>>> podatki = [('One Way Street', 'Aerosmith', 1973, 41, 0.457, 0.115, 0.834,
↪  422),
    ('It Will Rain (From T...', 'Bruno Mars', 2024, 5, 0.572, 0.082, 0.835,
    ↪  257),
     ('Somebody', 'Aerosmith', 1973, 38, 0.562, 0.373, 0.812, 225),
     ('Drunk On Love', 'Rihanna', 2012, 41, 0.655, 0.401, 0.873, 212),
   ('Where Have You Been ...', 'Rihanna', 2012, 33, 0.727, 0.0697, 0.885, 361),
    ('Bad Blood', 'Taylor Swift', 2024, 7, 0.656, 0.201, 0.792, 211)]
>>> tocke(podatki, ["Aerosmith", "Bruno Mars"])
    {'Aerosmith': 0.51, 'Bruno Mars': 0.57}
>>> tocke(podatki, ["Rihanna", "Beyonce"])
    {'Rihanna': 0.69, 'Beyonce': 0}
```

Rešitev

Možnost 1:

```
def tocke(seznam,izvajalci):
    slovar = {}

    for izvajalec in izvajalci:
        slovar[izvajalec] = []

    for _, izvajalec, _, _, ples, _, _, _ in seznam:
        if izvajalec in izvajalci:
            slovar[izvajalec].append(ples)

    for kljuc in slovar:
        if slovar[kljuc] == []:
            slovar[kljuc] = 0
        else:
            slovar[kljuc] = round(sum(slovar[kljuc])/len(slovar[kljuc]),2)

    return slovar
```

Možnost 2:

```
def tocke(seznam,izvajalci):
    slovar = {}

    for _, izvajalec, _, _, ples, _, _, _ in seznam:
        if izvajalec in izvajalci and izvajalec not in slovar:
            slovar[izvajalec] = [ples]
        elif izvajalec in slovar:
            slovar[izvajalec].append(ples)

    for kljuc in slovar:
        slovar[kljuc] = round(sum(slovar[kljuc])/len(slovar[kljuc]),2)

    for izvajalec in izvajalci:
        if izvajalec not in slovar.keys():
            slovar[izvajalec] = 0

    return slovar
```

4. naloga

Osnova

Napišite funkcijo `izpis_skladb`, ki kot argumenta sprejme seznam terk, kot ga vrača funckija `preberi_podatke` (1. naloga, tj. seznam terk oblike
`(naslov, izvajalec, leto, priljubljenost, ples, zivost, energicnost, trajanje)`)
in leto, zapisano kot `int`.

Funkcija naj vrača seznam naslovov skladb, ki so nastale v podanem letu.

Primera pravilne izvedbe funkcije:

```
>>> podatki = [('One Way Street', 'Aerosmith', 1973, 41, 0.457, 0.115, 0.834,
↪  422),
    ('It Will Rain (From T...', 'Bruno Mars', 2024, 5, 0.572, 0.082, 0.835,
    ↪  257),
   ('Somebody', 'Aerosmith', 1973, 38, 0.562, 0.373, 0.812, 225),
   ('Drunk On Love', 'Rihanna', 2012, 41, 0.655, 0.401, 0.873, 212),
   ('Bad Blood', 'Taylor Swift', 2024, 7, 0.656, 0.201, 0.792, 211)]
>>> izpis_skladb(podatki, 2024)
    ['It Will Rain (From T...', 'Bad Blood']
>>> izpis_skladb(podatki, 1990)
    []
```

Dodatek

Funkcija naj sprejme še izbirni argument `cas`. V primeru, da je ta argument pri klicu funkcije podan, naj funkcija vrača le skladbe, ki so enako dolge ali daljše od vrednosti izbirnega argumenta. Če izbirni argument ni podan, naj funkcija deluje kot opisano zgoraj.

Primera pravilne izvedbe funkcije:

```
>>> podatki = [('One Way Street', 'Aerosmith', 1973, 41, 0.457, 0.115, 0.834,
↪  422),
    ('It Will Rain (From T...', 'Bruno Mars', 2024, 5, 0.572, 0.082, 0.835,
    ↪  257),
   ('Somebody', 'Aerosmith', 1973, 38, 0.562, 0.373, 0.812, 225),
   ('Drunk On Love', 'Rihanna', 2012, 41, 0.655, 0.401, 0.873, 212),
   ('Bad Blood', 'Taylor Swift', 2024, 7, 0.656, 0.201, 0.792, 211)]
>>> izpis_skladb(podatki, 1973, cas = 225)
    ['One Way Street', 'Somebody']
>>> izpis_skladb(podatki, 2024, cas = 250)
    ['It Will Rain (From T...']
>>> izpis_skladb(podatki, 2024, cas = 500)
    []
```

Rešitev

Osnova:

```
def izpis_skladb(seznam, letnica):
    s = []
    for naslov, _, leto, _, _, _, _, trajanje in seznam:
        if leto == letnica:
            s.append(naslov)
    return s
```

Dodatek:

```
def izpis_skladb(seznam, letnica, cas = 0):
    s = []
    for naslov, _, leto, _, _, _, _, trajanje in seznam:
        if leto == letnica and cas == 0:
            s.append(naslov)
        elif leto == letnica and trajanje >= cas:
            s.append(naslov)
    return s
```

2. izpit 2024/2025

Naloge shranite v datoteko z imenom `naloge.py`.

1. naloga

Podatki o lastnostih izbranih aplikacij iz Trgovine GooglePlay, so zapisani v tekstovni datoteki. Predpostavljaj, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljeno podpičje (`;`).

Primer vsebine datoteke `apps5.txt`:

```
ime;kategorija;ocena;st_ocen;st_nalaganj;max_nalaganj;brezplacna;cena;valuta;v
↪ elikost;update;publika;oglasi;nakup
Netflix;Entertainment;4.4;11694633;1000000000;1437250170;TRUE;0;USD;Varies with
↪ device;14.6.21;Teen;FALSE;TRUE
Kobo Books - eBooks & Audiobooks;Books &
↪ Reference;4.3;270245;10000000;23837841;TRUE;0;USD;Varies with
↪ device;10.6.21;Teen;FALSE;FALSE
Headspace: Meditation & Sleep;Health & Fitness;4.6;229799;10000000;28900621;TR
↪ UE;0;USD;22M;15.6.21;Everyone;FALSE;TRUE
Simon's Cat Dash;Casual;4.6;26662;1000000;1492894;TRUE;0;USD;82M;19.2.19;Every
↪ one;TRUE;TRUE
Slack;Business;4.2;102572;10000000;23142235;TRUE;0;USD;Varies with
↪ device;14.6.21;Everyone;FALSE;FALSE
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(ime, kategorija, ocena, st_ocen, brezplacna, update, publika)`. Pri tem naj:

- `ime` predstavlja ime aplikacije, zapisano kot `str`,
- `kategorija` predstavlja kategorijo, v katero uvrščamo aplikacijo, zapisana kot `str`,
- `ocena` predstavlja povprečno uporabniško oceno aplikacije, zapisano kot `float`,
- `st_ocen` predstavlja število ocen aplikacije, zapisano kot `int`,
- `brezplacna` povzema brezplačnost aplikacije, zapisano kot `bool`,
- `update` predstavlja datum zadnje razpoložljive nadgradnje, zapisan kot `int`,
- `publika` predstavlja ciljno občinstvo aplikacije, zapisano kot `str`.

Dodatna navodila:

- če se v imenu aplikacije pojavljata bodisi `-` ali `:`, skrajšajte ime do tega znaka, poleg tega pa odstranite vse potencialne presledke na koncu imena (primer: ime `Kobo Books - eBooks & Audiobooks` naj bo v seznamu `Kobo Books`),
- `ocena` naj bo zaokrožena na 1 decimalno mesto,

- `update` naj vsebuje le polno letnico zadnje posodobitve (primer: če je zadnja posodobitev `15.6.21`, naj bo v seznamu `2021`).

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke("apps5.txt")
    [('Netflix', 'Entertainment', 4.4, 11694633, True, 2021, 'Teen'),
    ('Telegram', 'Communication', 4.5, 470676, True, 2021, 'Mature 17+'),
    ('Headspace', 'Health & Fitness', 4.6, 229799, True, 2021, 'Everyone'),
    ("Simon's Cat Dash", 'Casual', 4.6, 26662, True, 2019, 'Everyone'),
    ('Slack', 'Business', 4.2, 102572, True, 2021, 'Everyone')]
>>> preberi_podatke("apps_imena.txt")
    [('Forest', 'Productivity', 4.7, 340816, True, 2021, 'Everyone'),
    ('Calm', 'Health & Fitness', 4.3, 402349, True, 2021, 'Everyone'),
    ('TED', 'Education', 4.6, 208571, True, 2020, 'Everyone 10+'),
    ('Skillshare', 'Education', 4.5, 46058, True, 2021, 'Everyone'),
    ('Asana', 'Business', 4.5, 34128, True, 2021, 'Everyone')]
```

Rešitev

```
def preberi_podatke(datoteka):
    seznam = []
    f = open(datoteka, "r", encoding = "utf8")
    f.readline()
    for vrstica in f:
        podatki = vrstica.strip().split(";")
        ime, kategorija, ocena, st_ocen, st_nalaganj, max_nalaganj, brezplacna,
        ↪  cena, valuta, velikost, update, publika, oglasi, nakup = podatki

        ocena = round(float(ocena),1)
        st_ocen = int(st_ocen)

        if brezplacna == "TRUE":
            brezplacna = True
        else:
            brezplacna = False

        if "-" in ime:
            ime = ime.split("-")[0].strip()
        elif ":" in ime:
            ime = ime.split(":")[0].strip()

        update = int("20" + update.split(".")[-1])

        seznam.append((ime, kategorija, ocena, st_ocen, brezplacna, update,
        ↪  publika))
    f.close()
    return seznam
```

2. naloga

Napišite funkcijo `po_kategorijah`, ki kot argumenta sprejme seznam terk, kot ga vrača funkcija `preberi_podakte` (1. naloga) in leto, podano kot `int`.

Funkcija naj vrača slovar (`dict`), ki aplikacije razvrsti po kategorijah. Ključi naj bodo kategorije, vrednosti pa seznami terk s podatki o imenu, publiki, oceni in plačljivosti aplikacije iz podane kategorije (tj. `(ime, publika, ocena, brezplacna)`), katerih leto zadnje posodobitve je večje ali enako letu, podanem v argumentu funkcije. Če nobena od aplikacij ne izpolnjuje pogoja, naj funkcija vrne niz `"Nobena od aplikacij ne izpolnjuje pogoja."`.

Primeri pravilne izvedbe funkcije:

```
>>> podatki = [('Dumb Ways To Draw', 'Casual', 3.9, 50095, True, 2021, 'Teen'),
    ('SkyView Lite', 'Education', 4.4, 43828, True, 2019, 'Everyone'),
    ('Todoist', 'Productivity', 4.5, 226044, True, 2021, 'Everyone'),
    ('Forest', 'Productivity', 4.7, 340816, True, 2021, 'Everyone'),
    ('TED', 'Education', 4.6, 208571, True, 2020, 'Everyone 10+')]
>>> po_kategorijah(podatki, 2021)
    {'Casual': [('Dumb Ways To Draw', 'Teen', 3.9, True)],
    'Productivity': [('Todoist', 'Everyone', 4.5, True), ('Forest', 'Everyone',
    ↪  4.7, True)]}
>>> po_kategorijah(podatki, 2024)
    'Nobena od aplikacij ne izpolnjuje pogoja.'
```

Rešitev

```
def po_kategorijah(seznam, leto):
    slovar = {}
    for ime, kategorija, ocena, st_ocen, brezplacna, update, publika in seznam:
        if update >= leto:
            if kategorija not in slovar:
                slovar[kategorija] = []
            slovar[kategorija].append((ime, publika, ocena, brezplacna))
    if slovar == {}:
        return "Nobena od aplikacij ne izpolnjuje pogoja."
    return slovar
```

3. naloga

Napišite funkcijo `povprecje_po_kategorijah`, ki kot argument sprejme slovar, kot ga vrača funkcija `po_kategorijah` (2. naloga). Funkcija naj vrača slovar, v katerem kot ključi nastopajo kategorije, pod katerimi so kot vrednosti shranjene povprečne ocene aplikacij iz dane kategorije. Povprečna ocena naj bo zaokrožena na 2 decimalni mesti.

Namig: najprej si pripravite slovar, kjer se pod ključi kategorij nahajajo seznami s povprečnimi ocenami aplikacij.

Primer pravilne izvedbe funkcije:

```
>>> slovar = {'Casual': [('Dumb Ways To Draw', 'Teen', 3.9, True)],
    'Productivity': [('Todoist', 'Everyone', 4.5, True), ('Forest', 'Everyone',
    ↪  4.7, True)]}
>>> povprecje_po_kategorijah'(slovar)
    {'Casual': 3.9, 'Productivity': 4.6}
```

Rešitev

```python
def povprecje_po_kategorijah(slovar):
    slovar2 = {}
    for kljuc, vrednost in slovar.items():
        vsota = 0
        for ime, publika, ocena, brezplacna in vrednost:
            vsota += ocena
        slovar2[kljuc] = round(vsota/len(vrednost),2)
    return slovar2
```

4. naloga

Napišite funkcijo `avg_per`, ki kot argumenta sprejme seznam terk, kot ga vrača funkcija `preberi_podatke` (1. naloga) in decimalno število, ki ponazarja želeni percentil. Funkcija naj vrača terko oblike `(povprecje, percentil)`, pri čemer:

- `povprecje` predstavlja povprečno oceno vseh aplikacij v seznamu terk, zaokroženo na 2 decimalni mesti,
- `percentil` pa ocena, ki se nahaja na izbranem percentilu ranžirne vrste vseh števil ocen iz seznama terk.

Postopek računanja percentila:

- števila ocen vseh aplikacij iz seznama uredite po velikosti od najmanjšega do največjega (namig: uporabite funkcijo `sorted()`),
- za percentil izberite vrednost, ki se nahaja na indeksu, ki ponazarja vrednost na izbranem percentilu (primer in namig: če iščemo vrednost 90. percentila, mora funkcija vrniti vrednost, ki se v seznamu nahaja na indeksu `int(0.9 * len(seznam))`).

Primera pravilne izvedbe funkcije:

```
>>> podatki = [('Netflix', 'Entertainment', 4.4, 11694633, True, 2021, 'Teen'),
    ('Telegram', 'Communication', 4.5, 470676, True, 2021, 'Mature 17+'),
    ('Headspace', 'Health & Fitness', 4.6, 229799, True, 2021, 'Everyone'),
    ("Simon's Cat Dash", 'Casual', 4.6, 26662, True, 2019, 'Everyone'),
    ('Slack', 'Business', 4.2, 102572, True, 2021, 'Everyone')]
>>> avg_per(podatki, 0.9)
    (4.46, 470676)
>>> podatki2 = [('Forest', 'Productivity', 4.7, 340816, True, 2021,
↪   'Everyone'),
    ('Calm', 'Health & Fitness', 4.3, 402349, True, 2021, 'Everyone'),
    ('TED', 'Education', 4.6, 208571, True, 2020, 'Everyone 10+'),
    ('Skillshare', 'Education', 4.5, 46058, True, 2021, 'Everyone'),
    ('Asana', 'Business', 4.5, 34128, True, 2021, 'Everyone')]
>>> avg_per(podatki2, 0.5)
    (4.52, 208571)
```

Rešitev

```
def avg_per(seznam, percentil):
    s = []
    st = []

    for ime, kategorija, ocena, st_ocen, brezplacna, update, publika in seznam:
        s.append(ocena)
        st.append(st_ocen)

    avg = round(sum(s)/len(s),2)

    st = sorted(st)
    indeks = int(percentil*(len(st)-1))
    perc = st[indeks]

    return avg, perc
```

3. izpit 2024/2025

Naloge shranite v datoteko z imenom `naloge.py`.

1. naloga

Podatki o lastnostih izbranih kanalov na YouTube so zbrani v tekstovni datoteki. Predpostavljaj, da ima datoteka obliko, skladno s spodnjim zgledom. Kot ločilo je uporabljena vejica (`,`), prva vrstica pa predstavlja glave stolpcev.

Primer vsebine datoteke `youtube5.txt`:

```
uvrstitev,ime,sledilci,vsebin,ogledi,drzava
20,5-Minute Crafts,81.1M,7.391,27.907.581.110,US
29,UR.Cristiano,71.7M,80,721.228.815,PT
96,Alfredo Larin,46M,1.831,39.518.875.780,SV
1,MrBeast,336M,838,66.853.633.536,US
13,Stokes Twins,102M,335,16.139.307.959,US
```

Napišite funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke, vrne pa seznam terk oblike `(ime,sledilci,vsebine,ogledi,drzava)`. Pri tem naj:

- `ime` predstavlja ime kanala, zapisano kot `str`,
- `sledilci` predstavlja število sledilcev tega kanala podano v milijonih in zapisano kot `float`,
- `vsebine` predstavlja število vsebin na tem kanalu, zapisano kot `int`,
- `ogledi` predstavlja povprečno število ogledov, zapisano kot `int`,
- `drzava` predstavlja kratico države, iz katere izhaja kanal, zapisana kot `str`.

`sledilci` so v datoteki zapisani v obliki `"81.1M"`, v terki pa naj bodo zapisani kot `81.1`. Število zaokrožite na 1 decimalno mesto.

Podatka `vsebine` in `ogledi` v datoteki vsebujeta pike, v terki naj bosta zapisana kot celi števili (`int`). Primer: v datoteki je število zapisano kot `66.853.633.536`, v terki naj bo zapisano kot celo število `66853633536`.

Predpostavljajte, da lahko katerakoli izmed spremenljivk v posamezni vrstici datoteke manjka. V tem primeru takšne vrstice ne vključite v seznam terk.

Primera pravilne izvedbe funkcije:

```
>>> preberi_podatke("youtube5.txt")
    [('5-Minute Crafts', 81.1, 7391, 27907581110, 'US'),
    ('UR.Cristiano', 71.7, 80, 721228815, 'PT'),
    ('Alfredo Larin', 46.0, 1831, 39518875780, 'SV'),
    ('MrBeast', 336.0, 838, 66853633536, 'US'),
```

```
    ('Stokes Twins', 102.0, 335, 16139307959, 'US')]
>>> preberi_podatke("youtube5_missing.txt")
    [('T-Series Bhakti Sagar', 72.6, 29140, 37185631965, 'IN'),
    ('Set India', 180.0, 148727, 172709029653, 'IN'),
    ('Katy Perry', 45.6, 170, 27616850074, 'US')]
```

Rešitev

```
def preberi_podatke(datoteka):
    sez = []
    f = open(datoteka, encoding="utf8")
    f.readline()

    for vrstica in f:
        podatki = vrstica.strip().split(',')

        if "" in podatki:
            continue

        uvrstitev,ime,sledilci,vsebin,ogledi,drzava = podatki

        sledilci = round(float(sledilci.strip('M')),1)
        vsebin = int(vsebin.replace('.',''))
        ogledi = int(ogledi.replace('.',''))

        sez.append((ime, sledilci, vsebin, ogledi, drzava))

    f.close()
    return sez
```

2. naloga

Napiši funkcijo `po_drzavah`, ki kot argument sprejme seznam terk, kot ga vrača funkcija `preberi_podatke` iz 1. naloge (tj. seznam terk oblike `(ime,sledilci,vsebine,ogledi,drzava)`).

Funkcija naj vrača slovar, kjer so ključi posamezne države, vrednosti pa seznam terk oblike `(ime,vsebine,sledilci,ogledi)`.

Primera pravilno izvedene funkcije:

```
>>> podatki1 = [('5-Minute Crafts', 81.1, 7391, 27907581110, 'US'),
    ('UR.Cristiano', 71.7, 80, 721228815, 'PT'),
    ('Alfredo Larin', 46.0, 1831, 39518875780, 'SV'),
    ('MrBeast', 336.0, 838, 66853633536, 'US'),
    ('Stokes Twins', 102.0, 335, 16139307959, 'US')]
>>> po_drzavah(podatki1)
    {'US': [('5-Minute Crafts', 7391, 81.1, 27907581110),
    ('MrBeast', 838, 336.0, 66853633536),
    ('Stokes Twins', 335, 102.0, 16139307959)],
    'PT': [('UR.Cristiano', 80, 71.7, 721228815)],
    'SV': [('Alfredo Larin', 1831, 46.0, 39518875780)]}
>>> podatki2 = [('T-Series Bhakti Sagar', 72.6, 29140, 37185631965, 'IN'),
    ('Set India', 180.0, 148727, 172709029653, 'IN'),
    ('Katy Perry', 45.6, 170, 27616850074, 'US')]
>>> po_drzavah(podatki2)
    {'IN': [('T-Series Bhakti Sagar', 29140, 72.6, 37185631965),
    ('Set India', 148727, 180.0, 172709029653)],
    'US': [('Katy Perry', 170, 45.6, 27616850074)]}
```

Rešitev

```
def po_drzavah(seznam):
    slovar = {}
    for ime, sledilci, vsebin, ogledi, drzava in seznam:
        if drzava not in slovar:
            slovar[drzava] = []
        slovar[drzava].append((ime, vsebin, sledilci, ogledi))
    return slovar
```

3. naloga

Napišite funkcijo `ratio`, ki kot argumenta sprejme slovar, kot ga vrača funkcija `po_drzavah` (2. naloga) in ime države, zapisano kot `str`.

Funkcija naj za vsakega ustvarjalca iz države, določene z drugim argumentom, izračuna razmerje med ogledi in naročniki, ter vrne seznam terk oblike `(ime, razmerje)`.

Pri tem naj bo razmerje zaokroženo na 1 decimalno mesto natančno, izračunate pa ga po formuli: $ogledi/(sledilci * (10 * *6))$

V primeru, da podana država v slovarju nima nobenega izvajalca, naj funkcija vrne prazen seznam.

Primer pravilno izvedene funkcije:

```
>>> slovar = {'US': [('5-Minute Crafts', 7391, 81.1, 27907581110),
    ('MrBeast', 838, 336.0, 66853633536),
    ('Stokes Twins', 335, 102.0, 16139307959)],
    'PT': [('UR.Cristiano', 80, 71.7, 721228815)],
    'SV': [('Alfredo Larin', 1831, 46.0, 39518875780)]}
>>> ratio(slovar, "US")
    [('5-Minute Crafts', 344.1), ('MrBeast', 199.0), ('Stokes Twins', 158.2)]
```

Rešitev

```
def ratio(slovar, drzava):
    seznam = []
    if drzava in slovar:
        for ime, _, sledilci ,ogledi in slovar[drzava]:
            seznam.append((ime, round(ogledi/(sledilci*(10**6)),1)))
    return seznam
```

4. naloga

Napiši funkcijo `min_max`, ki kot argument sprejme slovar, kot ga vrača funkcija `po_drzavah` (2. naloga).

Funkcija naj vrača seznam terk oblike `(drzava, minimum, maksimum)`, pri čemer sta minimum in maksimum najnižje oziroma najvišje število ogledov izmed vseh kanalov iz dane države. Funkcija naj iskanje minimuma in maksimuma izvede le za države z vsaj dvema kanaloma. Ostale države naj iz seznama terk izpusti.

Primer pravilno izvedene funkcije:

```
>>> slovar = {'US': [('5-Minute Crafts', 7391, 81.1, 27907581110),
    ('MrBeast', 838, 336.0, 66853633536),
    ('Stokes Twins', 335, 102.0, 16139307959)],
    'PT': [('UR.Cristiano', 80, 71.7, 721228815)],
    'SV': [('Alfredo Larin', 1831, 46.0, 39518875780)]}
>>> min_max(slovar)
    [('US', 16139307959, 66853633536)]
```

Rešitev

```
def min_max(slovar):
    slovar2 = {}

    for drzava, seznam in slovar.items():
        if len(seznam) > 1:
            if drzava not in slovar2:
                slovar2[drzava] = []
            for terka in seznam:
                slovar2[drzava].append(terka[-1])

    seznam2 = []
    for drzava, seznam in slovar2.items():
        seznam2.append((drzava, min(seznam), max(seznam)))

    return seznam2
```

1. izpit 2023/2024

S portala Goodreads smo pridobili seznam knjig, tvoja naloga pa bo, da napišeš štiri funkcije, s katerimi bomo lahko na podlagi določenih kriterijev izbrali knjige za počitniška branja. Naloge shrani v datoteko z imenom `naloge.py`.

1. naloga

Podatki o knjigah so zapisani v tekstovnih datotekah. Predpostavljaj, da ima datoteka obliko, ki je skladna s spodnjim zgledom - v prvi vrstici so podana imena stolpcev, za ločilo pa je uporabljeno podpičje (`;`).

Primer datoteke `goodread5.csv`:

```
avtor;naslov;zbirka;ocena;strani;zanri
Frances Hodgson Burnett;The Secret
↪ Garden;;413;331;Classics,Fiction,Childrens,Young Adult,Historical
↪ Fiction,Middle Grade,Literature,Fantasy,Historical,Audiobook
John Green;Looking for Alaska;;402;221;Young
↪ Adult,Fiction,Contemporary,Romance,Realistic Fiction,Coming Of
↪ Age,Teen,Mystery,Young Adult Contemporary,High School
Louis Sachar;Holes;Holes;397;233;Young Adult,Fiction,Childrens,Middle
↪ Grade,Realistic Fiction,Classics,Mystery,Adventure,School,Contemporary
Arthur Golden;Memoirs of a Geisha;;412;503;Fiction,Historical
↪ Fiction,Romance,Historical,Classics,Japan,Adult,Novels,Asia,Adult Fiction
Stephenie Meyer;Twilight;The Twilight Saga;360;501;Young
↪ Adult,Fantasy,Romance,Vampires,Fiction,Paranormal,Paranormal
↪ Romance,Supernatural,Teen,Urban Fantasy
```

Napiši funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke (`str`) in vrne vsebino datoteke kot seznam terk oblike `(avtor, naslov, zbirka, ocena, strani, zanri)`. Pri tem naj

- `avtor` predstavlja ime avtorja knjige zapisano kot `str`,
- `naslov` predstavlja naslov knjige zapisan kot `str`,
- `zbirka` predstavlja ime zbirke, v kateri je bila knjiga izdana, zapisano kot `str`,
- `ocena` predstavlja oceno knjige na intervalu od 1 do 5 zapisano kot `float`,
- `strani` predstavlja število strani zapisano kot `int`,
- `zanr` predstavlja seznam (`list`) vodilnih treh žanrov, v katere lahko knjigo uvrstimo.

Napotki in namigi:

- prva vrstica predstavlja imena stolpcev in jo pri branju preskoči,

- podatek `ocena` pretvorimo na interval od 1 do 5 tako, da oceno zapisano v datoteki delimo s 100,
- v določenih primerih podatek `zbirka` v datoteki nima vrednosti (`''`) - v tem primeru naj terka namesto podatka o zbirki vsebuje vrednost `'standalone'`,
- podatek `zanr` lahko v datoteki vsebuje poljubno število žanrov, ki so v posamezni vrstici med seboj ločeni z vejico (`,`); terka, ki jo tvoja funkcija vrača, naj vsebuje seznam prvih treh žanrov, v katere lahko izbrano knjigo uvrstimo.

Primer pravilne izvedbe funkcije:

```
>>> preberi_podatke("goodread5.csv")
[('Frances Hodgson Burnett',
  'The Secret Garden',
  'standalone',
  4.13,
  331,
  ['Classics', 'Fiction', 'Childrens']),
 ('John Green',
  'Looking for Alaska',
  'standalone',
  4.02,
  221,
  ['Young Adult', 'Fiction', 'Contemporary']),
 ('Louis Sachar',
  'Holes',
  'Holes',
  3.97,
  233,
  ['Young Adult', 'Fiction', 'Childrens']),
 ('Arthur Golden',
  'Memoirs of a Geisha',
  'standalone',
  4.12,
  503,
  ['Fiction', 'Historical Fiction', 'Romance']),
 ('Stephenie Meyer',
  'Twilight',
  'The Twilight Saga',
  3.6,
  501,
  ['Young Adult', 'Fantasy', 'Romance'])]
```

Rešitev

```
def preberi_podatke(datoteka):
    seznam = []
    with open(datoteka, "r", encoding="utf8") as f:
        f.readline()
        for vrstica in f:
            avtor, naslov, zbirka, ocena, strani, zanri =
            ↪  vrstica.strip().split(";")

            strani = int(strani)

            ocena = float(ocena)/100
            if zbirka == "":
                zbirka = "standalone"

            zanri = zanri.split(",")[:3]

            seznam.append((avtor, naslov, zbirka, ocena, strani, zanri))

    return seznam
```

2. naloga

Napiši funkcijo `slovar_zanrov`, ki kot argument sprejme seznam terk, kot ga vrača Naloga 1, tj. seznam terk oblike `(avtor, naslov, zbirka, ocena, strani, zanri)`.

Funkcija naj vrne slovar (`dict`), ki kot ključe vsebuje imena žanrov, ki se pojavijo v seznamu terk (`zanri`). Na posamezen žanr naj bo kot vrednost vezan seznam terk oblike `(avtor, naslov)`, pri čemer posamezna terka predstavlja avtorja in naslov knjige, ki jo uvrščamo v določen žanr.

V primeru, da funkciji kot argument podamo prazen seznam, naj vrne prazen slovar.

Primer pravilne izvedbe funkcije:

```
>>> sez =[('Frances Hodgson Burnett', 'The Secret Garden', 'standalone', 4.13,
↪  331, ['Classics', 'Fiction', 'Childrens']),
  ('John Green', 'Looking for Alaska', 'standalone', 4.02, 221, ['Young Adult',
  ↪  'Fiction', 'Contemporary']),
  ('Stephenie Meyer', 'Twilight', 'The Twilight Saga', 3.6, 501, ['Young
  ↪  Adult', 'Fantasy', 'Romance'])]
>>> slovar_zanrov(sez)
{'Classics': [('Frances Hodgson Burnett', 'The Secret Garden')],
 'Fiction': [('Frances Hodgson Burnett', 'The Secret Garden'),
  ('John Green', 'Looking for Alaska')],
 'Childrens': [('Frances Hodgson Burnett', 'The Secret Garden')],
 'Young Adult': [('John Green', 'Looking for Alaska'),
  ('Stephenie Meyer', 'Twilight')],
 'Contemporary': [('John Green', 'Looking for Alaska')],
 'Fantasy': [('Stephenie Meyer', 'Twilight')],
 'Romance': [('Stephenie Meyer', 'Twilight')]}
```

Rešitev

```
def slovar_zanrov(seznam):
    slovar = {}
    if seznam == []:
        return slovar
    for avtor, naslov, zbirka, ocena, strani, zanri in seznam:
        for zanr in zanri:
            if zanr not in slovar:
                slovar[zanr] = []
            slovar[zanr].append((avtor, naslov))
    return slovar
```

3. naloga

Napiši funkcijo `statistika_strani`, ki kot argument sprejme seznam terk, kot ga vrača Naloga 1, tj. seznam terk oblike `(avtor, naslov, zbirka, ocena, strani, zanri)`.

Funkcija naj vrne slovar (`dict`), ki kot ključ:vrednost vsebuje sledeče pare podatkov:

- na ključ `'min'` (`str`) naj bo vezano minimalno število strani (`int`) vseh knjig v seznamu terk,
- na ključ `'max'` (`str`) naj bo vezano maksimalno število strani (`int`) vseh knjig v seznamu terk,
- na ključ `'avg'` (`str`) naj bo vezano povprečno število strani (`float`) vseh knjig v seznamu terk.

Če funkciji podamo prazen seznam terk, naj bo na ključa `'min'` in `'max'` vezana vrednost `-1`, na ključ `'avg'` pa vrednost `'ni podatkov'`.

Primera pravilne izvedbe funkcije:

```
>>> sez = [('Frances Hodgson Burnett', 'The Secret Garden', 'standalone', 4.13,
↪  331, []),
       ('John Green', 'Looking for Alaska', 'standalone', 4.02, 221, []),
       ('Louis Sachar', 'Holes', 'Holes', 3.97, 233, []),
       ('Arthur Golden', 'Memoirs of a Geisha', 'standalone', 4.12, 503, []),
       ('Stephenie Meyer', 'Twilight', 'The Twilight Saga', 3.6, 501, [])]
>>> statistika_strani(sez)
{'min': 221, 'max': 503, 'avg': 357.8}
>>> statistika_strani([])
{'min': -1, 'max': -1, 'avg': 'ni podatkov'}
```

Rešitev

Krajša rešitev:

```
def statistika_strani(seznam):
    slovar = {}
    s = []

    for _, _, _, _, strani, _ in seznam:
        s.append(strani)

    if seznam != []:
        slovar["min"] = min(s)
        slovar["max"] = max(s)
        slovar["avg"] = sum(s)/len(s)
    else:
        slovar["min"] = -1
        slovar["max"] = -1
        slovar["avg"] = "ni podatkov"

    return slovar
```

Daljša rešitev:

```python
def statistika_strani(seznam):
    slovar = {}

    maksimum = -1

    if seznam != []:
        minimum = seznam[0][4]
    else:
        minimum = -1

    vsota = 0

    for _, _, _, _, strani, _ in seznam:
        if strani > maksimum:
            maksimum = strani
        if strani < minimum:
            minimum = strani
        vsota += strani

    slovar["min"] = minimum
    slovar["max"] = maksimum
    if seznam != []:
        slovar["avg"] = vsota/len(seznam)
    else:
        slovar["avg"] = "ni podatkov"

    return slovar
```

4. naloga

Napiši funkcijo `zbirke`, ki kot argument sprejme seznam terk, kot ga vrača Naloga 1, tj. seznam terk oblike `(avtor, naslov, zbirka, ocena, strani, zanri)`.

Funkcija naj vrne po abecedi sortiran seznam (`list`) vseh zbirk, ki se pojavijo v podanem seznamu terk. Pri tem naj se posamezna zbirka v seznamu pojavi največ enkrat, vrednost `'standalone'` pa v seznamu ne sme nastopati.

Primer pravilne izvedbe funkcije:

```
>>> sez = [('Stephenie Meyer', 'Twilight', 'The Twilight Saga', 3.6, 501, []),
       ('J.K. Rowling', 'Harry Potter and the Deathly Hallows', 'Harry Potter',
       ↪  4.62, 759, []),
       ('Frances Hodgson Burnett', 'The Secret Garden', 'standalone', 4.13,
       ↪  331, []),
       ('John Green', 'Looking for Alaska', 'standalone', 4.02, 221, []),
       ('Louis Sachar', 'Holes', 'Holes', 3.97, 233, []),
       ('J.K. Rowling', "Harry Potter and the Sorcerer's Stone", 'Harry
       ↪  Potter', 4.47, 309, [])]
>>> zbirke(sez)
['Harry Potter', 'Holes', 'The Twilight Saga']
```

Rešitev

Uporaba množic:

```
def zbirke(seznam):
    mn = set()

    for _, _, zbirka, _, _, _ in seznam:

        if zbirka != "standalone":
            mn.add(zbirka)

    return sorted(list(mn))
```

Uporaba seznama:

```
def zbirke(seznam):
    sez = []

    for _, _, zbirka, _, _, _ in seznam:

        if zbirka != "standalone" and zbirka not in sez:
            sez.append(zbirka)

    return sorted(sez)
```

2. izpit 2023/2024

Z Netflixa smo pridobili seznam serij, ki jih platforma ponuja, tvoja naloga pa je, da napišeš štiri funkcije, s katerimi lahko analiziramo pridobljen seznam. Naloge shrani v datoteko z imenom `naloge.py`.

1. naloga

Podatki o serijah so zapisani v tekstovnih datotekah, kjer so v prvi vrstici zapisana imena stolpcev. Ločilo datoteke je podpičje (`;`).

Primer datoteke `netflix_s.csv`:

```
naslov;leto;skupina;cas;zanri;drzava;sezone;ocena
The Last Kingdom;2015;TV-MA;55;action,drama,war,history,european;GB;50;85
Monty Python's Flying Circus;1969;TV-14;30;comedy,european;GB;40;88
Top Gear;2002;TV-PG;58;comedy,reality,european,music;GB;320;87
Our Planet;2019;TV-G;50;documentary;GB;10;93
The Crown;2016;TV-MA;56;drama,history,european;GB,US;50;87
The Queen's Gambit;2020;TV-MA;56;drama,sport;US;10;86
Narcos;2015;TV-MA;52;drama,thriller,crime;US;30;88
The Great British Baking Show;2010;TV-PG;57;reality,european;GB;120;86
Stranger Things;2016;TV-14;61;scifi,thriller,drama,fantasy,horror;US;50;87
Napiši funkcijo preberi_podatke, ki kot argument sprejme ime datoteke (str),
↪  vrne pa vsebino datoteke v obliki seznama terk, pri čemer je posamezna
↪  terka (naslov, leto, cas, sezone, zanr, drzava, ocena, skupina).
```

Napiši funkcijo `v_seznam`, ki kot argument sprejme ime datoteke (`str`), vrne pa vsebino datoteke v obliki seznama terk, pri čemer je posamezna terka
`(naslov, leto, cas, sezone, zanr, drzava, ocena, skupina)`.

Za posamezne elemente terke naj velja:

- `naslov` predstavlja naslov serije, zapisan kot `str`,
- `leto` predstavlja leto izida, zapisano kot `int`,
- `cas` predstavlja trajanje posamezne epizode v minutah, zapisano kot `int`,
- `sezone` predstavlja število vseh sezon, zapisano kot `int`,
- `zanr` predstavlja prvi naveden žanr iz stolpca zanri, zapisan kot `str`,
- `drzava` predstavlja oznako države, zapisano kot `str`,
- `ocena` predstavlja povprečno oceno serije s strani IMDb na intervalu od 1-10, zapisano kot `float`,
- skupina predstavlja oznako primernosti serije za starostne skupine, zapisana kot `str`.

Napotki:

- prva vrstica vsebuje imena stolpcev, zato jo pri branju preskoči,
- zanimajo nas le serije, ki so daljše od 30 minut, vse ostale pri branju preskoči,
- podatek `ocena` pretvorimo na interval 1-10 tako, da število, zapisano v datoteki, delimo z 10,
- v nekaterih primerih `skupina` še nima določene primernosti serije za starostne skupine, zato naj v takšnih primerih terka vsebuje vrednost `'ni_podano'`,
- stolpec `zanri` lahko v datoteki vsebuje poljubno število žanrov, ki so v posamezni vrstici med seboj ločeni z vejico (`,`); terka pa naj pod `zanr` vsebuje le žanr, naveden na prvem mestu v `zanri`.

Primer pravilne izvedbe funkcije:

```
>>> preberi_podatke("netflix_s.csv")
[('The Last Kingdom', 2015, 55, 50, 'action', 'GB', 8.5, 'TV-MA'),
 ('Top Gear', 2002, 58, 320, 'comedy', 'GB', 8.7, 'TV-PG'),
 ('Our Planet', 2019, 50, 10, 'documentary', 'GB', 9.3, 'TV-G'),
 ('The Crown', 2016, 56, 50, 'drama', 'GB', 8.7, 'TV-MA'),
 ("The Queen's Gambit", 2020, 56, 10, 'drama', 'US', 8.6, 'TV-MA'),
 ('Narcos', 2015, 52, 30, 'drama', 'US', 8.8, 'TV-MA'),
 ('The Great British Baking Show', 2010, 57, 120, 'reality', 'GB', 8.6,
 ↪  'TV-PG'),
 ('Stranger Things', 2016, 61, 50, 'scifi', 'US', 8.7, 'TV-14')]
```

Rešitev

```
def v_seznam(datoteka):
    seznam = []
    with open(datoteka, "r", encoding = "utf8") as f:
        f.readline()
        for vrstica in f:

            naslov, leto, skupina, cas, zanri, drzava, sezone, ocena =
            ↪  vrstica.strip().split(";")

            cas = int(cas)

            if cas > 30:
                zanr = zanri.split(",")[0]

                ocena = float(ocena)/10

                if skupina == "":
                    skupina = "ni_podano"

                seznam.append((naslov, int(leto), cas, int(sezone), zanr,
                ↪  drzava, ocena, skupina))

        return seznam
```

2. naloga

2a

Napiši funkcijo `povprecje`, ki kot argumenta sprejme

- seznam terk, kot ga vrača naloga 1 (tj. seznam terk oblike `(naslov, leto, cas, sezone, zanr, drzave, ocena, skupina)`) in
- leto zapisano kot `int`.

Funkcija naj vrne povprečno oceno serij iz seznama terk, ki so izšle v podanem letu. Povprečna ocena naj bo zaokrožena na dve decimalni mesti natančno. V primeru, da ocene ne moremo izračunati, naj funkcija vrne vrednost `False`.

Primer pravilne izvedbe funkcije:

```
>>> seznam = [('The Last Kingdom', 2015, 55, 50, 'action', 'GB', 8.5, 'TV-MA'),
 ('Top Gear', 2002, 58, 320, 'comedy', 'GB', 8.7, 'TV-PG'),
 ("The Queen's Gambit", 2020, 56, 10, 'drama', 'US', 8.6, 'TV-MA'),
 ('The Crown', 2016, 56, 50, 'drama', 'GB', 8.7, 'TV-MA'),
 ('Stranger Things', 2016, 61, 50, 'scifi', 'US', 8.7, 'TV-14')]
>>> povprecje(seznam, 2016)
8.7
>>> povprecje(seznam, 1883)
False
```

2b

Napiši funkcijo `povprecja_po_letih`, ki kot argument sprejme seznam terk, kot ga vrača naloga 1 (tj. seznam terk oblike `(naslov, leto, cas, sezone, zanr, drzave, ocena, skupina)`). Funkcija naj vrne slovar, v katerem ključi predstavljajo leta izida serij zapisana kot `int`, vrednosti pa povprečne ocene serij v posameznih letih (kot jih vrača funkcija `povprecje`). Ključi naj bodo le tista leta, za katera lahko izračunamo povprečne ocene glede na podan seznam terk.

Primer pravilne izvedbe funkcije:

```
>>> seznam = [('The Last Kingdom', 2015, 55, 50, 'action', 'GB', 8.5, 'TV-MA'),
 ('Top Gear', 2002, 58, 320, 'comedy', 'GB', 8.7, 'TV-PG'),
 ("The Queen's Gambit", 2020, 56, 10, 'drama', 'US', 8.6, 'TV-MA'),
 ('The Crown', 2016, 56, 50, 'drama', 'GB', 8.7, 'TV-MA'),
 ('Stranger Things', 2016, 61, 50, 'scifi', 'US', 8.7, 'TV-14')]
>>> povprecja_po_letih(seznam)
{2002: 8.7, 2015: 8.5, 2016: 8.7, 2020: 8.6}
```

Rešitev

2a:

```
def povprecje(seznam, letnica):
    ocene = []

    for _, leto, _, _, _, _, ocena, _ in seznam:
        if letnica == leto:
            ocene.append(ocena)

    if ocene == []:
        return False

    return round(sum(ocene)/len(ocene),2)
```

2b[2]:

```
def povprecja_po_letih(seznam):
    slovar = {}
    leta = []
    for _, leto, _, _, _, _, ocena, _ in seznam:
        leta.append(leto)

    for leto in sorted(leta):
        slovar[leto] = povprecje(seznam, leto)

    return slovar
```

Krajša verzija[3]:

```
def povprecja_po_letih(seznam):
    slovar = {}
    for _, leto, _, _, _, _, ocena, _ in seznam:
        slovar[leto] = povprecje(seznam, leto)
    return slovar
```

[2]Uporaba `sorted()` zaobide napako v 8. testu.
[3]Pri uporabi te verziji bo 8. test javil napako zaradi pomanjkljivosti testa.

3. naloga

Napiši funkcijo `binge_watching`, ki kot argumenta sprejme seznam terk, kot ga vrača naloga 1 (tj. seznam terk oblike `(naslov, leto, cas, sezone, zanr, drzave, ocena, skupina)`) in čas zapisan kot `float`, ki predstavlja ure, ki jih ima gledalec na voljo za ogled serije. Pri tem bomo predpostavili, da ima posamezna sezona 10 epizod in je dolžina celotne serije v urah enaka:

$dolžina = čas/60 * sezone * 10$

Funkcija naj vrne seznam serij, ki jih lahko gledalec v celoti pogleda v navedenem času. Pazite, da se bodo časovne enote ujemale.

Če nobene izmed serij ni mogoče pogledati v navedenem času, naj funkcija vrne prazen seznam.

Primeri pravilne izvedbe funkcije:

```
>>> seznam = [('The Last Kingdom', 2015, 55, 50, 'action', 'GB', 8.5, 'TV-MA'),
 ('Top Gear', 2002, 58, 320, 'comedy', 'GB', 8.7, 'TV-PG'),
 ("The Queen's Gambit", 2020, 56, 10, 'drama', 'US', 8.6, 'TV-MA'),
 ('The Crown', 2016, 56, 50, 'drama', 'GB', 8.7, 'TV-MA'),
 ('Stranger Things', 2016, 61, 50, 'scifi', 'US', 8.7, 'TV-14')]
>>> binge_watching(seznam, 1200)
['The Last Kingdom', "The Queen's Gambit", 'The Crown', 'Stranger Things']
>>> binge_watching(seznam, 120)
["The Queen's Gambit"]
>>> binge_watching(seznam, 12)
[]
```

Rešitev

```
def binge_watching(seznam, ure):
    serije = []

    for naslov, _, cas, sezone, _, _, _, _ in seznam:
        dolzina = cas/60 * sezone * 10
        if ure >= dolzina:
            serije.append(naslov)

    return serije
```

4. naloga

Napiši funkcijo `priporocila`, ki kot argumenta sprejme ime serije, zapisano kot `str` in seznam terk kot ga vrača naloga 1 (tj. seznam terk oblike
`(naslov, leto, cas, sezone, zanr, drzave, ocena, skupina)`). Funkcija naj vrne seznam serij, ki so podobne navedeni seriji v vseh naslednjih vidikih:

- `ocena` podobne serije je lahko od izbrane serije nižja za največ 0.5, navzgor omejitve ni,
- `zanr` podobnih serij mora biti enak žanru izbrane serije,
- `skupina` podobnih serij je lahko enaka ali nižja od izbrane serije in sicer:
  - če izbrana serija sodi v katerokoli od skupin `'TV-G', 'TV-PG', 'TV-Y'` ali `'TV-Y7'`, so lahko podobne serije iz vseh teh skupin,
  - če izbrana serija sodi v skupino `'TV-14'`, lahko podobne serije sodijo v skupine `'TV-G', 'TV-PG', 'TV-Y', 'TV-Y7'` in `'TV-14'`,
  - če izbrana serija sodi v skupino `'TV-MA'`, lahko podobne serije iz vseh možnih skupin (tj. `'TV-G', 'TV-PG', 'TV-Y', 'TV-Y7', 'TV-14', 'TV-MA'`).

Če izbrane serije ni v seznamu, naj funkcija vrne `False`. Če v seznamu ni nobene serije, ki bi bila podobna izbrani seriji, naj funkcija vrne prazen seznam.

Opomba: iz seznama podobnih serij izločite ime serije, ki je podano kot argument funkcije.

Primeri pravilne izvedbe funkcije:

```
>>> seznam = [('Anne with an E', 2017, 46, 30, 'drama', 'CA', 8.7, 'TV-G'),
 ('The Last Kingdom', 2015, 55, 50, 'action', 'GB', 8.5, 'TV-MA'),
 ('Top Gear', 2002, 58, 320, 'comedy', 'GB', 8.7, 'TV-PG'),
 ('Call the Midwife', 2012, 55, 110, 'family', 'GB', 8.5, 'TV-PG'),
 ('The Crown', 2016, 56, 50, 'drama', 'GB,US', 8.7, 'TV-MA'),
 ("The Queen's Gambit", 2020, 56, 10, 'drama', 'US', 8.6, 'TV-MA'),
 ('The Great British Baking Show',2010,57,120,'reality','GB',8.6,'TV-PG'),
 ('Top Gear', 2002, 58, 320, 'comedy', 'GB', 8.7, 'TV-PG'),
 ('Black Mirror', 2011, 59, 50, 'drama', 'GB', 8.8, 'TV-MA'),
 ('Stranger Things', 2016, 61, 50, 'scifi', 'US', 8.7, 'TV-14')]
>>> priporocila('The Crown', seznam)
['Anne with an E', "The Queen's Gambit", 'Black Mirror']
>>> priporocila('Game of Thrones', seznam)
False
>>> priporocila('Top Gear', seznam)
[]
```

Rešitev

```python
def priporocila(serija, seznam):
    najdena = False
    for naslov, leto, cas, sezone, zanr, drzave, ocena, skupina in seznam:
        if serija == naslov:
            ocena1 = ocena - 0.5
            zanr1 = zanr
            skupina1 = skupina
            najdena = True

    if not najdena:
        return False

    if zanr1 == "TV-14":
        dovoljeno = ['TV-G', 'TV-PG', 'TV-Y', 'TV-Y7', 'TV-14']
    elif zanr1 == 'TV-MA':
        dovoljeno = ['TV-G', 'TV-PG', 'TV-Y', 'TV-Y7', 'TV-14', 'TV-MA']
    else:
        dovoljeno = ['TV-G', 'TV-PG', 'TV-Y', 'TV-Y7']

    priporoceno = []
    for naslov, leto, cas, sezone, zanr, drzave, ocena, skupina in seznam:
        if serija != naslov and ocena >= ocena1 and zanr1 == zanr and skupina
        ↪  in dovoljeno:
            priporoceno.append(naslov)

    if priporoceno == []:
        return []
    return priporoceno
```

3. izpit 2023/2024

V datoteki se nahaja standardna statistika nogometašev, ki so nastopali na EURO 2024. Tvoja naloga bo, da napišeš funkcije, s katerimi bomo lahko izvedli osnovne analize podatkovnega seta. Naloge shrani v datoteko z imenom `naloge.py`.

1. naloga

Podatki o nogometaših, ki so nastopali na EURO 2024, so zapisani v datoteki `.csv`. Predpostavljaj, da ima datoteka obliko, ki je skladna s spodnjim zgledom - v prvi vrstici so podana imena stolpcev, za ločilo pa je uporabljena vejica (`,`).

Primer datoteke `EURO24_10.csv`:

```
ID,ime,pozicija,drzava,leto_rojstva,st_tekem,zacetna_poz,minute,goli,asistence
↪  ,rumeni,rdeci
44,Victor Bernth Kristiansen,DF-MF,Denmark,2002,4,2,156,0,1,0,0
85,Diogo Costa,GK,Portugal,1999,5,5,510,0,0,0,0
128,Joao Felix,MF-FW,Portugal,1999,2,1,105,0,0,0,0
154,Alex Grimaldo,DF,Spain,1995,2,1,115,0,0,0,0
185,Goncalo Inacio,DF,Portugal,2001,2,1,118,0,0,0,0
230,Aymeric Laporte,DF,Spain,1994,6,6,525,0,0,0,0
282,Alvaro Morata,FW,Spain,1992,7,6,449,1,1,2,0
302,Matheus Nunes,MF,Portugal,1998,2,0,18,0,0,0,0
327,Jordan Pickford,GK,England,1994,7,7,690,0,0,0,0
357,Fabian Ruiz Pena,MF,Spain,1996,6,6,540,2,2,1,0
389,Unai Simon,GK,Spain,1997,6,6,570,0,0,1,0
444,Dayot Upamecano,DF,France,1998,6,6,570,0,0,0,0
492,Martin Zubimendi,MF,Spain,1999,4,1,141,0,0,0,0
187,Erik Janza,DF,Slovenia,1993,3,3,269,1,0,2,0
```

Napiši funkcijo `preberi_podatke`, ki kot argument sprejme ime datoteke (`str`) in vrne vsebino datoteke kot seznam terk oblike
`(ime, drzava, starost, st_tekem, goli, asistence, rumeni, rdeci)`.

Pri tem naj

- `ime` predstavlja ime nogometaša, zapisano kot `str`,
- `drzava` predstavlja državo, zapisano kot `str`,
- `starost` predstavlja starost igralca, zapisano kot `int`,
- `st_tekem` predstavlja število tekem, ki jih je igralec odigral, zapisano kot `int`,
- `goli` predstavlja število doseženih golov, zapisanih kot `int`,
- `asistence` predstavljajo število asistenc, zapisanih kot `int`,

- `rumeni` predstavlja število dodeljenih rumenih kartonov, zapisanih kot `int` in
- `rdeci` predstavlja število dodeljenih rdečih kartonov, zapisanih kot `int`.

Napotki:

- prva vrstica vsebuje imena stolpcev, zato jo pri branju preskoči,
- starost igralca izračunamo tako, da od 2024 odštejemo letnico rojstva dotičnega nogometaša.

Primer pravilne izvedbe funkcije:

```
>>> preberi_podatke("EURO24_10.csv")
[('Victor Bernth Kristiansen', 'Denmark', 22, 4, 0, 1, 0, 0),
 ('Diogo Costa', 'Portugal', 25, 5, 0, 0, 0, 0),
 ('Joao Felix', 'Portugal', 25, 2, 0, 0, 0, 0),
 ('Alex Grimaldo', 'Spain', 29, 2, 0, 0, 0, 0),
 ('Goncalo Inacio', 'Portugal', 23, 2, 0, 0, 0, 0),
 ('Aymeric Laporte', 'Spain', 30, 6, 0, 0, 0, 0),
 ('Alvaro Morata', 'Spain', 32, 7, 1, 1, 2, 0),
 ('Matheus Nunes', 'Portugal', 26, 2, 0, 0, 0, 0),
 ('Jordan Pickford', 'England', 30, 7, 0, 0, 0, 0),
 ('Fabian Ruiz Pena', 'Spain', 28, 6, 2, 2, 1, 0),
 ('Unai Simon', 'Spain', 27, 6, 0, 0, 1, 0),
 ('Dayot Upamecano', 'France', 26, 6, 0, 0, 0, 0),
 ('Martin Zubimendi', 'Spain', 25, 4, 0, 0, 0, 0),
 ('Erik Janza', 'Slovenia', 31, 3, 1, 0, 2, 0)]
```

Rešitev

```
def preberi_podatke(datoteka):
    with open(datoteka, "r", encoding = "utf8") as f:
        seznam = []

        f.readline()
        for vrstica in f:
            podatki = vrstica.strip().split(",")
            for i in range(len(podatki)):
                if i in [4,5,7,8,9,10,11]:
                    podatki[i] = int(podatki[i])

            _, ime, _, drzava, leto_rojstva, st_tekem, _, minute, goli,
            ↪  asistence, rumeni, rdeci = podatki

            starost = 2024-leto_rojstva

            seznam.append((ime, drzava, starost, st_tekem, goli, asistence,
            ↪  rumeni, rdeci))

        return seznam
```

2. naloga

Napiši funkcijo `drzavni_seznami`, ki kot argumenta sprejme:

- seznam terk, kot ga vrača 1. naloga (tj. seznam terk oblike `(ime, drzava, starost, st_tekem, goli, asistence, rumeni, rdeci)`) in
- ime države, zapisano kot `str`.

Funkcija naj vrne slovar, z naslednjimi pari ključ:vrednost:

- pod ključem `'starosti'` naj bo kot vrednost seznam starosti igralcev, ki so igrali za izbrano državo,
- pod ključem `'goli'` naj bo seznam vseh golov, ki so jih zadeli igralci izbrane države,
- pod ključem `'rumeni_kartoni'` naj bo seznam s števili rumenih kartonov igralcev izbrane države in
- pod ključem `'rdeci_kartoni'` naj bo seznam s števili rdečih kartonov igralcev izbrane države.

Vrednosti v slovarju torej predstavljajo seznami, v katerih posamezen element pripada enemu igralcu iz izbrane države.

Če izbrane države ni v seznamu, naj funkcija vrne niz (`str`): `Te drzave ni v seznamu.` (pozor: ne uporabite šumnika).

Primer pravilne izvedbe funkcije:

```
>>> seznam = preberi_podatke("EURO24_25.csv")
[('Alexander Bah', 'Denmark', 27, 4, 0, 0, 0, 0),
 ('Jacob Bruun Larsen', 'Denmark', 26, 1, 0, 0, 0, 0),
 ('Kingsley Coman', 'France', 28, 1, 0, 0, 0, 0),
 ('Thomas Delaney', 'Denmark', 33, 3, 0, 0, 0, 0),
 ('Joao Felix', 'Portugal', 25, 2, 0, 0, 0, 0),
 ('Conor Gallagher', 'England', 24, 5, 0, 0, 1, 0),
 ('Marc Guehi', 'England', 24, 6, 0, 1, 2, 0),
 ('Pierre Hojbjerg', 'Denmark', 29, 4, 0, 0, 0, 0),
 ("N'Golo Kante", 'France', 33, 6, 0, 0, 0, 0),
 ('Aymeric Laporte', 'Spain', 30, 6, 0, 0, 0, 0),
 ('Kylian Mbappe', 'France', 26, 5, 1, 1, 1, 0),
 ('Jamal Musiala', 'Germany', 21, 5, 3, 0, 0, 0),
 ('Joao Neves', 'Portugal', 20, 2, 0, 0, 0, 0),
 ('Joao Palhinha', 'Portugal', 29, 4, 0, 0, 2, 0),
 ('Jordan Pickford', 'England', 30, 7, 0, 0, 0, 0),
 ('Declan Rice', 'England', 25, 7, 0, 1, 0, 0),
 ('William Saliba', 'France', 23, 6, 0, 0, 1, 0),
 ('Antonio Silva', 'Portugal', 21, 2, 0, 0, 0, 0),
 ('Aurelien Tchouameni', 'France', 24, 5, 0, 0, 2, 0),
 ('Dayot Upamecano', 'France', 26, 6, 0, 0, 0, 0),
 ('Nico Williams', 'Spain', 22, 6, 2, 1, 0, 0),
 ('Jaka Bijol', 'Slovenia', 25, 4, 0, 0, 2, 0),
 ('Jon Gorenc Stankovic', 'Slovenia', 28, 4, 0, 0, 1, 0),
 ('Jan Oblak', 'Slovenia', 31, 4, 0, 0, 0, 0)]
>>> drzavni_seznami(seznam, "France")
{'starosti': [28, 33, 26, 23, 24, 26],
 'goli': [0, 0, 1, 0, 0, 0],
 'rumeni_kartoni': [0, 0, 1, 1, 2, 0],
 'rdeci_kartoni': [0, 0, 0, 0, 0, 0]}
>>> drzavni_seznami(seznam, "Switzerland")
'Te drzave ni v seznamu.'
```

Rešitev

```
def drzavni_seznami(seznam, drzava):
    slovar = {}

    slovar["starosti"] = []
    slovar["goli"] = []
    slovar["rumeni_kartoni"] = []
    slovar["rdeci_kartoni"] = []

    najdena = False
    for ime, drzava1, starost, st_tekem, goli, asistence, rumeni, rdeci in
    ↪ seznam:
        if drzava == drzava1:
            slovar["starosti"].append(starost)
            slovar["goli"].append(goli)
            slovar["rumeni_kartoni"].append(rumeni)
            slovar["rdeci_kartoni"].append(rdeci)
            najdena = True

    if not najdena:
        return "Te drzave ni v seznamu."
    return slovar
```

3. naloga

Napiši funkcijo `drzavno_povprecje`, ki kot argument sprejme slovar, kot ga vrača 2. naloga (tj. slovar oblike `{'starosti': [28, 33, 26, 23, 24, 26], 'goli': [0, 0, 1, 0, 0, 0], 'rumeni_kartoni': [0, 0, 1, 1, 2, 0], 'rdeci_kartoni': [0, 0, 0, 0, 0, 0]}`).

Funkcija naj vrne slovar, kjer se pod naslednjimi ključi nahajajo ustrezne povprečne vrednosti:

- pod ključem `'povprecna_starost'` naj bo povprečna starost igralcev zaokrožena na dve decimalni mesti natančno (v primeru, da povprečne starosti ne moremo izračunati, naj bo pod tem ključem vrednost `False`),
- pod ključem `'najvec_golov'` naj bo kot vrednost največje število golov, ki jih je zadel posamezen igralec (v primeru, da je seznam z goli prazen, naj bo pod ključem vrednost `0`),
- pod ključem `'kazni'` naj bo terka oblike `(rumeni, rdeci)`, kjer `rumeni` ponazarja seštevek vseh rumenih kartonov, `rdeci` seštevek vseh rdečih kartonov (v primeru, da je katerikoli seznam s kaznimi prazen, naj bo na njegovem mestu v terki vrednost `0`).

Povprečno število golov naj bo zaokroženo na 2 decimalni mesti.

Primer pravilne izvedbe funkcije:

```
>>> slovar = drzavni_seznami(seznam, "France")
>>> slovar
{'starosti': [28, 33, 26, 23, 24, 26],
 'goli': [0, 0, 1, 0, 0, 0],
 'rumeni_kartoni': [0, 0, 1, 1, 2, 0],
 'rdeci_kartoni': [0, 0, 0, 0, 0, 0]}
>>> drzavno_povprecje(slovar)
{'povprecna_starost': 26.67, 'najvec_golov': 1, 'kazni': (4, 0)}
```

Rešitev

```
def drzavno_povprecje(slovar):
    slovar2 = {}

    if sum(slovar["starosti"]) != 0:
        slovar2["povprecna_starost"] =
        ↪  round(sum(slovar["starosti"])/len(slovar["starosti"]),2)
    else:
        slovar2["povprecna_starost"] = False

    if slovar["goli"] != []:
        slovar2["najvec_golov"] = max(slovar["goli"])
    else:
        slovar2["najvec_golov"] = 0

    if slovar["rumeni_kartoni"] == []:
        rumeni = 0
    else:
        rumeni = sum(slovar["rumeni_kartoni"])
    if slovar["rdeci_kartoni"] == []:
        rdeci = 0
    else:
        rdeci = sum(slovar["rdeci_kartoni"])

    slovar2["kazni"] = (rumeni, rdeci)

    return slovar2
```

4. naloga

Napiši funkcijo tocke, ki kot argumente sprejme seznam terk, kot ga vrača 1. naloga (tj. seznam oblike `(ime, drzava, starost, st_tekem, goli, asistence, rumeni, rdeci)`), število točk za dosežen gol (`tocke_goli`) in število točk za asistenco (`tocke_asist`). Funkcija naj po formuli:

$tocke = (goli - rdeci) * tocke_goli + (asistence - rumeni) * tocke_asist$

izračuna točke za vsakega igralca iz seznama in vrne nov seznam terk oblike `(igralec, tocke)`. V seznamu naj bodo le igralci, ki nimajo negativnih točk.

Primer pravilne izvedbe funkcije:

```
>>> seznam = preberi_podatke("EURO24_10.csv")
[('Victor Bernth Kristiansen', 'Denmark', 22, 4, 0, 1, 0, 0),
 ('Diogo Costa', 'Portugal', 25, 5, 0, 0, 0, 0),
 ('Joao Felix', 'Portugal', 25, 2, 0, 0, 0, 0),
 ('Alex Grimaldo', 'Spain', 29, 2, 0, 0, 0, 0),
 ('Goncalo Inacio', 'Portugal', 23, 2, 0, 0, 0, 0),
 ('Aymeric Laporte', 'Spain', 30, 6, 0, 0, 0, 0),
 ('Alvaro Morata', 'Spain', 32, 7, 1, 1, 2, 0),
 ('Matheus Nunes', 'Portugal', 26, 2, 0, 0, 0, 0),
 ('Jordan Pickford', 'England', 30, 7, 0, 0, 0, 0),
 ('Fabian Ruiz Pena', 'Spain', 28, 6, 2, 2, 1, 0),
 ('Unai Simon', 'Spain', 27, 6, 0, 0, 1, 0),
 ('Dayot Upamecano', 'France', 26, 6, 0, 0, 0, 0),
 ('Martin Zubimendi', 'Spain', 25, 4, 0, 0, 0, 0),
 ('Erik Janza', 'Slovenia', 31, 3, 1, 0, 2, 0)]
>>> tocke(seznam, 10, 5)
[('Victor Bernth Kristiansen', 5),
 ('Diogo Costa', 0),
 ('Joao Felix', 0),
 ('Alex Grimaldo', 0),
 ('Goncalo Inacio', 0),
 ('Aymeric Laporte', 0),
 ('Alvaro Morata', 5),
 ('Matheus Nunes', 0),
 ('Jordan Pickford', 0),
 ('Fabian Ruiz Pena', 25),
 ('Unai Simon', -5),
 ('Dayot Upamecano', 0),
 ('Martin Zubimendi', 0),
 ('Erik Janza', 0)]
```

Rešitev

```
def tocke(seznam, tocke_goli, tocke_asist):
    s = []

    for ime, _, _, _, goli, asistence, rumeni, rdeci in seznam:
        tocke1 = (goli - rdeci) * tocke_goli + (asistence - rumeni) *
        ↪  tocke_asist
        if tocke1 >= 0:
            s.append((ime, tocke1))

    return s
```